



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

CyberSecurity

3 - “Basic Cryptographic Concepts” *[Chapter 2]*

Gabriele D'Angelo <g.dangelo@unibo.it>

<https://www.unibo.it/sitoweb/g.dangelo/en>

Outline



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Scope** of Cryptography
- **Basic** Operations
- **Cipher** and **Key**
- Example: **Caesar Cipher**
- **Symmetric** Key Encryption
- Modes of Operation
- **Stream** Encryption
- **Secure Hash** Functions
- **Public-Key** Encryption
- Digital Signatures
- Public-Key **Certificates**
- **Random Numbers** Generation
- **Post Quantum** Encryption (NOT IN THE BOOK: study using slides)



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Cryptography (intro)

Scope of Cryptography

Different Cybersecurity Properties



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Different encryption tool will provide different properties (not all, only a subset)

The usage of **cryptographic techniques** is necessary for:

Most common property when we use Cryptography

- **Confidentiality**: “*only the right people **can read** the information*”
- **Authentication**: “*the ability to verify the **sender's identity***” (example, email can be forged: so no Auth property)
- **Integrity**: “*protecting information from **being modified** by **unauthorized parties***” (example, email content can be modified: so no Integrity property)
- **Anonymity**: “*ensuring the **users' anonymity** while communicating*”

Basic Operations



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Encryption**: the process of converting **plaintext** into **ciphertext**
- **Decryption**: the process of converting **ciphertext** into **plaintext**

Basic Operations



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Encryption**: the process of converting **plaintext** into **ciphertext**
- **Decryption**: the process of converting **ciphertext** into **plaintext**

Can be read

Basic Operations



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Encryption**: the process of converting **plaintext** into **ciphertext**
- **Decryption**: the process of converting **ciphertext** into **plaintext**

Can not be read

(not readable by attacker: for example, in a MITM attack, if the packets are encrypted the attacker cannot read it)

Basic Operations



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Encryption**: the process of converting **plaintext** into **ciphertext**
- **Decryption**: the process of converting **ciphertext** into **plaintext**

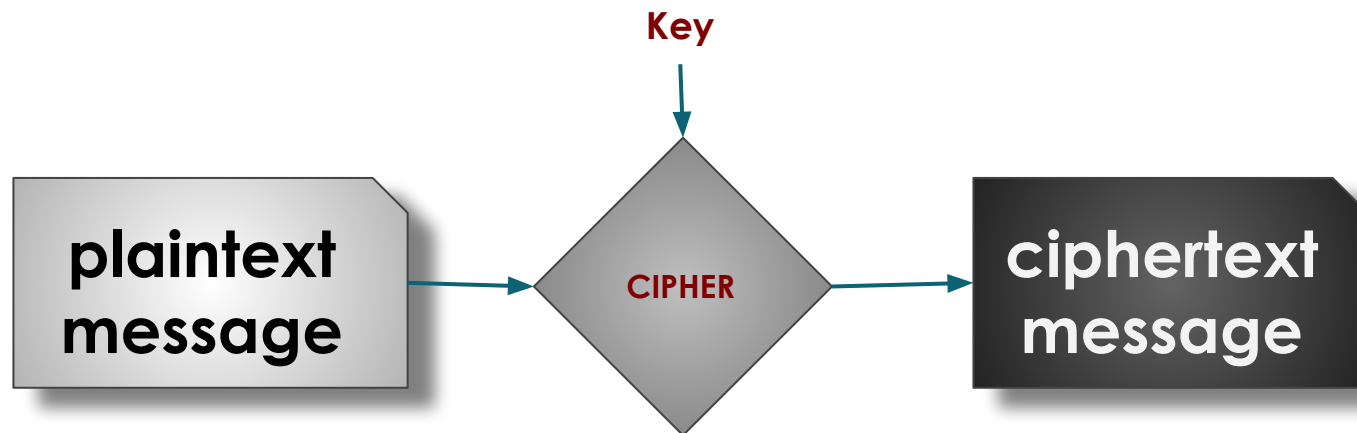


Cipher and Key



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Cipher**: an **algorithm** used to **encrypt** and **decrypt** data (must be secure and efficient)
- **Key**: a **piece of information** used by the cipher to encrypt the plaintext (and that is **necessary to decrypt the ciphertext**)



Example: Caesar Cipher



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- A very old example of cipher (i.e. Gaius Julius Caesar) Only letters are encrypted (not numbers and special character like '!' and '?')
- It is a substitution cipher in which each character in the message is substituted by another character
- **For example:** “*substitute the letters in the second row for the letters in the top row to encrypt a message*”

A	B	C	<u>D</u>	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
			↓+3																						
<u>D</u>	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C

Example: Caesar Cipher



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

The letters in the second
row are shifted of a given
number of positions

er (i.e. Gaius Julius Caesar)

which **each character in the message is**
character

the letters in the second row for the letters in
the top row to encrypt a message"

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C

Example: Caesar Cipher



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

The letters in the second row are **shifted** of a **given number of positions**

In this example the letters are shifted of 3 positions

...p row to encrypt a message

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C

Example: Caesar Cipher



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

The letters in the second row are **shifted** of a **given number of positions**

In this example the letters are shifted of **3 positions**
("3" is the secret key!)

...p row to encrypt a message

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C

The "Big Enough" threshold in modern cryptography is generally considered to be 2^{128} or higher to resist brute-force attacks from modern hardware. The Caesar Cipher's keyspace doesn't just fall short; it's practically non-existent by comparison.

- Must be BIG ENOUGH to be difficult to evaluate all keys to find the correct one from the POV of the attacker (BIG ENOUGH depends on Computer power)

possible in the

$n-1$ if n is the number of letters in the Alphabet

A B n-1 if n is the number of letters in the Alphabet M N O P Q R S T U V W X Y Z

D E F G H I J K L M N O P Q R S T U V W X Y Z A B C

Example: Caesar Cipher



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- A very old example of cipher (i.e. Julius Caesar)
- It is a simple cipher which **each character in the message is**

How many keys are

possible in the

Caesar Cipher?

Is it secure?

No because it has a short number of keys

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
D E F G H I J K L M N O P Q R S T U V W X Y Z A B C

Example: Caesar Cipher



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- A very old example of cipher (i.e. Julius Caesar)
- It is a simple cipher which each character in the message is

shifted by a certain number of letters

- For example, if the shift is 3, then the letter 'A' becomes 'D', 'B' becomes 'E', and so on.

How many keys are possible in the Caesar Cipher?

Is it secure?

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

Let's play with it!



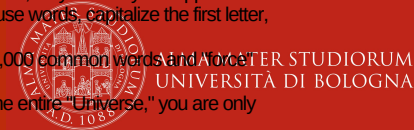
There is a third type of attack that is the Combination of Brute Force and Cryptanalytic Attack. If a cipher has a massive universe of keys, Brute Force is impossible. However, if Cryptanalysis can reveal even a tiny bit of information about the key, the math changes instantly (Reducing the Search Space).

Caesar Cipher: Attacks

Example: Dictionary & Rule-Based Attacks

When attacking passwords, hackers don't just try random characters (aaaa, aaab, aaac). That is pure Brute Force and it's too slow. Instead, they use a hybrid approach:

- Cryptanalysis (Pattern Study): They know humans use words, capitalize the first letter, and put "123" at the end.
- Brute Force (The Execution): They take a list of 100,000 common words and ~~only~~ only the variations that fit the human patterns.
- Why this is powerful: You are no longer searching the entire "Universe," you are only searching the "Likely Universe."



(The two types of attack are used to attack all the types of ciphers, not only the Caesar one)

How is possible to **attack** the Caesar Cipher? (i.e. **to get the cleartext without having the secret key**)

It is normal that if i steal the key i can get the clear text (we are attacking the Cipher, not the device). If we get the key, is not a failure of the Cipher, but a failure of the security of the device

In Modern Ciphers, is difficult to do Brute Force Attacks (universe of keys and time required are too large)

Brute Force attack: **we can try all the possible keys**

- *is it ^{fattibile} feasible? It depends on the **number of keys** (i.e. **key size**) and the **time required to check each key** (... *computers are fast!*)*

The system designer selects an algorithm with a sufficiently large key space so that, given current computing power and its expected future development, the time required to test every key renders the attack impractical for the entire period during which the information must remain confidential

if the Brute Force attack is difficult to do or costly, we do this

Cryptanalytic attack: (in this case) **we can use the frequency of characters** (or other **weaknesses** of think kind)

that are in the Design of the Cipher

- *in human languages **each letter** has a **different frequency of use** the Caesar Cipher translates letters and therefore **it translates also their frequency***

This is an example of how you can exploit vulnerabilities in the Design of the Cipher

The Caesar Cipher provides zero confusion. It hides the letters, but it leaks the pattern of the language perfectly. This means an attacker doesn't even need to try all the keys

For example, in English the letter E is the most used. So, if we see that a letter of the cipher text has the same frequency, that letter can be the E (so, we can find the key from the pattern of the language)

50% is not the probability of guessing correctly on the first try, but the probability that the key lies in the first half of your search. Since the key is chosen at random, there is no reason why it should necessarily be in the second half rather than the first. It's a coin toss: it's either in the first half or the second. So, you need to have half of the universe large enough

Caesar Cipher: Attacks

On average half of
all possible keys
must be tried to
achieve success

How is possible to **attack** the Caesar Cipher
without having the secret key)

- **Brute Force** attack: **we can try all the possible keys**
 - *is it feasible?* It depends on the **number of keys** (i.e. **key size**) and the **time required to check each key** (... computers are fast!)
- **Cryptanalytic** attack: (in this case) we can use **the frequency of characters** (or other **weaknesses** of think kind)
 - in human languages **each letter** has a **different frequency of use**
 - the Caesar Cipher translates letters and therefore **it translates also their frequency**

Caesar Cipher: Attacks



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

How is possible to **attack** the Caesar Cipher? (*i.e. to get the cleartext without having the secret key*)

- **Brute Force** attack: **we can try all the possible keys**
 - *is it feasible?* It depends on the **number of keys** (*i.e. key size*) and the **time required to check each key** (... computers are fast!)
- **Cryptanalytic** attack: (in this case) we can use **the frequency of characters** (or other **weaknesses** of think kind)
 - in human languages **each letter** has a **different frequency of use**
 - the Caesar Cipher translates letters and therefore **it translates also their frequency**

Caesar Cipher: Attacks



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

How is possible to **attack** the C
without having the secret key)

This can be exploited
by the attacker!

How?

- **Brute Force** attack: we can
 - *is it feasible?* It depends on the *time required*
to check each key (... computer power)
- **Cryptanalytic** attack: (in this case) we can use *frequency of*
characters (or other **weaknesses** of think kind)
 - in human languages **each letter** has a **different frequency of use**
 - the Caesar Cipher translates letters and therefore **it translates also their frequency**

In crittografia, questo significa che l'algoritmo non fornisce confusione: la struttura statistica del messaggio originale (il plaintext) rimane perfettamente visibile nel messaggio cifrato. DETTO NELLE SLIDES PRECEDENTI



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Symmetric Encryption

Symmetric Key Encryption



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



Isn't obsolete.

- The universal technique for providing **confidentiality** for **transmitted** or **stored data**
- Two **requirements** for **secure use**:
 - a **strong encryption algorithm** against the three type of attacks that we saw
 - **sender** and **receiver** must have **obtained copies of the secret key**
in a secure fashion and **must keep the key secure**

Symmetric Key Enc

Use Cipher Algorithms that are standards

Please remember the **Kerckhoffs's principle**:

“A cryptosystem should be secure even if everything about the system, except the key, is public knowledge”

- The universal technique for secure use of symmetric key encryption is to use a **secret key** to encrypt and decrypt the **stored data**
- Two **requirements** for **secure use**
 - a **strong encryption algorithm**
 - **sender** and **receiver** must have **obtained copies of the secret key in a secure fashion** and **must keep the key secure**

Symmetric Key Encryption



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- The universal technique for providing **confidentiality** for **transmitted** or **stored data**
- Two **requirements** for **secure use**:
 - a **strong encryption algorithm**
 - **sender** and **receiver** must have **obtained copies of the secret key**
in a secure fashion and **must keep the key secure**

Symmetric Key Encryption



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

This is quite ridiculous: “to get a
secure communication channel
you need a secure channel”

to exchange the key to be used to encrypt/decrypt the message (to create a secure channel, i need a secure channel:
BIG LIMITATION, is not suited to be used in the internet because if the destination is far away it is difficult to find a
secure way to exchange key (Internet is insecure by nature, so attacker can get the key if they are listening))

confidentiality for transmitted or

- sender and receiver must have obtained copies of the secret key
in a secure fashion and must keep the key secure

Symmetric Key Encryption



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- The universal technique for **securely storing** **stored data**

- Two **requirements** for **secret communication**

- need a **strong encryption**

- **sender** and **receiver** must have **obtainable copies of the secret key**

in a secure fashion and **must keep the key secure**

We don't know if the key is leaked or not

The confidentiality of
communications depends on the
secrecy of the key

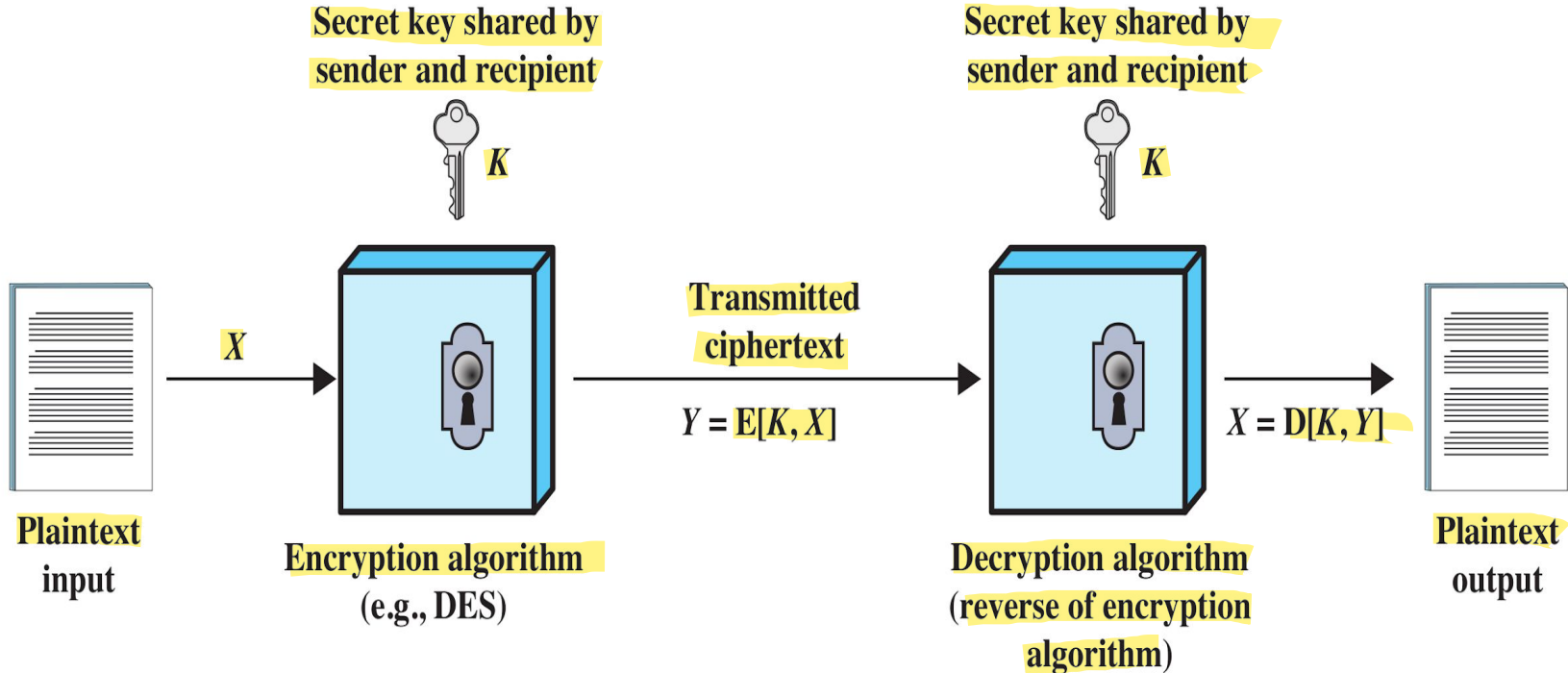
I have to trust even the destination because if it is malicious can give the key to others to encrypt/decrypt so there is a compromise of the communication channel

Simplified Model of Symmetric Encryption



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

B
O
B



A
L
I
C
E

Simplified Model of **Symmetric** Encryption



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Secret message to be sent by

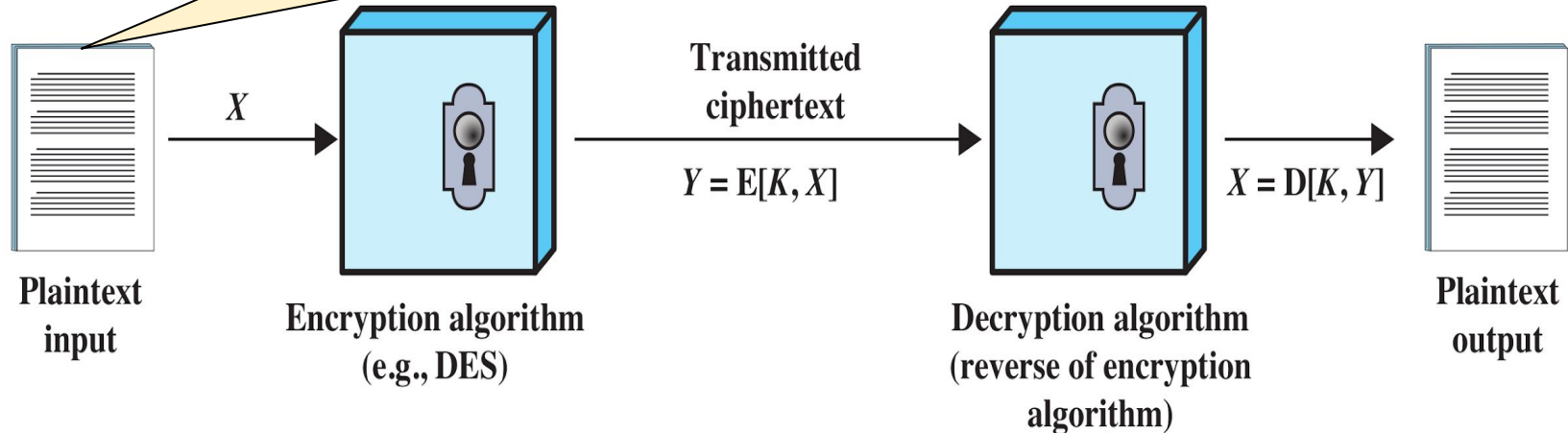
Bob to Alice. Bob wants

confidentiality

Secret key shared by
sender and recipient



A
L
I
C
E



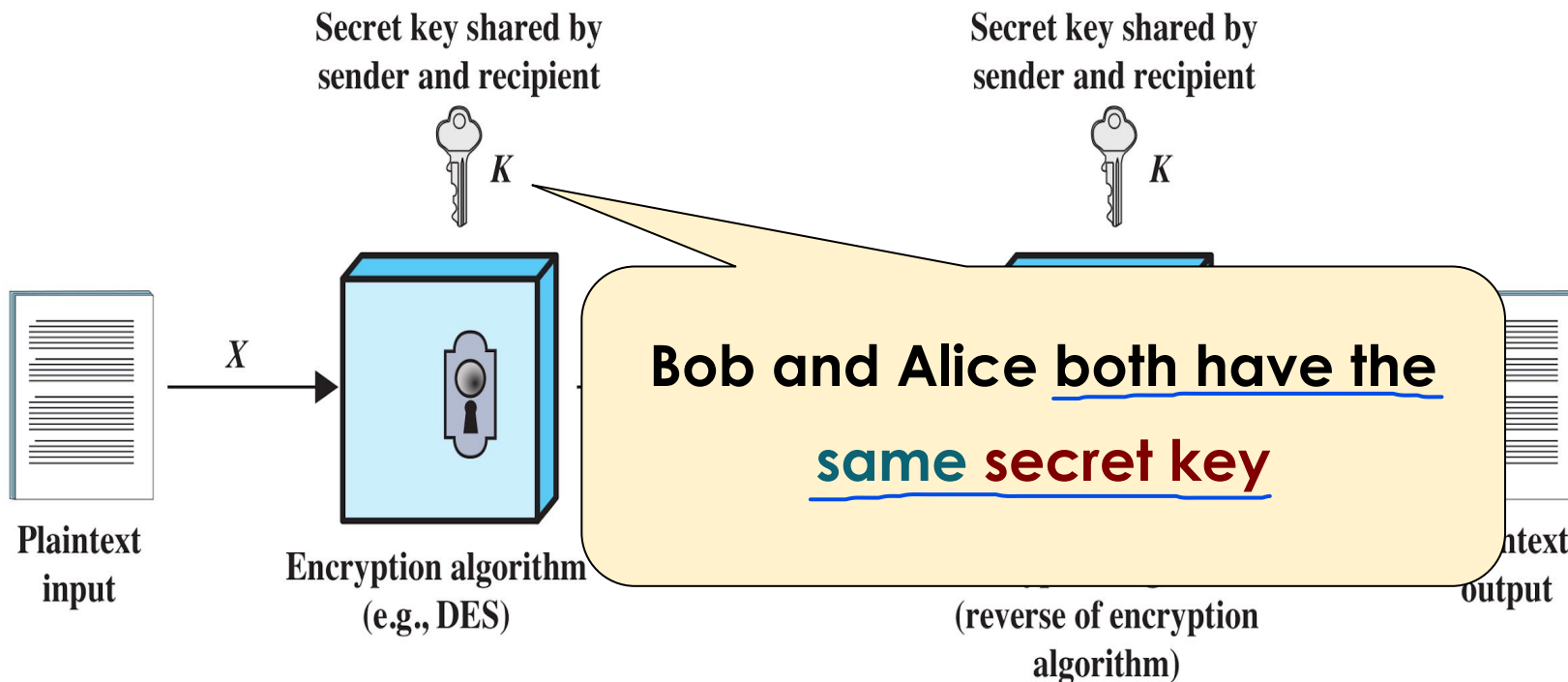
Simplified Model of Symmetric Encryption



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

A
L
I
C
E

B
O
B

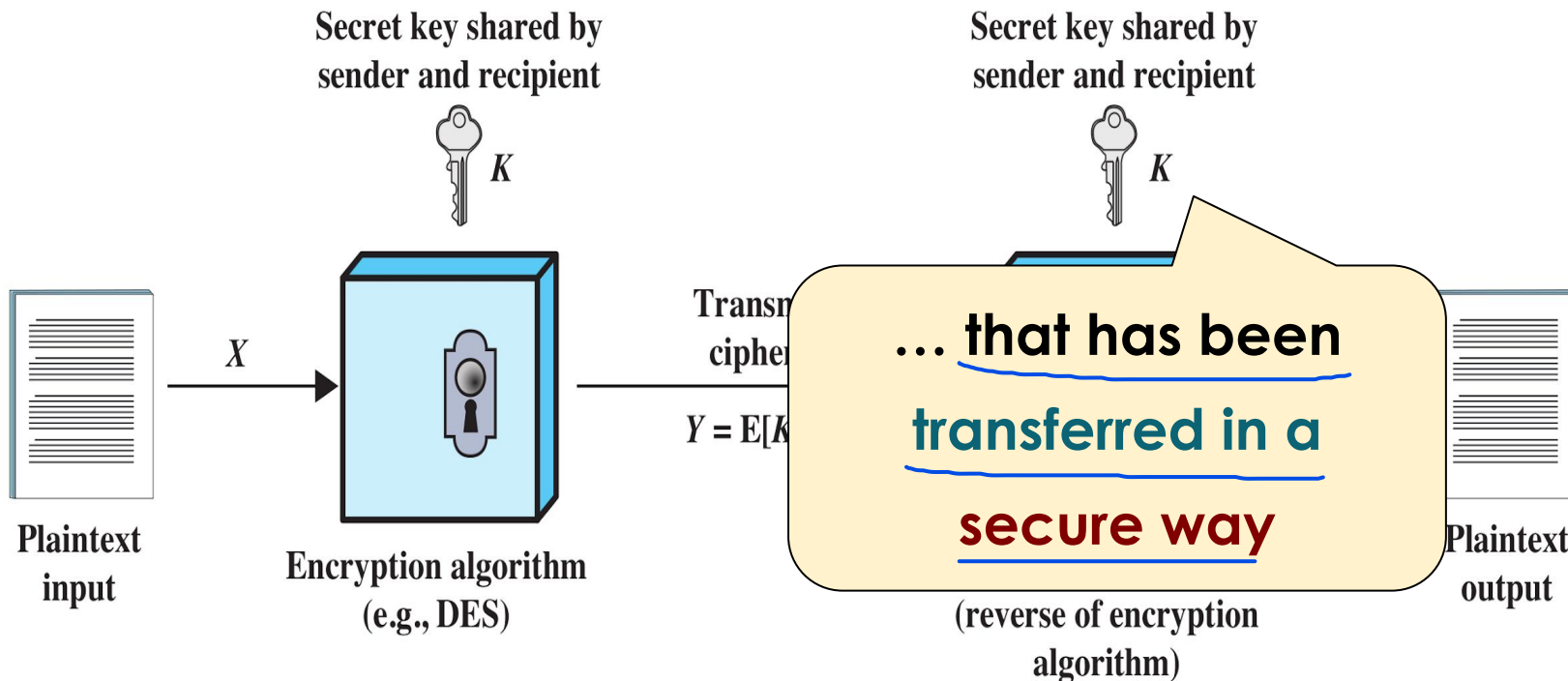


Simplified Model of Symmetric Encryption



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

B
O
B



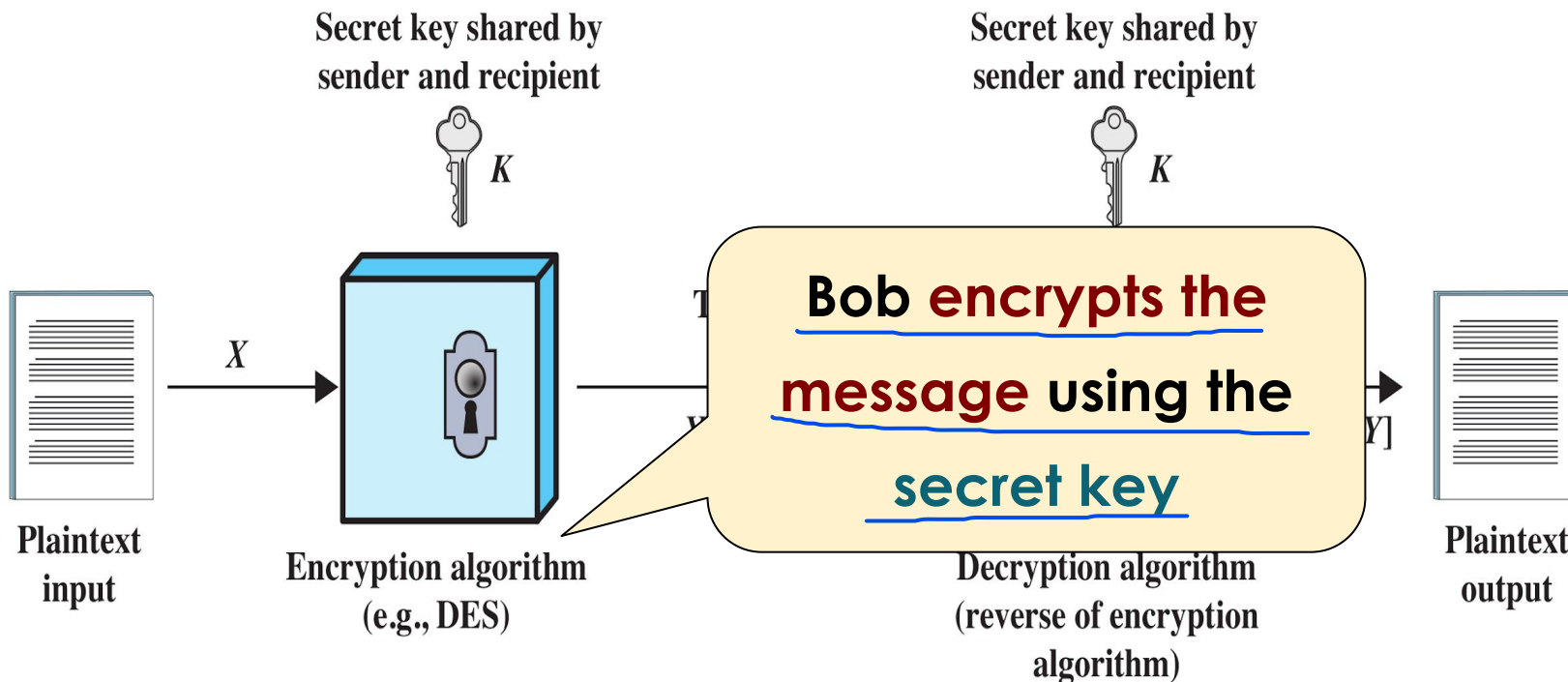
A
L
I
C
E

Simplified Model of Symmetric Encryption



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

**B
O
B**



**A
L
I
C
E**

Simplified Model of Symmetric Encryption

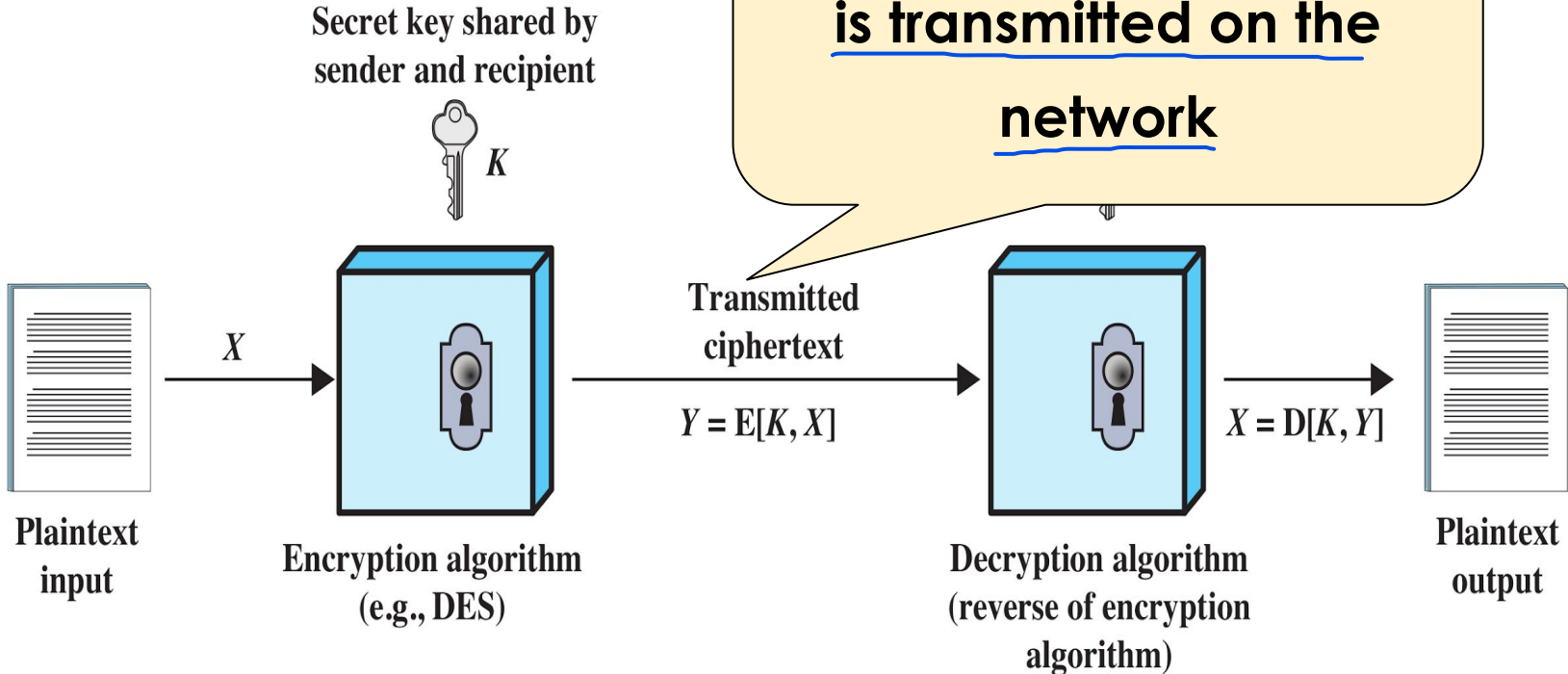


ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

The encrypted message
is transmitted on the
network

A
L
I
C
E

B
O
B



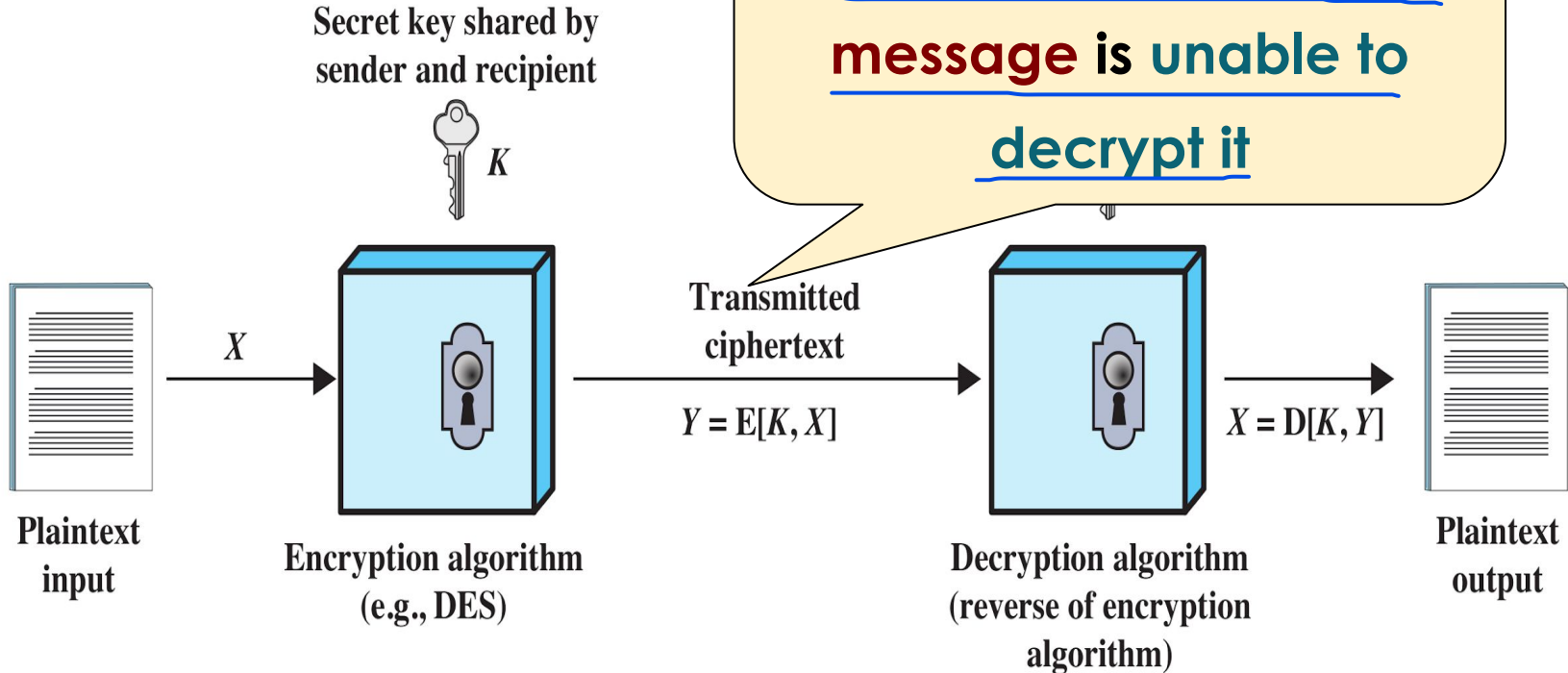
Simplified Model of Symmetric Encryption



MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

A
L
I
C
E

An attacker that
intercepts the encrypted
message is unable to
decrypt it

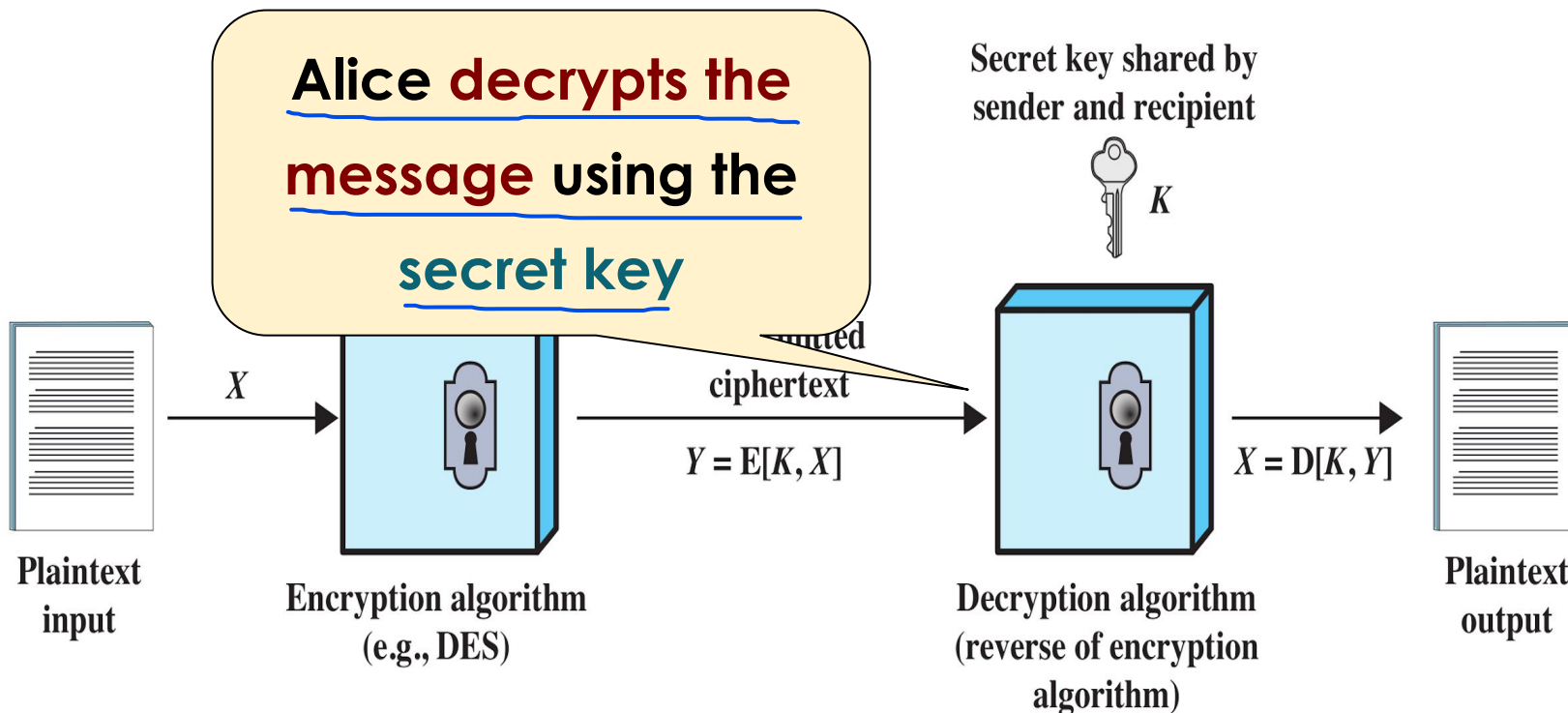


Simplified Model of Symmetric Encryption



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

B
O
B



A
L
I
C
E

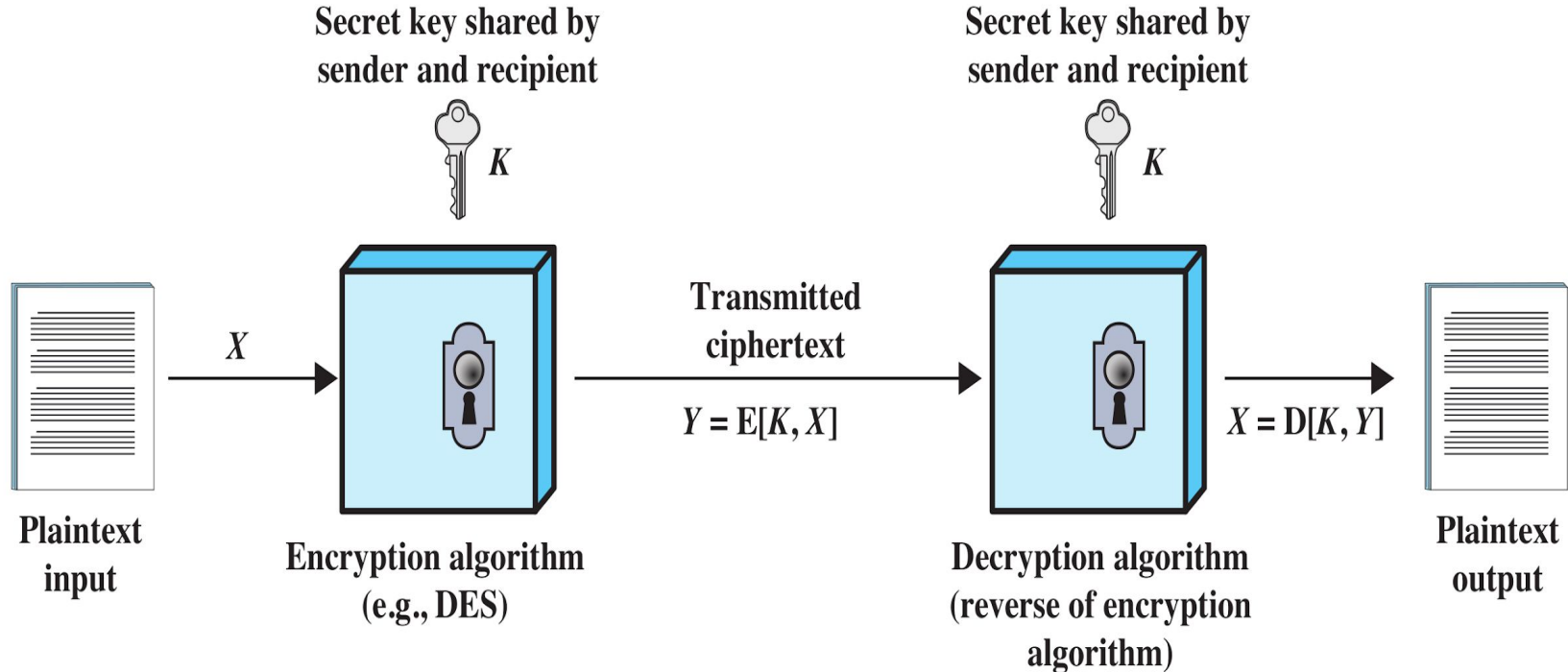
Simplified Model of Symmetric Encryption



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

**A
L
I
C
E**

**B
O
B**



If a key is random, an attacker has zero information before they start. Every key in the universe (N) is equally likely. This forces the attacker into the "Worst Case" scenario: a slow, linear Brute Force attack. If the key isn't random (e.g., it's a word from a dictionary, a date of birth, or a common pattern), the attacker can use a Cryptanalytic Attack to bypass the "Universe of Keys."



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Symmetric Encryption: Attacks

The goal of Cipher Designers is to make this type of attack as costly as possible (if it is very costly, the attacker will not pursue this approach)

● **Brute Force** attack: **try all the possible keys**

The assumption that a key is random is fundamental because:

- It is the only way to guarantee that the attacker must perform a full Brute Force.
- It allows us to use Probability and Statistics to prove how long the data will stay safe.
- It prevents Dictionary Attacks and Pattern Recognition from providing a shortcut.

(fundamental assumption)

○ again, it is about the **key size** (if the key is **randomic** and **not a dictionary word**) and how fast is our computer (or cluster of computers)

when build a Cipher we take into consideration computer speed of today and future computers to assess the strongness of a Cipher

● Attack using vulnerabilities caused by weak design of Ciphers

● **Cryptanalytic** attack: if the cipher is **weak** then it could be possible to find ways to decrypt the ciphertexts (i.e. WW2 Enigma Machine)

● If the attack is **successful** then **all future and past messages** encrypted with that key are **compromised** (it is better to change frequently the key to avoid compromission (we don't know if the communication channel is compromised))



Before trying all the possible combinations of characters, I would try with all the words in dictionaries

- again, it is about the **key size** (if the key is **randomic** and **not a dictionary word**) and how fast is our computer (or cluster of computers)
- **Cryptanalytic** attack: if the cipher is **weak** then it could be possible to find ways to decrypt the ciphertexts (i.e. WW2 Enigma Machine)
- If the attack is **successful** then **all future and past messages** encrypted with that key are **compromised**



Before trying all the possible
combinations of characters, I would
try with all the words in **dictionaries**

- again, it is about the **key size** (if the key is **randomic** and **not a dictionary**

What is a

dictionary?

- If the cipher is **weak** then it could be possible to find ways to decrypt the ciphertexts (i.e. WW2 Enigma Machine)
- If the attack is **successful** then **all future and past messages** encrypted with that key are **compromised**



Before trying all the possible combinations of characters, I would try with all the words in dictionaries

- again, it is about the **key size** (if the key is **randomic** and **not a dictionary**

What is a
dictionary?

- If the cipher is **weak** then it could be possible to find ways to decrypt the ciphertext (i.e. WW2 Enigma Machine)

What is the best
dictionary?

- If the attacker has access to **future and past messages** encrypted with the same key...

Symmetric Encryption: Attacks



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Brute Force** attack: **try all the possible keys**
 - *again, it is about the **key size** (if the key is **randomic** and **not a dictionary word**) and how fast is our computer (or cluster of computers)*
- **Cryptanalytic** attack: if the cipher is **weak** then it could be possible to find ways to decrypt the ciphertexts (i.e. WW2 Enigma Machine)
- If the attack is **successful** then **all future and past messages** encrypted with that key are **compromised**



Symmetric Encryption: Attacks

Depends on the computer power (can be general or specialized hardware)

Key size (bits)	Number of alternative keys	Time required at 1 decryption/ μ s	Time required at 10 ⁶ decryptions/ μ s
32	$2^{32} = 4.3 \times 10^9$	35.8 minutes	2.15 milliseconds
DES Key size 56	$2^{56} = 7.2 \times 10^{16}$	1142 years	10.01 hours
AES Key Size 128	$2^{128} = 3.4 \times 10^{38}$	5.4×10^{24} years	5.4×10^{18} years
3DES Key Size 168	$2^{168} = 3.7 \times 10^{50}$	5.9×10^{36} years	5.9×10^{30} years
26 characters (permutation)	$26! = 4 \times 10^{26}$	6.4×10^{12} years	6.4×10^6 years

Average time required for exhaustive key search (i.e., Brute-Force)

If you find a weakness in the cipher, all the messages that use this cipher are compromised. In case of brute forcing, only the messages that use that key are compromised, if you find it (so the cipher is not broken and if they change the key, the attackers need to do again the brute force attack)

Symmetric Encryption: algorithms



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- Many different **symmetric encryption algorithms** have been proposed

- It is better to use algorithms that are an **international standard**

(Considered only standards and not the one published by researched and not defined as standards)

- DES** (Data Encryption Standard, 1977):

- key size = 56 bits

- It is **slow** and **very insecure** (the key size is too small)

(it is slow using Software, so they introduced hardware to perform the same task fastly (HW acc))

- 3DES**: "execute 3 times the standard DES"

More slow than DES (3x) but more secure

Standard of Symmetric Cipher

- it is **very slow** but quite secure (key size = 56×3)

Key size was increased without changing the encryption/decryption algorithm (so it is possible to use the same hardware accelerators of the DES for the 3DES: in order to avoid exorbitant costs when switching to a new secure symmetric encryption algorithm, as DES was no longer secure)

- AES** (Advanced Encryption Standard, 2002):

- key size = {128, 192 and 256} bits

You can select the key size (this makes brute-force attacks more difficult and encryption will be slower as the key size

- very **fast** and **secure** (now... in the future?)

It is designed to be extremely fast in terms of both hardware and software

Performing the same encryption operation three times with two or three different keys (message encrypted with the first, then second and then third key (sequential pipeline))

So, with 3DES, they resolved both the cost issue (by keeping the same hardware accelerators) and the security issue (by increasing the key size)

Symmetric Enc

Likely the most studied encryption algorithm: **some cryptanalytic issues but no relevant weaknesses**

- Many different
- It is better to use algo
- **DES** (Data Encryption Standard, 1977):
 - **key size** = 56 bits
 - It is **slow** and **very insecure** (the key size is too small)
- **3DES**: "execute 3 times the standard DES"
 - it is **very slow** but quite secure (key size = 56×3)
- **AES** (Advanced Encryption Standard, 2002):
 - **key size** = {128, 192 and 256} bits
 - very **fast** and **secure** (**now**... in the future?)

Symmetric Enc

Likely the most studied encryption
(few minor problems: not so important)
algorithm: some cryptanalytic issues but
no relevant weaknesses of the cipher

- Many different
- It is better to use algo
- **DES** (Data Encryption Standard, 1977):
 - key size = 56 bits
 - It is **slow** and **very insecure** (the key size is too small)
- **3DES**: "execute 3 times the standard DES"

In 1998, Electronic Frontier Foundation (EFF) announced that
it had broken a DES in < 3 days with specialized hardware

Symmetric Enc



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

56 bits (!?)

- Many different **sym** have been proposed
- It is better to use algo **international standard**
- **DES** (Data Encryption Standard, 1977):
 - **key size** = 56 bits
 - It is **slow** and **very insecure** (the key size is too small)
- **3DES**: "execute 3 times the standard DES"
 - it is **very slow** but quite secure (key size = 56*3)
- **AES** (Advanced Encryption Standard, 2002):
 - **key size** = {128, 192 and 256} bits
 - very **fast** and **secure** (**now**... in the future?)

Symmetric Enc



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Why not 64 bits?

56 bits (?!?)

- Many different **sym** have been proposed
- It is better to use algo **ional standard**
- **DES** (Data Encryption Standard, 1977):
 - key size = 56 bits
 - It is **slow** and **very insecure** (the key size is too small)
- **3DES**: "execute 3 times the st"
 - it is **very slow** but quite sec
- **AES** (Advanced Encryption)
 - key size = {128, 192 and 256}
 - very **fast** and **secure** (now)

It is not a power
of 2 :-()

Symmetric Encryption: algorithms



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Explanation for why 56 instead of 64 (or a 2-power number)

- Many different **symmetric**
- It is better to use algorithm
- **DES** (Data Encryption Standard)
 - **key size** = 56 bits
 - It is **slow** and **very insecure**
- **3DES**: "execute 3 times the DES"
 - it is **very slow** but quite secure
- **AES** (Advanced Encryption Standard)
 - **key size** = {128, 192 and 256 bits}
 - very **fast** and **secure** (**now**... in the future?)

"NSA worked closely with IBM to strengthen the algorithm against all **except brute force attacks** and to strengthen substitution tables, called S-boxes. Conversely, NSA tried to convince IBM to **reduce the length of the key from 64 to 48 bits**. Ultimately they compromised on a **56-bit key**"

it is a Political Decision, not technical one

Thomas R. Johnson

Symmetric Encryption: algorithms



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- Many different **symmetric encryption algorithms** have been proposed
- It is better to use algorithms that are an **international standard**
- **DES** (Data Encryption Standard, 1977):
 - **key size** = 56 bits
 - It is **slow** and **very insecure** (the key size is too small)
- **3DES**: "execute 3 times the standard DES"
 - it is **very slow** but quite secure (key size = 56×3)
- **AES** (Advanced Encryption Standard, 2002):
 - **key size** = {128, 192 and 256} bits
 - very **fast** and **secure** (**now**... in the future?)

Symmetric Encryption: algorithms



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- Many different **symm**
- It is better to use algo
- **DES** (Data Encryption)
 - **key size** = 56 bits
 - It is **slow** and **very insecure** (key size is too small)
- **3DES**: "execute 3 times the standard DES"
 - it is **very slow** but quite secure (key size = 56×3)
- **AES** (Advanced Encryption Standard, 2002):
 - **key size** = {128, 192 and 256} bits
 - very **fast** and **secure** (now... in the future?)

The same hardware
accelerators used for DES can
still be used with 3DES

To overcome the speed limitations of general-purpose processors (CPUs), dedicated hardware accelerators known as ASICs (Application-Specific Integrated Circuits) were developed for DES. An ASIC is a chip designed to perform a single task, but at extreme speed. In the case of DES, all the operations of the algorithm are 'hard-wired' into the silicon of the ASIC. Since the hardware does not have to interpret software instructions, it can perform an entire encryption cycle in just a few clock cycles. Hundreds of 'DES units' can be incorporated onto a single ASIC chip, all working simultaneously. 3DES is not a new algorithm, but the application of DES three times in succession using different (or partially different) keys.

Symmetric Encryption: algorithms



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- Many different **symmetric** algorithms have been proposed
- It is better to use algorithms that are **fast** and **secure**
- **DES** (Data Encryption Standard, 1976)
 - **key size** = 56 bits
 - It is **slow** and **very insecure** (the **key size** is too small)
- **3DES**: "execute 3 times the standard DES"
 - it is **very slow** but quite secure (key size = 56×3)
- **AES** (Advanced Encryption Standard, 2002):
 - **key size** = {128, 192 and 256} bits
 - very **fast** and **secure** (**now**... in the future?)

The **block size** is **too small**
(see next slides)

It's important to remember that as the key size increases, the number of mathematical rounds (computation time and energy) increases. Each round takes time and energy. If you are a bank processing billions of transactions per second, choosing AES-256 over AES-128 might actually slow down your servers or increase your electricity costs significantly without providing a "noticeable" increase in actual safety against today's attackers.



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Symmetric Encryption: algorithms

These algorithms are unable to encrypt the entire input data if it exceeds the specified limit; the data must therefore be divided into several blocks. Of course, there are algorithms that do not use a block-based approach and therefore take the input and encrypt it directly

	DES	Triple DES	AES
<u>Plaintext block size (bits)</u>	64	64	128
<u>Ciphertext block size (bits)</u>	64	64	128
<u>Key size (bits)</u>	56	2x or 3x the DES Key size 112 or 168	128, 192, or 256

In the DES and 3DES, the only change is on the key size not in the block size of the input

Performance will be higher if the block size is bigger (AES has a better performance than DES and 3DES: not only for block size)

The choice of AES key size is a strategic decision based on the specific threat model, performance requirements, and data longevity (The decision is almost always a balance between Performance (Speed/Power) and Security (Longevity/Threat Resistance)). For example, in Messaging about not so important data, you can use 128 (because the primary goal is speed), but if you need to encrypt your backups you can use a 256 key (because speed is secondary and requires more security on the data than the first one: need to resist future (Quantum) computers.)

Symmetric Encryption: block ciphers



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **DES, 3DES** and **AES** are **symmetric block ciphers**
- Typically data units (e.g., messages or files) are **larger than a single 64-bit or 128-bit block**
- Possible solution... **Electronic CodeBook (ECB)** mode:
 - (the simplest approach to multiple-block encryption)
 - 1 ○ **divide** the input data in a **sequence of blocks**
 - 2 ○ apply **padding** to the last block (if necessary) (if the block hasn't the length required)
 - 3 ○ each block is **encrypted** using **the same key**
 - (**regularities** in the plaintext could be exploited for **cryptanalysis**)

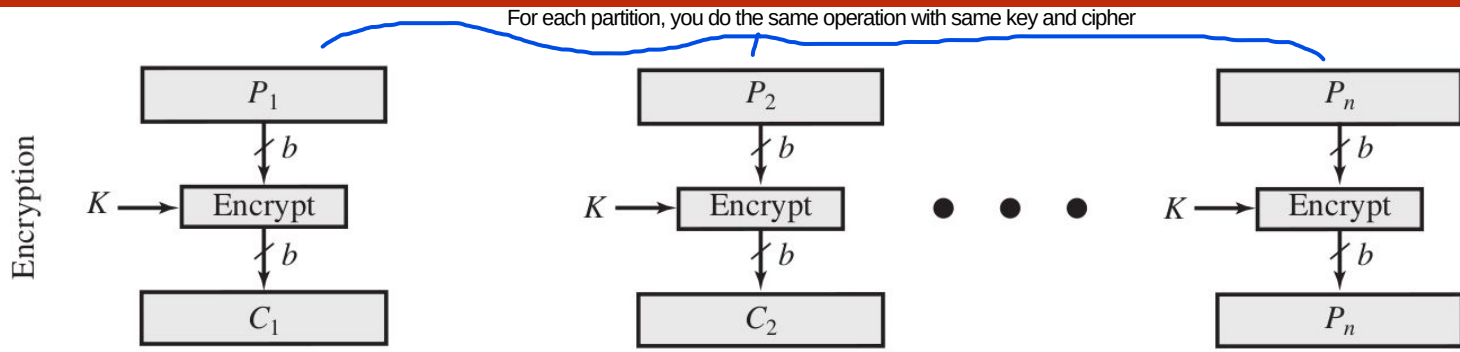
If I am the man-in-the-middle (MITM), even if I do not understand the blocks, I can reorder them and the recipient will not be able to tell that the blocks have been reordered (each block is encrypted in such a way that it does not depend on the others: no dependency between blocks): The ECB does not protect the logical structure of the message, which allows an attacker to manipulate the order of the pieces without the recipient ultimately noticing anything, as the decrypted message appears 'valid', because the order of the message depends only on its entire semantics.

Modes of Operation: ECB

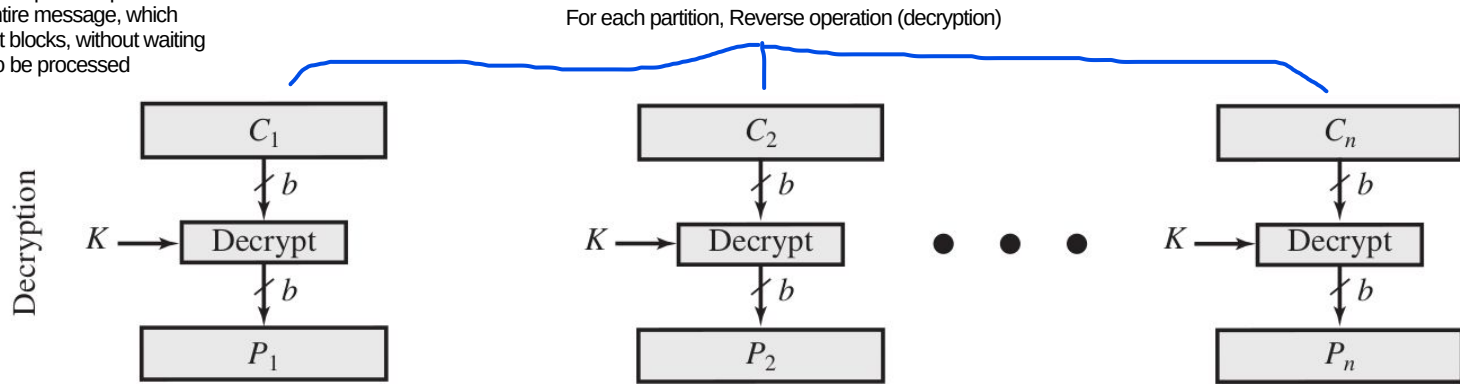
Using OpenSSL, there are different modes of operation to divide a message in blocks: this is the simplest one



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



Another issue is the parallel nature of this approach: attackers can exploit this parallelism to try to decipher the entire message, which consists of independent blocks, without waiting for the previous block to be processed



To mitigate block reordering attacks, modern operating modes use chaining mechanisms to enforce a strict sequence (in which each block k is functionally dependent on the ciphertext of the previous block). Although this ensures structural integrity, it introduces a sequential dependency that eliminates the possibility of parallel processing during encryption/decryption : a significant trade-off in terms of performance compared to the independent nature of the ECB mode (negative thing for each the attacker and user).

Replay attack: (the attackers know there is no dependency between blocks), so the attacker can intercept this block and repeat again the transmission of this block (the attacker knows that the block has important infos but doesn't know its content: because the attacker can't decrypt it). If the implementation of the protocol that is on top has no mitigation for reordering and replay, I have no solution: If your encryption doesn't chain blocks (no structural integrity) AND your top protocol doesn't check for sequence/timing (no protocol integrity), you are completely defenseless. Even the strongest encryption in the world (AES-256) won't save you from a Replay Attack.

Modes of Operation: ECB



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

In cryptography, diffusion is the property where, if you change a single bit of the plaintext, many bits of the ciphertext should change in a seemingly random way. The most critical failure of ECB is that diffusion is strictly contained within each block: If you have a message with 100 blocks and you change one bit in Block 1, only Ciphertext Block 1 changes. Blocks 2 through 100 remain identical. Because ECB lacks diffusion between blocks, it preserves the structure of the data. If your plaintext has a pattern (like a header, a repeated string of zeros, or a specific image format), the ciphertext will show that same pattern. This is why the "ECB Penguin" is the most famous example in crypto: even though the pixels are encrypted, the diffusion of the information is so poor that you can still see the shape of the penguin (next slide you can see the picture).

- Consider the **advantages** vs. **disadvantages** of **ECB** in terms of:
 - **performance** (e.g., encryption throughput ^{and decryption throughput})
 - potential **attacks** (e.g., message **reordering** and **replay** attacks)
 - there is a huge problem of "**diffusion**" (see the next slide)
- Other "more complex" **modes of operation** are available ^{but they have} **(at a cost)**

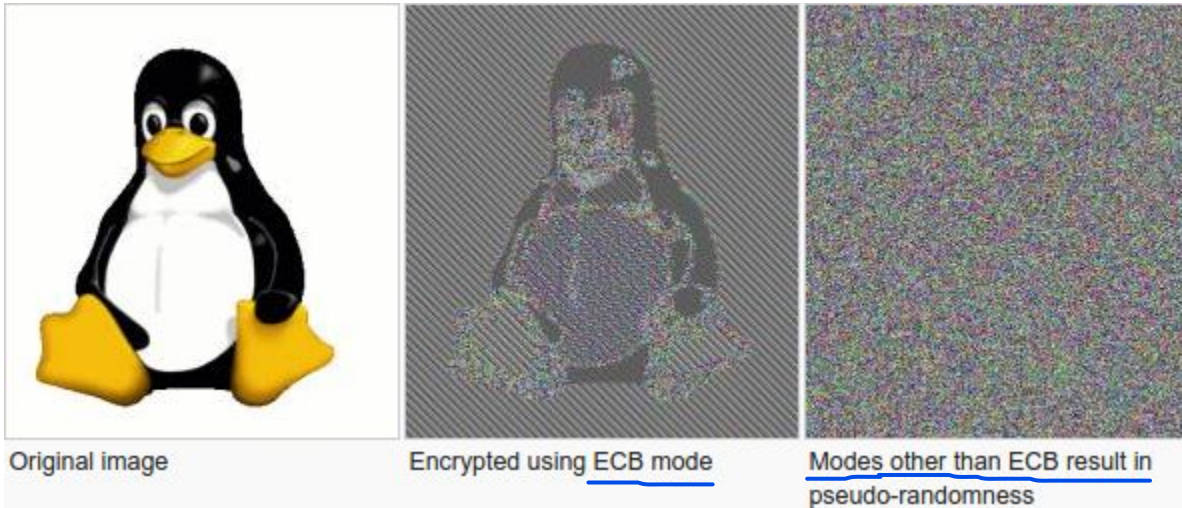
For example, when a dependency is introduced between blocks, the ability to parallelise the computation is lost

Modes of Operation: ECB



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

There is a problem regarding lack of diffusion: what happens is that regularities of the plain text are reported as, more or less, regularities also in the cipher text (leaking a lot of information in the cipher text). No error in the selection of the key and cipher: the problem is the mode of operation (ECB). If you use a different mode of operation (that ensure diffusion), you mitigate this problem (no leaking of information about structure of the plaintext)

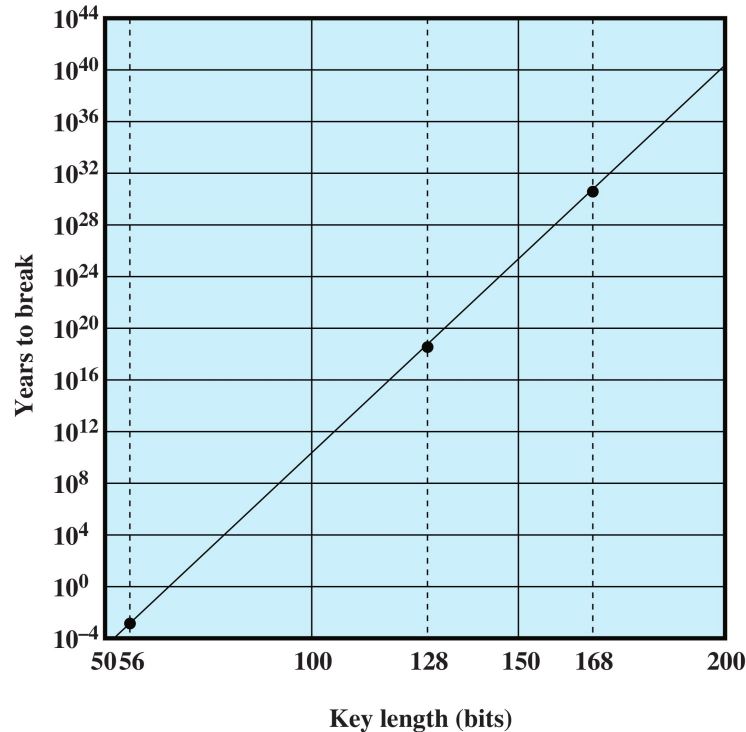


In ECB mode, the encryption process is entirely deterministic. If two plaintext blocks contain the same data, they will produce identical ciphertext blocks because the mode lacks any mechanism for randomization. Although the AES algorithm itself is a robust pseudorandom function, the flaw lies in the mode of operation. This results in a 'leakage of patterns' where the structure of the original data remains visible in the encrypted output.



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Brute-Force



Time To Break a Code

(assuming 10^6 decryptions/ μ s)

Is this assumption realistic?

Check it out!

T2

Symmetric Encryption: **block ciphers**



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Assumption: data produced in fixed blocks.

- **DES**, **3DES** and **AES** are symmetric **block** ciphers ↗

Why block ciphers struggle in a "stream" environment:

1. The Latency Problem (The "Waiting" Factor):
A block cipher is like a factory that only operates when a shipping container is 100% full. If your block size is 128 bits (AES) but your application produces data 8 bits at a time (like a keystroke or a voice sample), the algorithm must wait for the buffer to fill up before it can encrypt anything or to add a lot of padding in respect to the meaningful data. This introduces "Lag" in a VoIP call or a gaming session, waiting for blocks to fill creates noticeable delays that ruin the real-time experience. A lot of computation (adding padding or pseudo-random content) for nothing.

2. Data Expansion (The "Padding" Problem):
Block ciphers must process a full block. If your message isn't a perfect multiple of the block size, you have to add Padding Example: If you want to send a single "Yes" (3 bytes) using a 16-byte block cipher, you have to add 13 bytes of "trash" data just to make the encryption work. Impact: You are wasting bandwidth. If you are sending millions of tiny updates (like GPS coordinates from a fleet of trucks), padding can increase your data costs by 50% or more. A stream cipher, by contrast, results in a ciphertext that is exactly the same size as the plaintext.

How can we deal with data that is
NOT produced in blocks?



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Stream Encryption

this technique is good for lower latency (don't need to wait to full the block to send it) and good usage of bandwidth (only useful data, not padding or metadata)



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Stream Encryption

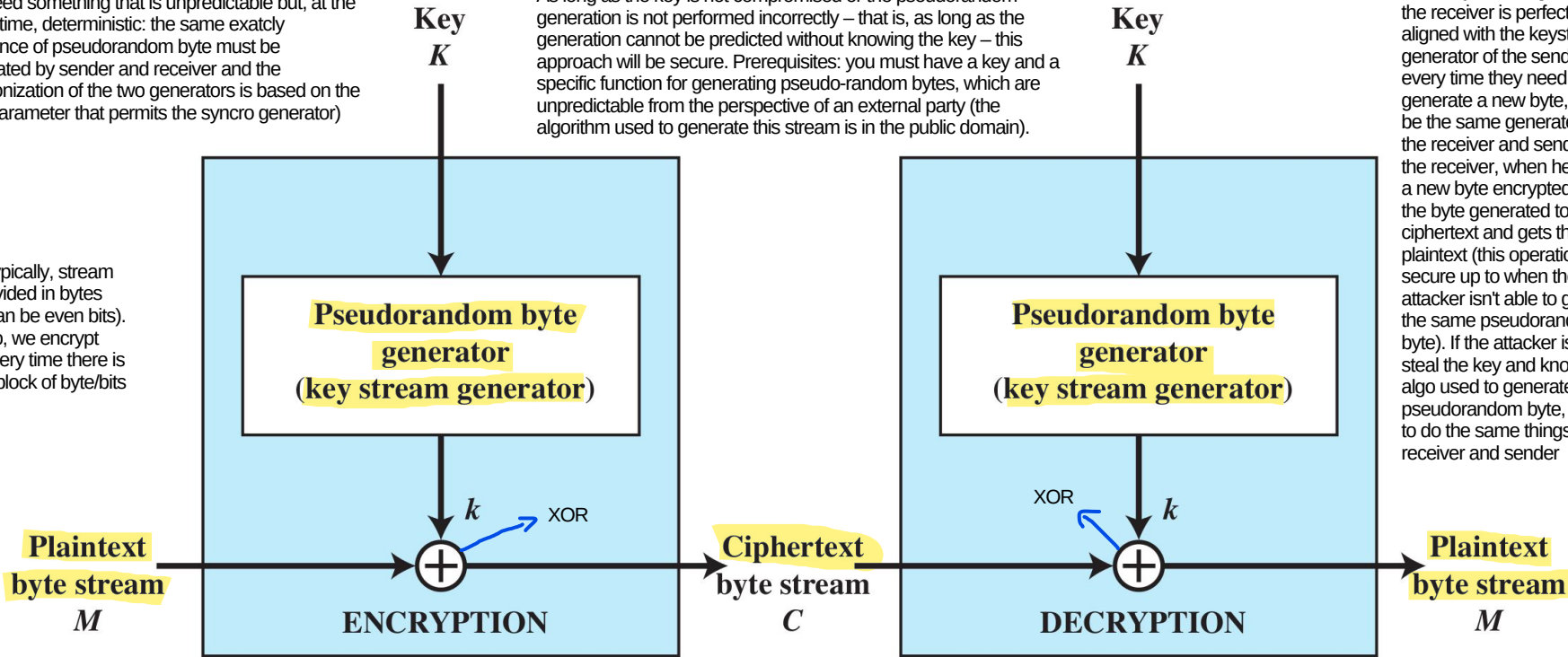
no blocks are required

We need something that is unpredictable but, at the same time, deterministic: the same exactly sequence of pseudorandom byte must be generated by sender and receiver and the synchronization of the two generators is based on the key (parameter that permits the syncro generator)

As long as the key is not compromised or the pseudorandom generation is not performed incorrectly – that is, as long as the generation cannot be predicted without knowing the key – this approach will be secure. Prerequisites: you must have a key and a specific function for generating pseudo-random bytes, which are unpredictable from the perspective of an external party (the algorithm used to generate this stream is in the public domain).

Trick key stream generator of the receiver is perfectly aligned with the keystream generator of the sender: every time they need to generate a new byte, it will be the same generated by the receiver and sender. So, the receiver, when he receive a new byte encrypted, adds the byte generated to the ciphertext and gets the plaintext (this operation is secure up to when the attacker isn't able to generate the same pseudorandom byte). If the attacker is able to generate pseudorandom byte, it is able to do the same things of the receiver and sender

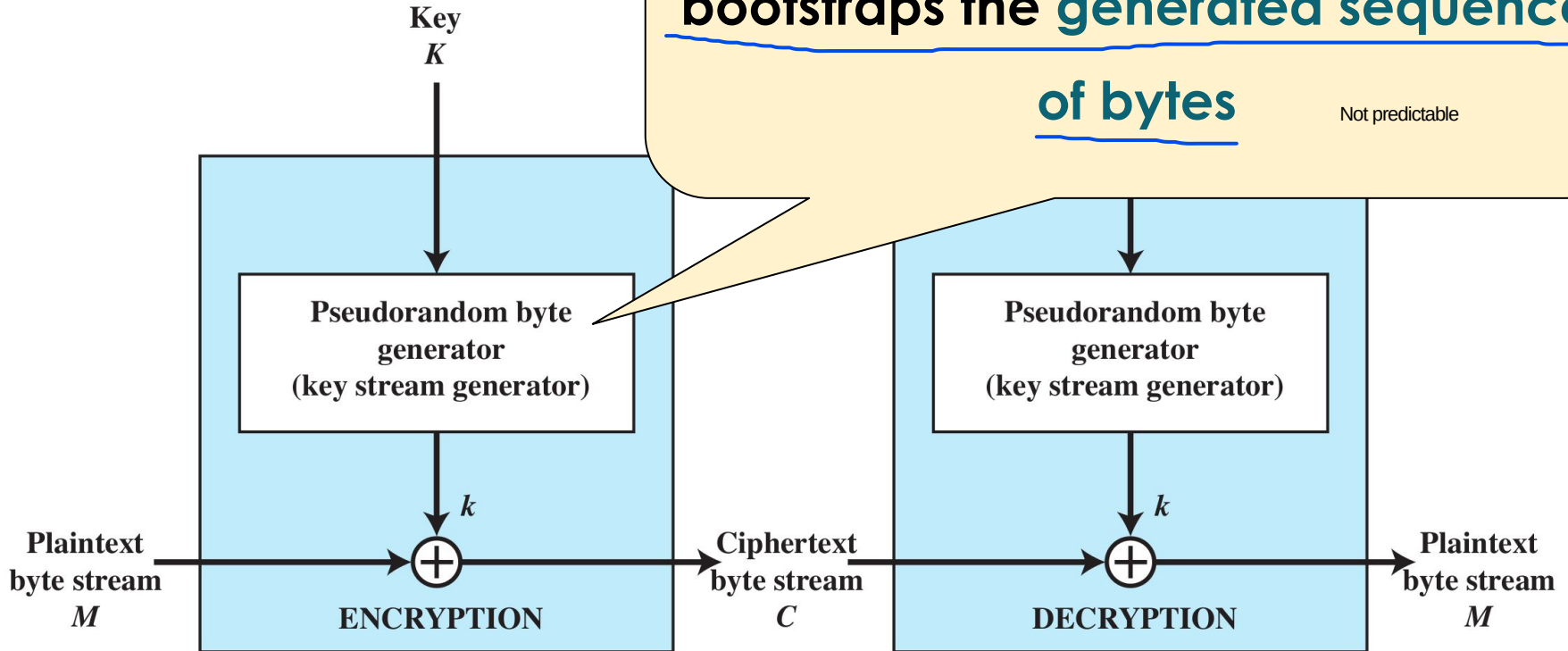
Typically, stream divided in bytes (can be even bits). So, we encrypt every time there is a block of byte/bits



Stream Encryption

The **key** is the parameter that
bootstraps the **generated sequence**
of bytes

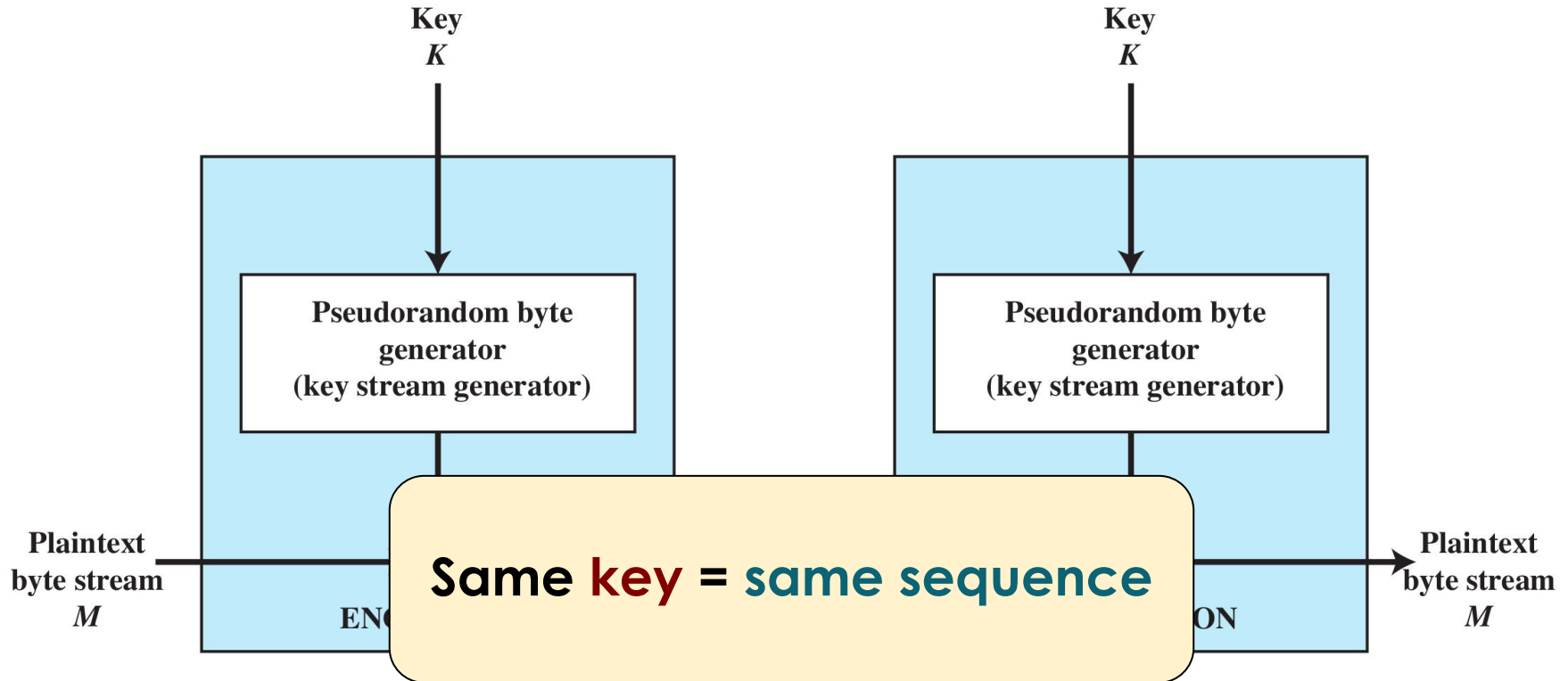
Not predictable



Stream Encryption



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



In a stream cipher, The generator does not work with 'fixed blocks' in the traditional sense. It is like a tap left running: it keeps outputting bytes for as long as needed. There is no need for 'padding' (adding empty bytes). If your data is 7 bits, the generator gives you 7 bits. If it is 1 gigabyte, it gives you 1 gigabyte.

Stream Encryption



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

At the time of the operation (XOR), the length of the data 'block' and the length of the keystream 'segment' must match.

Key
 K

The secret **key** is never transmitted "in clear"

Pseudorandom byte
generator
(key stream generator)

Pseudorandom byte
generator
(key stream generator)

Same key means that it is possible to generate the same sequence of pseudorandom byte

Same **key** = same sequence

Plaintext
byte stream
 M

ENC

ON

Plaintext
byte stream
 M

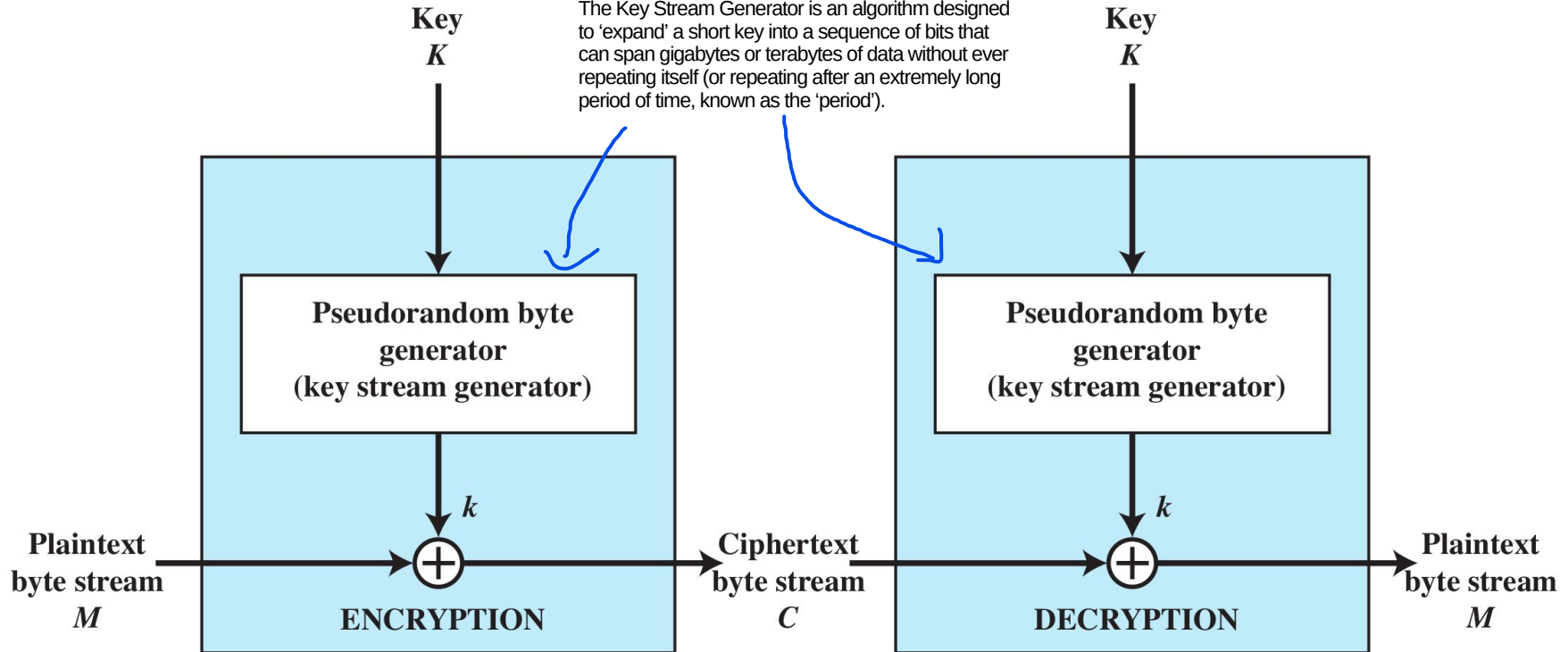
L'aspetto più importante è questo: Input (Chiave): Molto corta (es. 256 bit). Output (Keystream): Potenzialmente infinito. Il generatore è un algoritmo che "espande" una chiave piccola in una sequenza di byte casuali lunghissima. Quindi, anche se la tua chiave è minuscola, il generatore produce abbastanza byte random per coprire l'intero flusso di dati, qualunque sia la sua lunghezza.

Stream Encryption



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

The Key Stream Generator is an algorithm designed to 'expand' a short key into a sequence of bits that can span gigabytes or terabytes of data without ever repeating itself (or repeating after an extremely long period of time, known as the 'period').





ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Using a Block or Stream Cipher gives information to the attacker:

1. Leakage in Block Ciphers: The "Structure"

The biggest risk with block ciphers (especially in simple modes like ECB) is that they preserve patterns.

Pattern Leakage: If you encrypt a file where many blocks are identical (like the black pixels in a logo or the "0"s in a database), every identical plaintext block turns into an identical ciphertext block.

What the attacker learns: They don't see your data, but they see the shape of it. They can tell if a file is an image, a spreadsheet, or a specific type of document just by looking at the repetitions in the encrypted file.

2. Leakage in Stream Ciphers: The "Length"

Since stream ciphers encrypt bit-by-bit or byte-by-byte without padding, the ciphertext is exactly the same length as the plaintext.

Length Leakage: If I send the word "Yes" (3 bytes), the encrypted message is 3 bytes. If I send "No" (2 bytes), it's 2 bytes.

What the attacker learns: By monitoring the size of encrypted packets in a chat app, an attacker can guess which words are being used.

Example: If a "Help me" message is always 7 bytes, an attacker seeing a 7-byte burst of traffic knows exactly what you just asked for, even if they can't "read" the encrypted bytes.

Block vs. Stream Ciphers

Block vs. Stream Ciphers

The use or not of zeros (so using other things to do padding) as padding depends on the robustness of the cipher against cryptanalytic attacks



- **Block ciphers:**

- process the **input** one block of elements at a time

- produce an **output block** for each input block

- can **reuse keys** (and usually they do that)

- can require to add **padding** if the block is not completely filled (fixed block size), you need to add trash data or zeros

- bad for **efficiency!**

- how is **padding generated?** Using zeros is not a good idea because it is highly likely to cause diffusion problems. It is very likely that a random data generator ('trash') will be needed to resolve the diffusion problems: but we need to assess how fast it is

- are **more common** (than stream ciphers) In typical modern CPU, block ciphers are implemented in hardware (not the stream ones)

Block vs. Stream Ciphers



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Stream ciphers:**

- process the **input** elements **continuously** (immediately, without having to wait: no introduction of latency)
- produces **output one element at a time** every single time there is a new byte to be encrypted, immediately they are able to do it and send it
- almost always **faster** and **use far less code** (In block ciphers, the complexity is in the algo: AES is very complex. In stream ciphers, the algo is the pseudorandom byte generator: the complexity is external, not in the direct encryption, and the complexity is simpler than the block ciphers)
- encrypts plaintext **one byte at a time**
- pseudorandom stream is one that is **unpredictable without knowledge of the input key**



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

TLS (used for HTTPS) is based on Symmetric Encryption, Secure hash functions and Asymmetric Encryption: all of three is required

It's not just for TLS: it's also used to verify software integrity

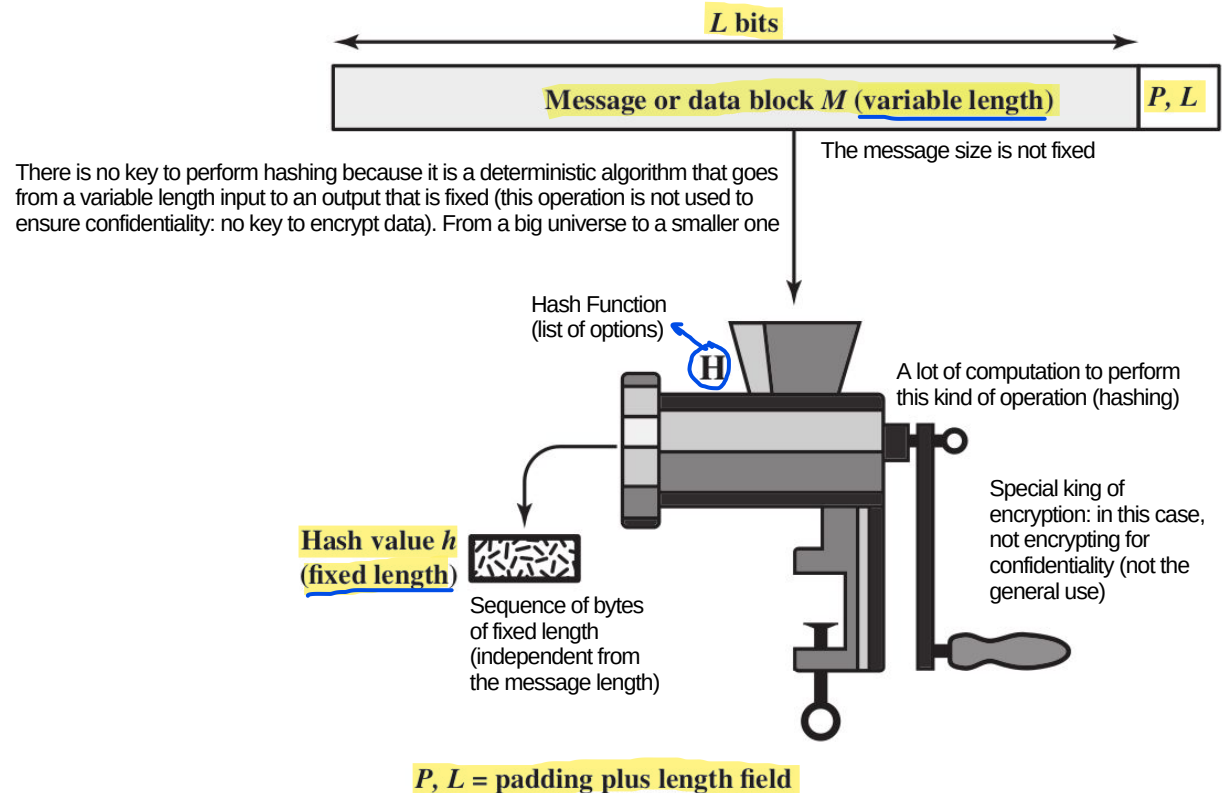
Secure Hash Functions

Secure Hash Functions

isn't for confidentiality



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

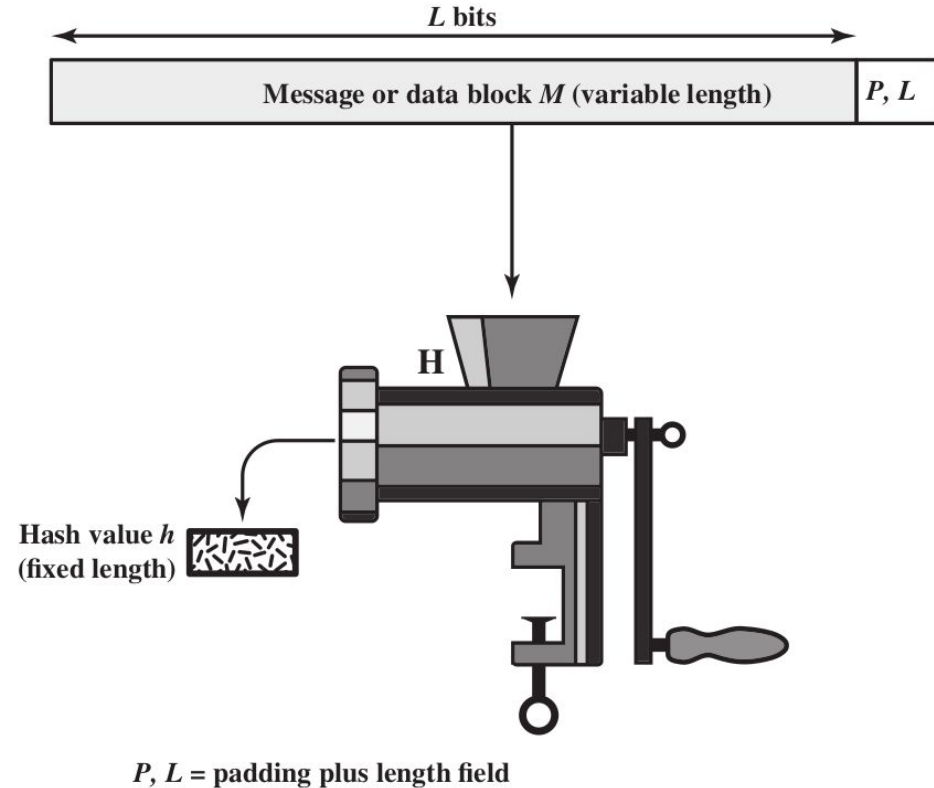


Secure Hash Functions



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

The **input size** is **variable**



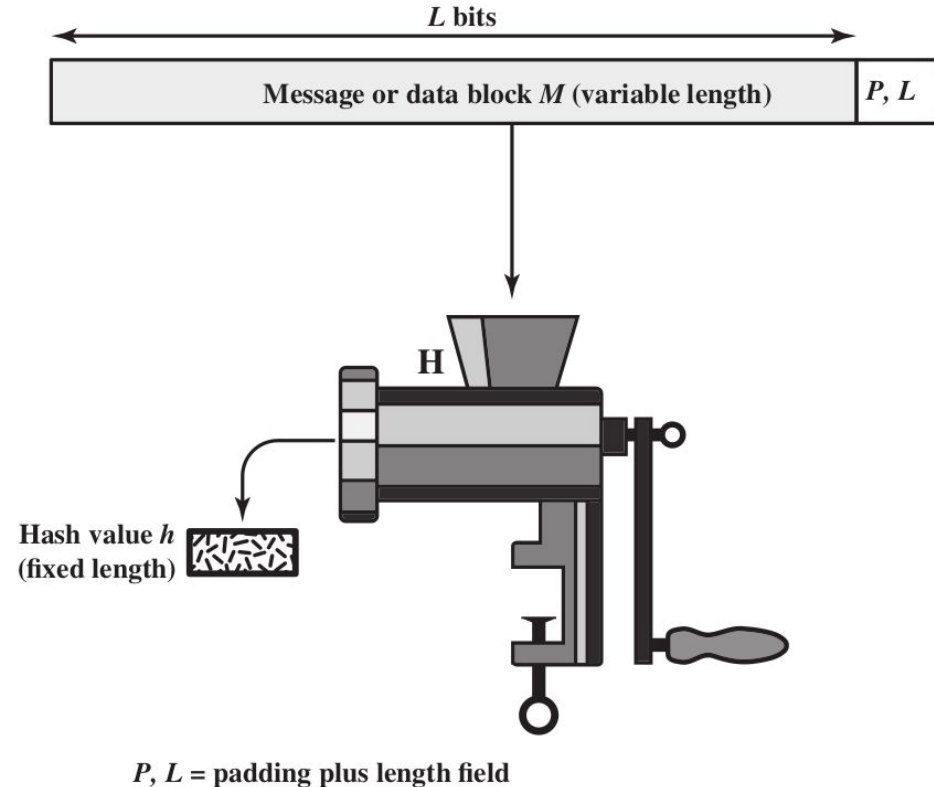
Secure Hash Functions



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

The **input size** is **variable**

H is a mathematical function



Secure Hash Functions



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

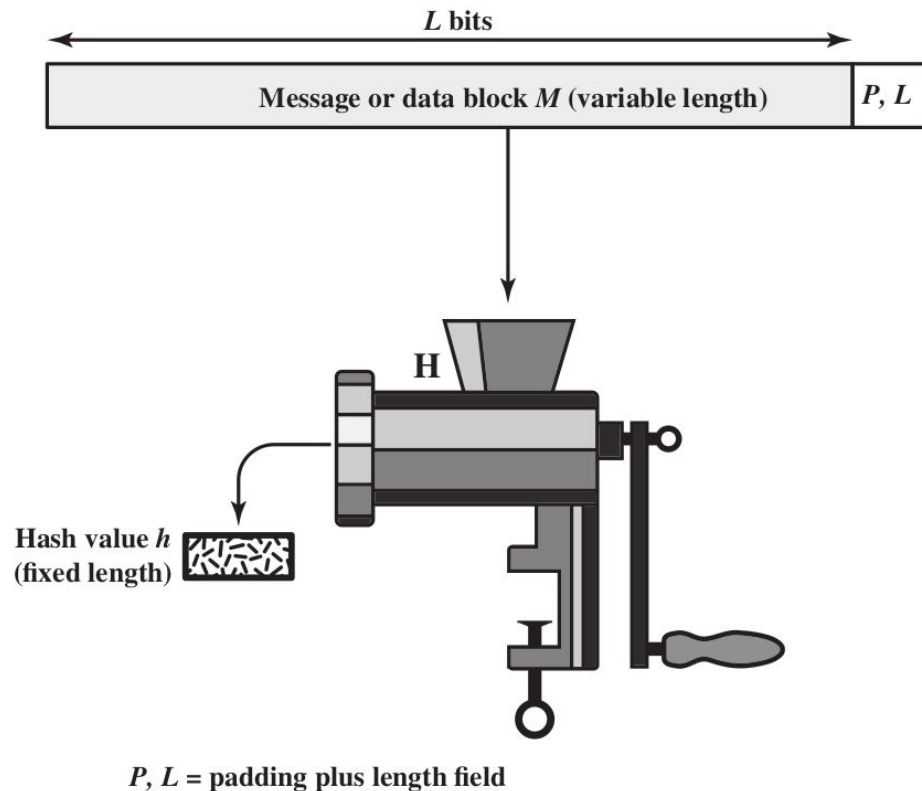
The input size is variable

The hash function must be chosen carefully to guarantee specific required properties

H is a mathematical function

The output size is fixed

cannot be changed (it is a characteristic of hash function that we are using: design choice)



if it isn't like this, it is easy to get the plaintext through bruteforcing. For example, firmware update with hash (signature) of the update to check if it is compromised; but if the attacker can get the pre-signature (data block before hashing) easily can create a compromised version of the firmware (having the same signature of the original version of the firmware)

Secure Hash Functions: properties



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

What are the properties that are important for us?

1. **H can be applied to a block of data of any size**
2. **H produces a fixed-length output** if i change the size of the output, i am changing the function itself (so, its design)
3. **$H(x)$ is relatively easy to compute** for any given x (qualitative property: reasonable amount of time to do hashing)
4. For any given ^{hash}code h , it is computationally infeasible to find x such that

$H(x) = h$ (one-way or preimage resistant)

The inverse operation – that is, going from the hash back to the input – must be incredibly computationally intensive (it's not impossible, but it must be difficult to get back to the input from the output). The point 4 says: there is no easy way for performing the inverse operation of hashing and the only way to perform it is try every single combination that is in the start universe (huge: exponential) and check if the output match the hash

We are fixing a specific message (the attacker isn't able to generate a specific collision with this specific message)

5. **For any given block x , it is computationally infeasible to find $y \neq x$ with $H(y)$**

$= H(x)$ (second preimage resistant or weak collision resistant)

different messages
but same hash code

6. **It is computationally infeasible to find any pair (x, y) such that $H(x) = H(y)$**

(strong collision resistant)

It must not be possible to find any two messages (x and y) that produce the same hash. The difference from the 5th property: Here, the attacker has complete freedom; they do not need to start with a specific message, but simply need to find two different inputs that collide to produce the same output.

A small modification on the input will cause a complete different hash than the original message (if the hash function is secure)

We are fixing a specific message (the attacker isn't able to generate a specific collision with this specific message)

5. **For any given block x , it is computationally infeasible to find $y \neq x$ with $H(y)$**

$= H(x)$ (second preimage resistant or weak collision resistant)

different messages
but same hash code

6. **It is computationally infeasible to find any pair (x, y) such that $H(x) = H(y)$**

(strong collision resistant)

It must not be possible to find any two messages (x and y) that produce the same hash. The difference from the 5th property: Here, the attacker has complete freedom; they do not need to start with a specific message, but simply need to find two different inputs that collide to produce the same output.

Point 6 is
the father
generalisation
of
Point 5

Secure Hash Functions: security



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- Approaches to **attack** a **secure hash function**

→ cryptanalytic attacks: something wrong in the hash function used to perform the mapping

- **cryptanalysis**: exploits **logical weaknesses** in the algorithm

→ similar to the key length of Symmetric Encryption (if it is not enough the length of the hash code, it is possible to have a collision: less size, more probability of collisions). We increase the length to put the attacker in a harder situation in performing brute force attack

- **brute-force**: the strength of the hash function depends solely on

the **length of the hash code** produced by the algorithm

- **SHA** (Secure Hash Algorithm) is the most widely used hash function

- many different “families” (SHA0, SHA1, SHA2, SHA3) and hash value lengths

Secure Hash Functions: security



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- Approaches to **attack** a **secure hash function**

- **crypt**

- **brut**

the

- **SHA** (Se

- many different “families” (SHA0, SHA1, SHA2, SHA3) and hash value lengths

If a collision is found, the hash function is no longer secure (because the property 6, part of properties to define a secure hash function, is no longer valid)

Lifetime of some famous

cryptographic hash functions

Some hash function are no longer safe

Secure Hash Functions: applications



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Fundamental for these two tasks

- **Passwords:**

- the operating systems (and most online services) store only the

Other problems: only a part of the system can be checked (for data is difficult, only binary). If you have updates, you need to update the DB of hash code of the binary

hash of passwords (not the plaintext of passwords)

- **Intrusion detection:** Create a fingerprint of each file and check if there is a compromise (check the hash code with the original program one)

- somehow compute and securely store the hash code of relevant

files in the system. This permits to verify their integrity of the software running on the computer

1st Problem: asking to a program in the file system if the file system or a specific program is compromised (the attacker, if is in the system, is easy for him to compromise even the program that do checks about hash code). Solution: let the check be done by a program that is external to the system to be checked
2nd Problem: you need a DB to store the hash codes of the original programs and it can be compromised if it is online. Solution: store a copy offline of the DB in a secure environment

Secure Hash Functions: applications



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Passwords:**

- the operating systems (and most online services) **store only** the

hash of passwords

- **Intrusion detection:**

- somehow **compute** a
files in the system. This

Plaintext passwords are never
stored. Why? To be discussed in
the authentication chapter

Secure Hash Functions: applications



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Passwords:**
 - the operating systems (and most online services) **store only** the **hash of passwords**
- **Intrusion detection:**
 - somehow **compute** and **securely store** the **hash code of relevant files** in the system. This permits to verify their **integrity**

Secure Hash Functions: applications



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Passwords:**

- the operating system (and each application) stores **only the**

hash of password

- **Intrusion detection**

- somehow **compute** and **securely store** the **hash code of relevant**

files in the system. This permits to verify their **integrity**

If you want to do fingerprint for each file and update of the system, in terms of computation, is heavy

This operation is quite costly
in terms of computation

Secure Hash Functions: applications



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Passwords:**

- the operating system

hash of passwords

Need to update the fingerprint when there is update of programs

It is not so easy! Why?

For different reasons...

- **Intrusion detection:**

- somehow **compute** and **securely store** the **hash code of relevant**

files in the system. This permits to verify their **integrity**

Secure Hash Functions: applications



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Passwords:**

- the operating systems (and applications) store the

hash of passwords

Why do we not check all
the files in the system?

- **Intrusion detection:**

- somehow **compute** and **securely store** the **hash code of relevant**

files in the system. This permits to verify their **integrity**

Secure Hash Functions: applications



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

The threat model must be considered very carefully and what part of system can be checked

- **Passwords:**

- the operating systems

hash of passwords

It works... if the **tool used for**
verifying the integrity is not
compromised

- **Intrusion detection:**

- somehow **compute** and **securely** the **hash code of relevant**

files in the system. This permits to verify their **integrity**

We lack of tools for checking the integrity of systems



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Dictionary Attack

DICTIONARY ATTACK!



Practical **dictionary attack** to
the symmetric encryption

How does it works?

Check it out!

T3

Symmetric Key Encryption



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Symmetric encryption alone cannot guarantee secure communication over the internet because it requires both participants to have the same key before they can begin communicating. However, there is no simple way to send this key to someone on the other side of the world without an attacker being able to intercept it along the way.

This is quite ridiculous: “to get a secure communication channel you need a secure channel”

- sender and receiver must have obtained

in a secure fashion and must keep the key secure

confid

How is possible to implement secure communication in online banking or e-commerce with such a limitation?



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

To solve the problem about key distribution (talked in the previous slide), we use Public-Key encryption (asymmetric)

Public-Key Encryption

Public-Key Encryption

Why does this solve the problem?

- No keys are transmitted: The key that is exchanged is not the final key. The key is generated simultaneously at both locations.

- Resistance to interception: A hacker who intercepts the messages sent between the two parties to generate the key would be faced with a mathematical problem that would take thousands of years to solve using today's computers.



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Bits of history**

The Diffie-Hellman (DH) protocol was the first truly revolutionary solution to the problem of key distribution. It is a method that allows two people (let's call them Alice and Bob) to establish a shared secret key over a public channel (the internet), without ever having met before and without the key ever actually being "sent."

- **1969** - United Kingdom - James Ellis (GCHQ): **theoretical basis**
- **1976** - USA - Whitfield Diffie and Martin Hellman: **Diffie-Hellman**
asymmetric key algorithm
key exchange (solution to the key distribution problem)
- **1977** - USA - **R**onald Rivest, Adi **S**hamir and Leonard **A**delman: **RSA**
public-key cryptosystem Plenty of current implementation are based on this
- **1991** - USA - Phil Zimmermann, **Pretty Good Privacy (PGP)**

Public-Key Encryption



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Bits of history**

- **1969** - United Kingdom - James Ellis (GCHQ): **theoretical basis**

- **1976** - USA - Whitfield Diffie

key exchange (solution to

- **1977** - USA - Ronald Rivest

public-key cryptosystem

Classified (i.e., secret) and
disclosed only in 1997

- **1991** - USA - Phil Zimmermann, **Pretty Good Privacy (PGP)**

Public-Key Encryption

The security of public-key infrastructure depends on the binding between a public key and its owner. While the public key is intended for open distribution, its provenance must be guaranteed. If a sender cannot verify that a key belongs to the intended recipient, they risk encrypting data for an adversary who is impersonating that identity. This highlights the necessity of Digital Certificates or a Web of Trust to prevent impersonation.



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Asymmetric (public-key) encryption is based on mathematical problems that are easy to solve in one direction but extremely difficult to reverse.

The **public-key encryption** is based on some **complex mathematical problems**

- Factorization: given a big number, find the factors of this big number
- Elliptic curves

4 keys for a bi-directional communication

It is a form of **asymmetric encryption** since **every user has 2 keys**:

- one **public key**: *that is made as public as possible*

(must be known to everyone: a public key associated with the identity of the user who possesses the corresponding private key)

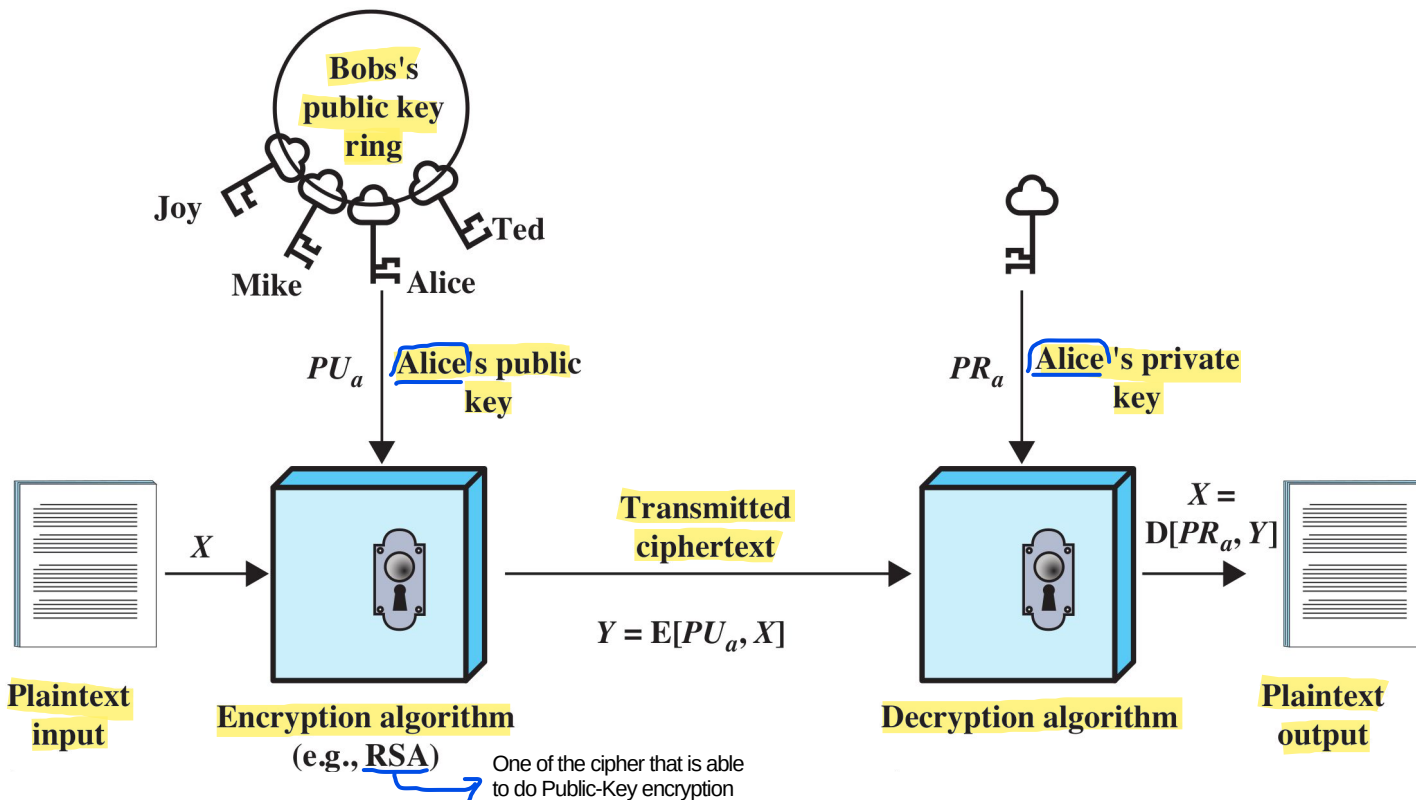
- one **private key**: *that is maintained carefully secret*

- Every **couple of keys** is **generated together**, the two keys are **linked by some mathematical properties**

Public-Key Encryption



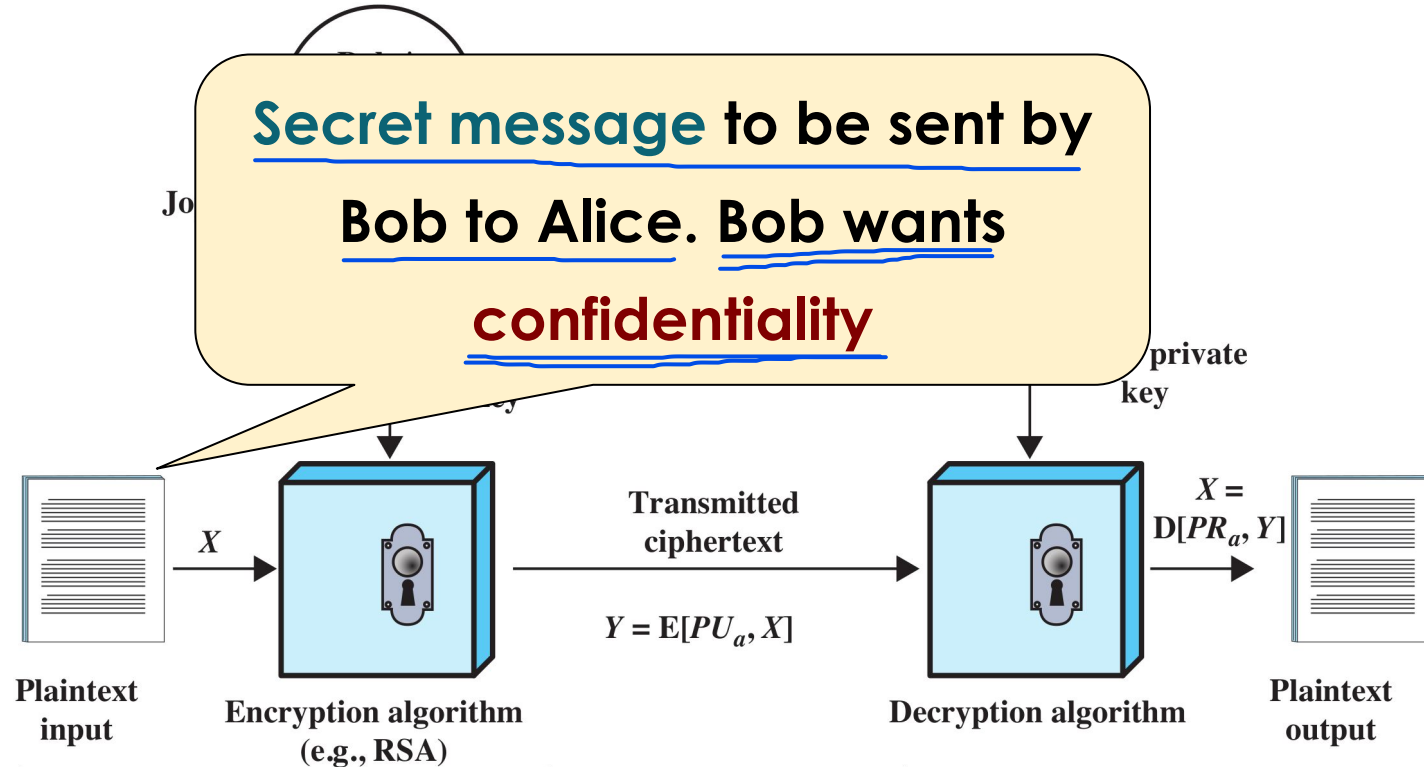
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



Public-Key Encryption



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

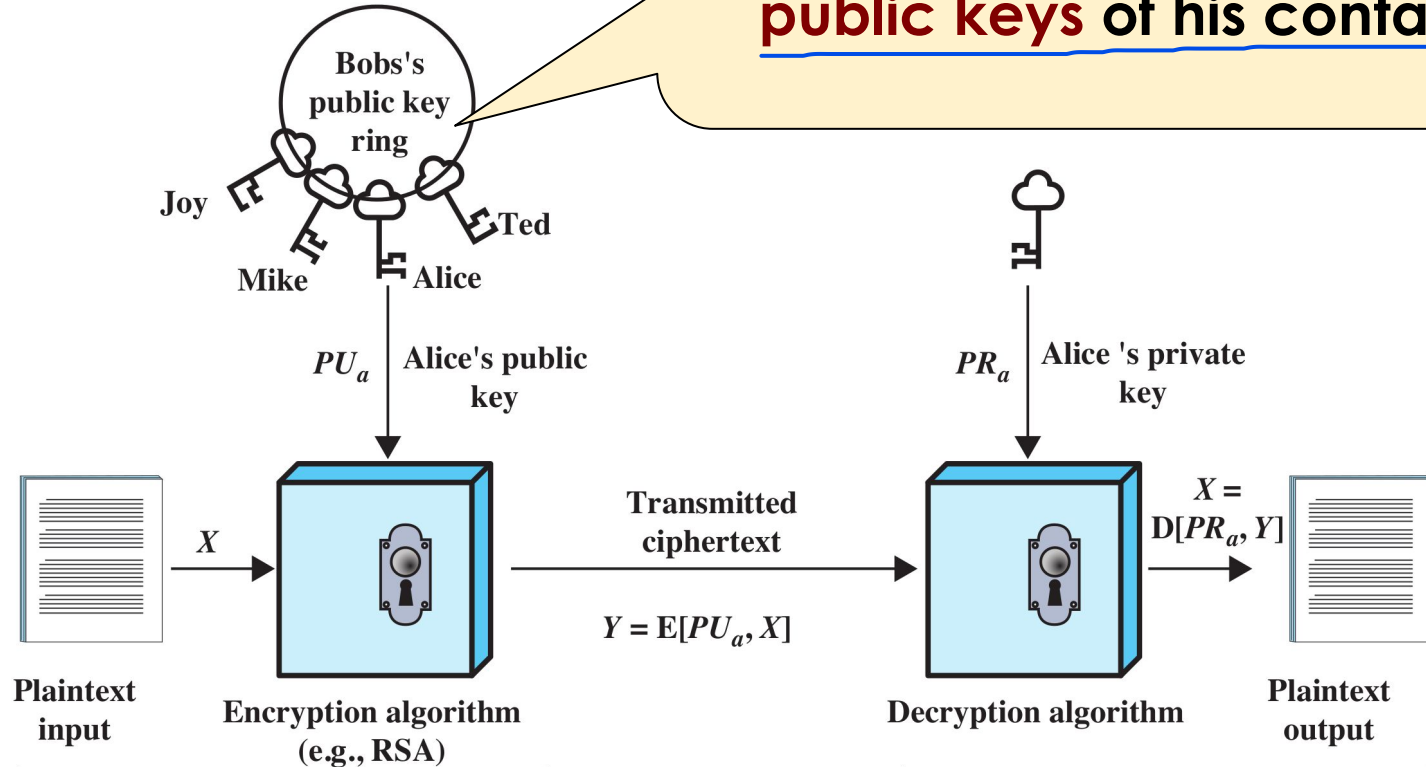


B
O
B

A
L
I
C
E

Public-Key Encryption

Bob has a keyring with all the public keys of his contacts



B
O
B

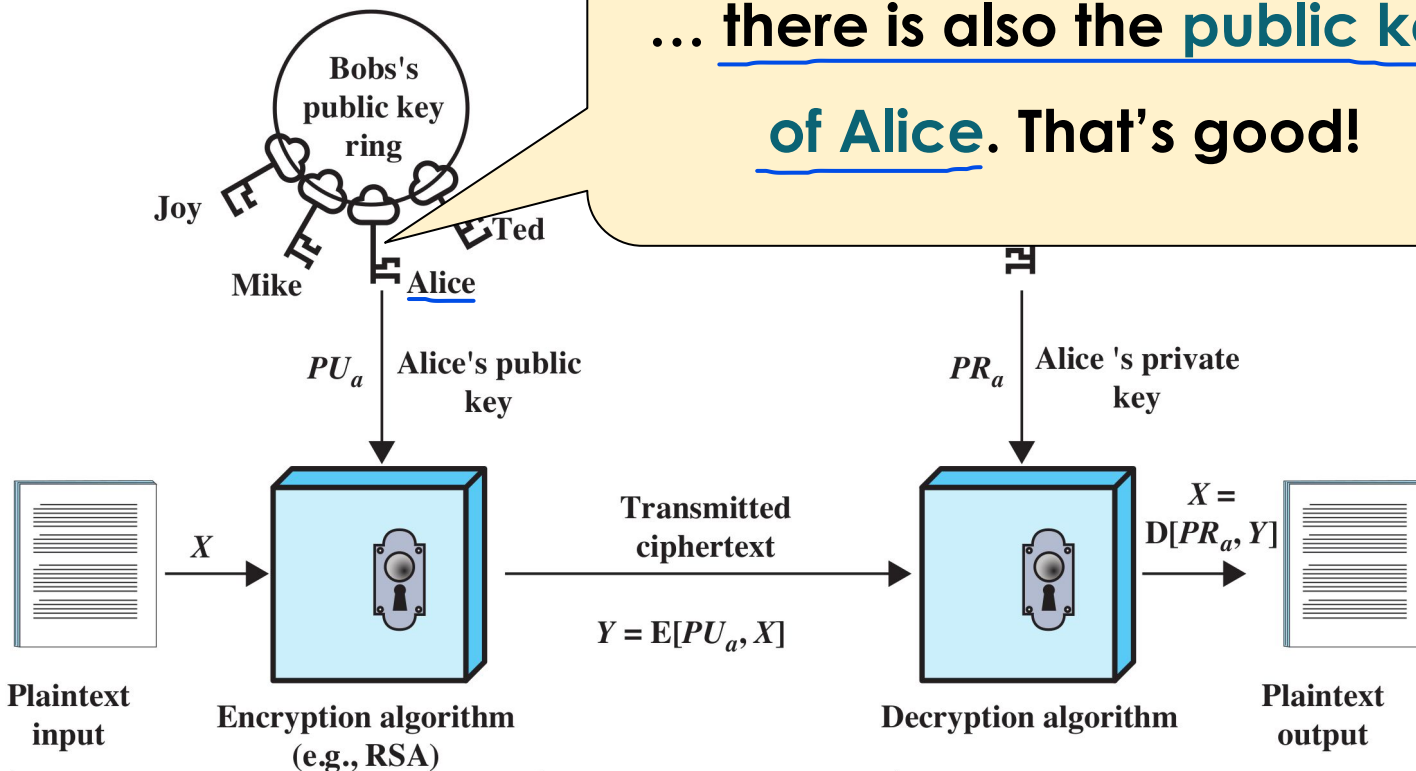
A
L
I
C
E

Public-Key Encryption



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

... there is also the public key
of Alice. That's good!



B
O
B

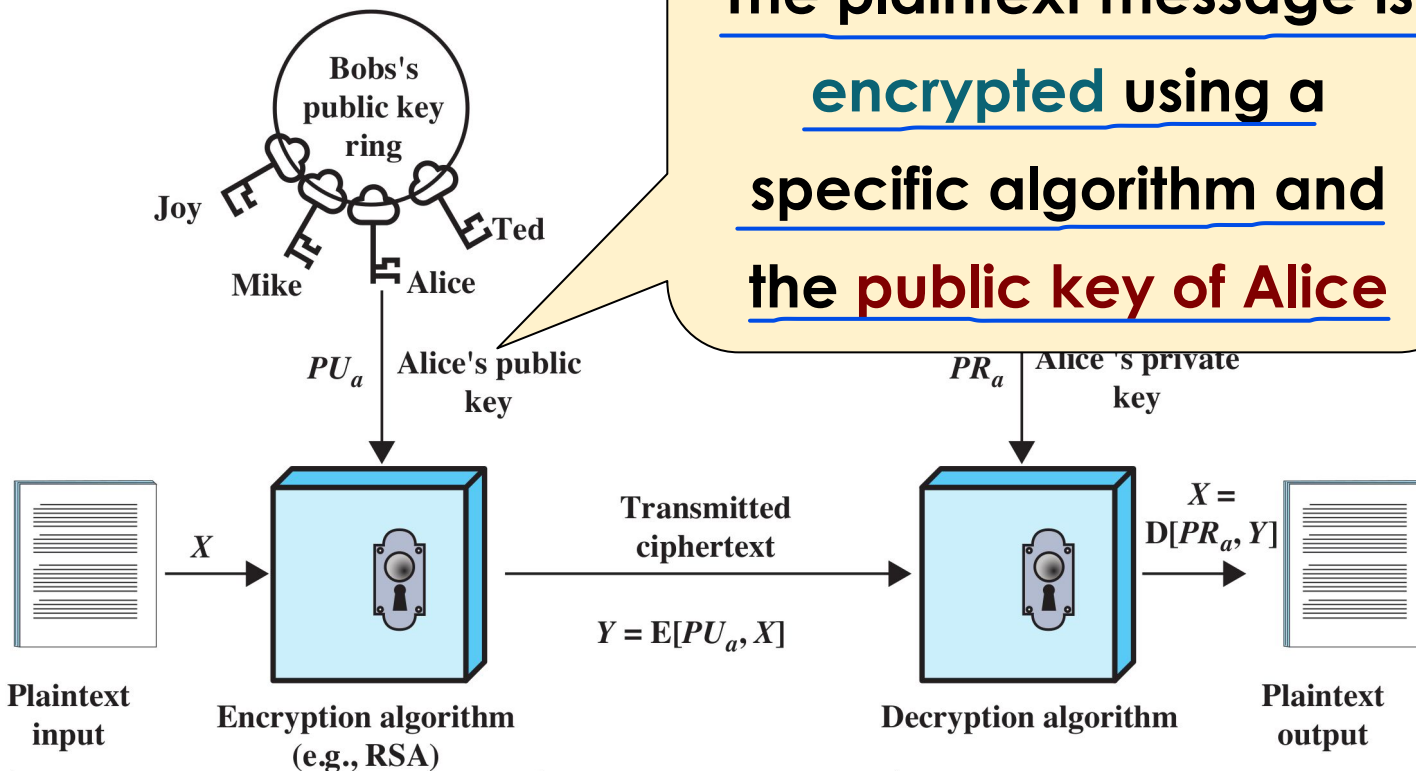
A
L
I
C
E

Public-Key Encryption



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

The plaintext message is
encrypted using a
specific algorithm and
the public key of Alice

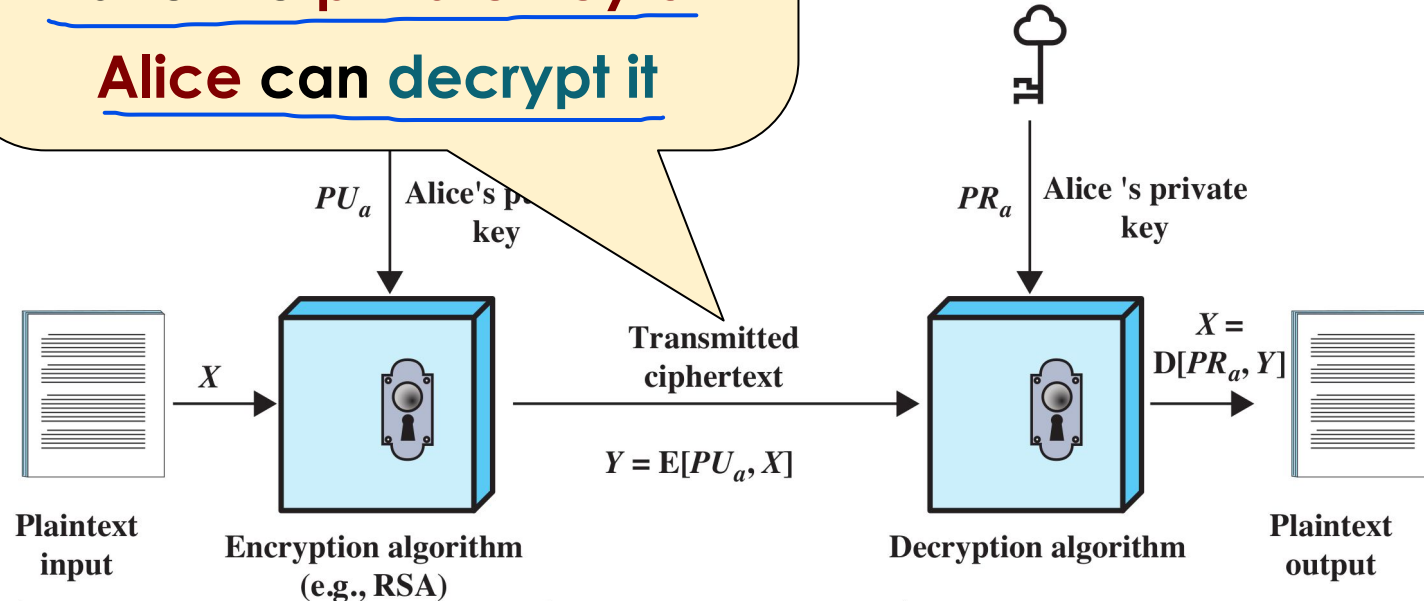


B
O
B

A
L
I
C
E



The ciphertext can be
transmitted on the Internet
since only those who
have the private key of
Alice can decrypt it



A
L
I
C
E

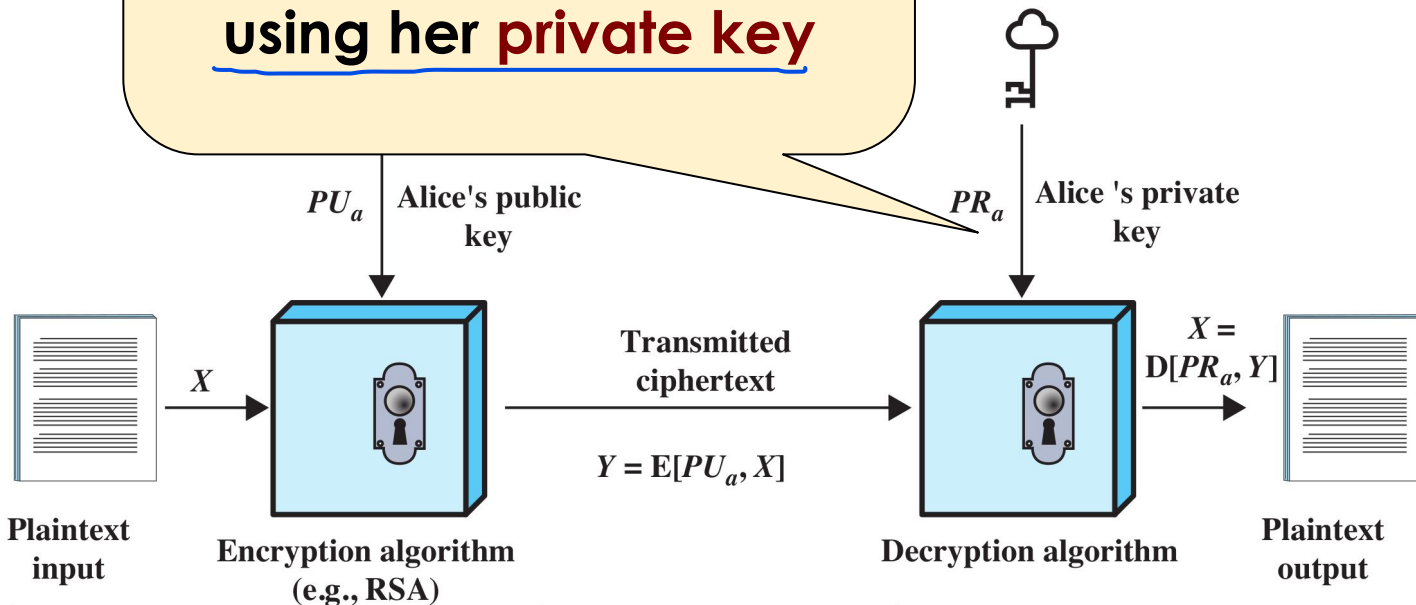
B
O
B

Public-Key



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Alice decrypts the
encrypted message
using her private key



Public-Key Encryption

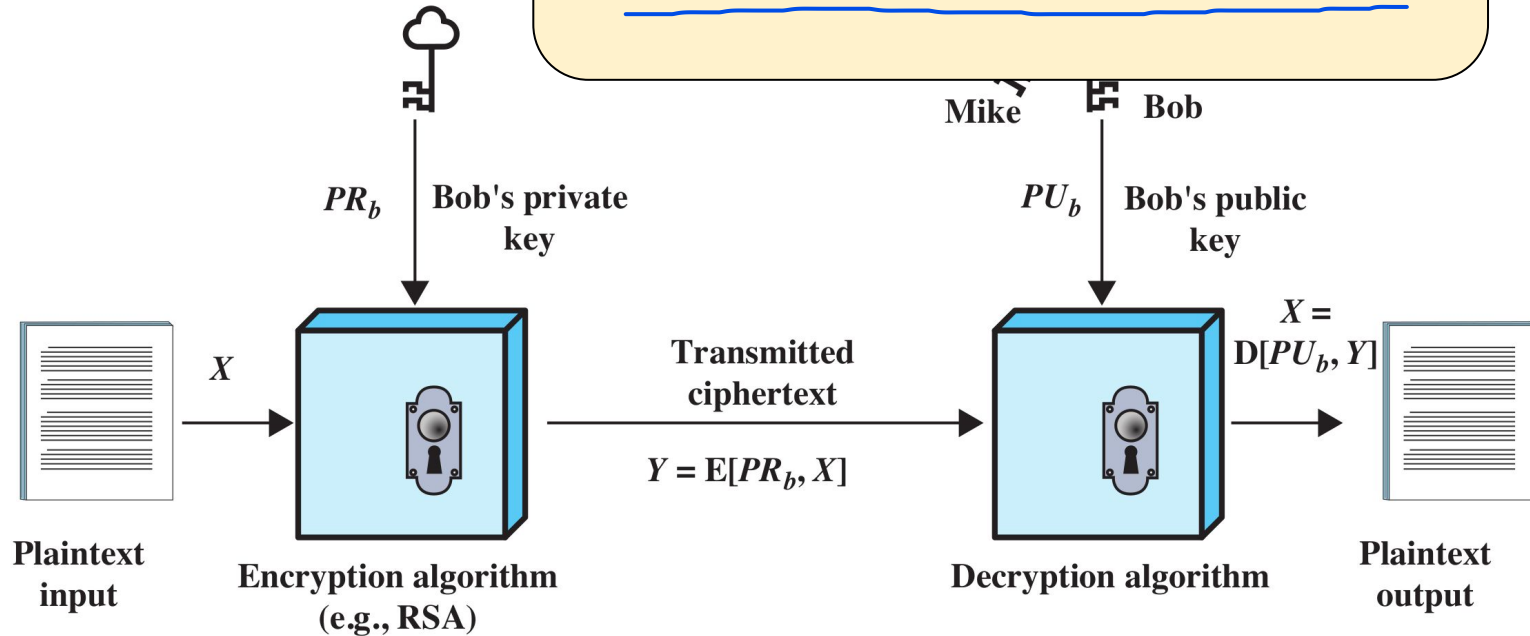


ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Public key encryption can be
also used in a different way

A
L
I
C
E

B
O
B



Public-Key Encryption

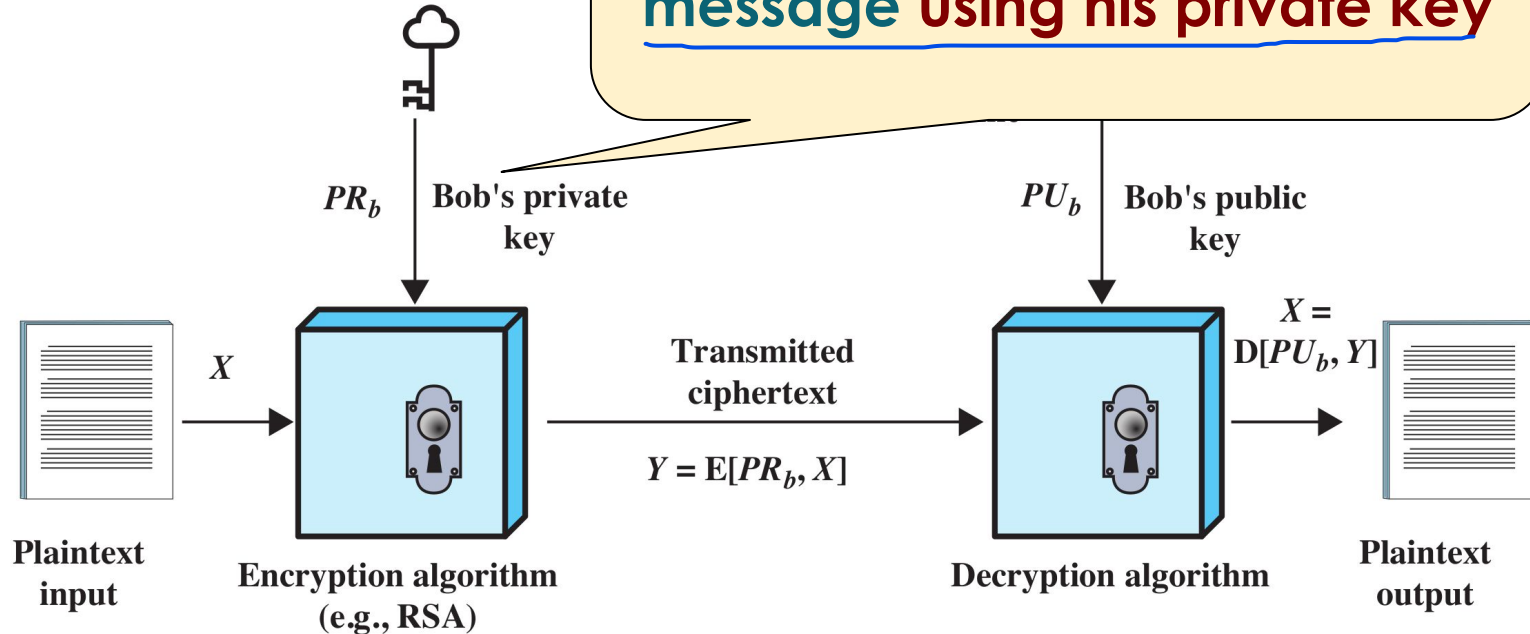


ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

In this case, Bob encrypts the message using his private key

A
L
I
C
E

B
O
B



Public-Key Encryption



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

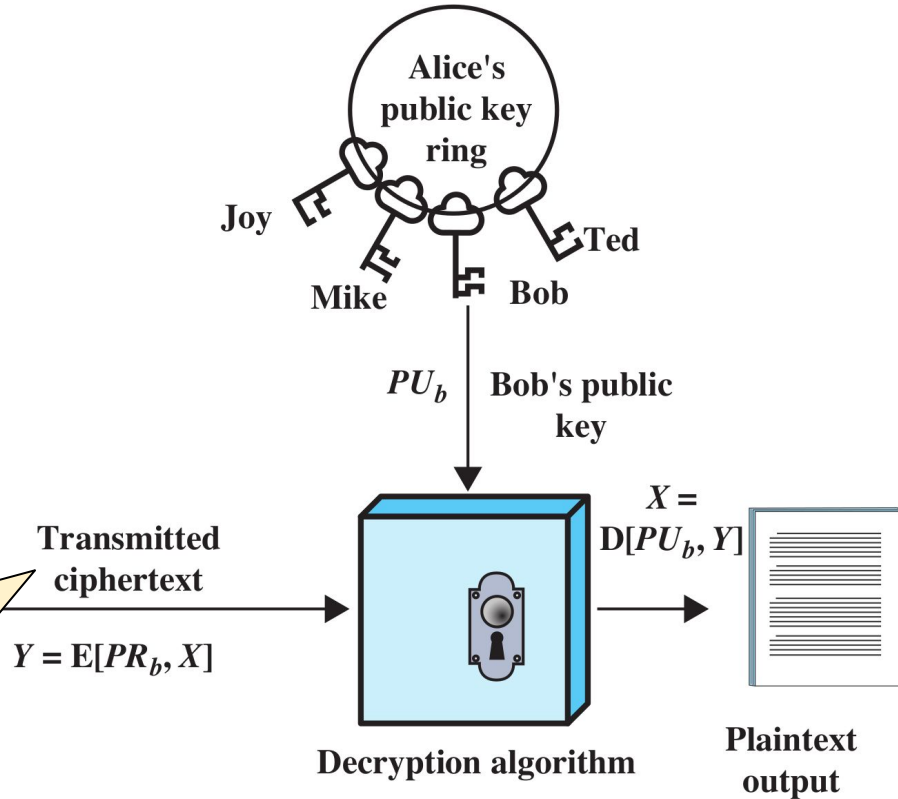
A
L
I
C
E

B

The **ciphertext** is
transmitted on the
Internet but it is **NOT**
confidential!



private
key



Public-

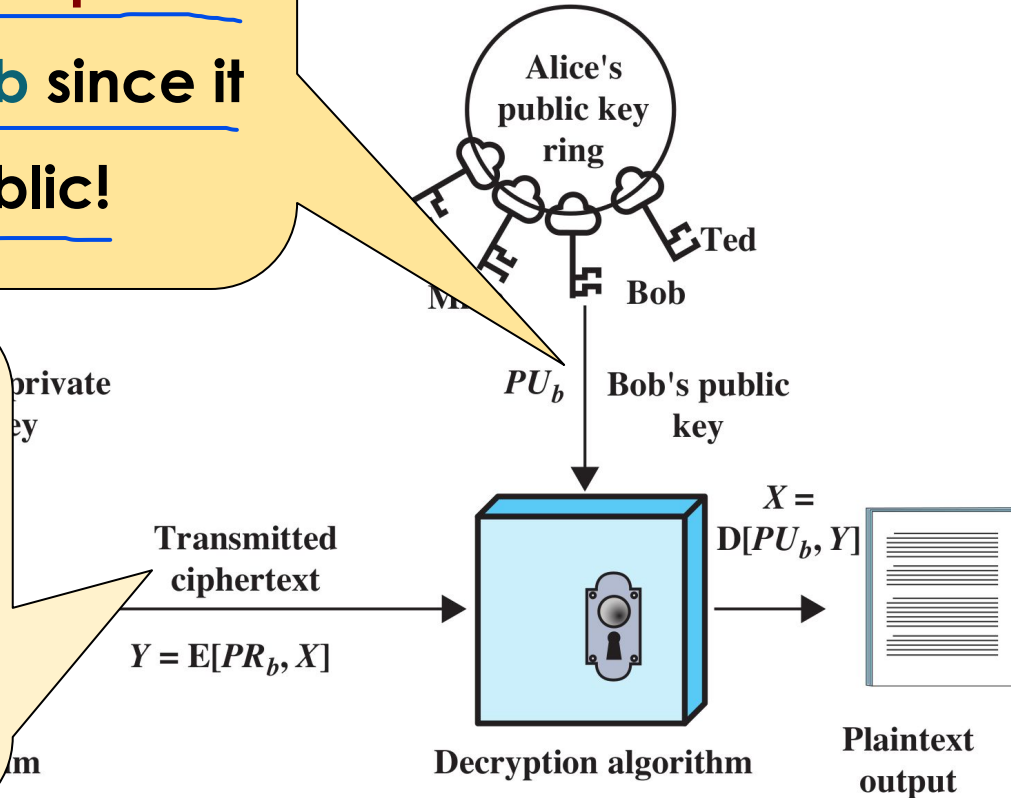
Because everyone
can get the public
key of Bob since it
is public!



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

A
L
I
C
E

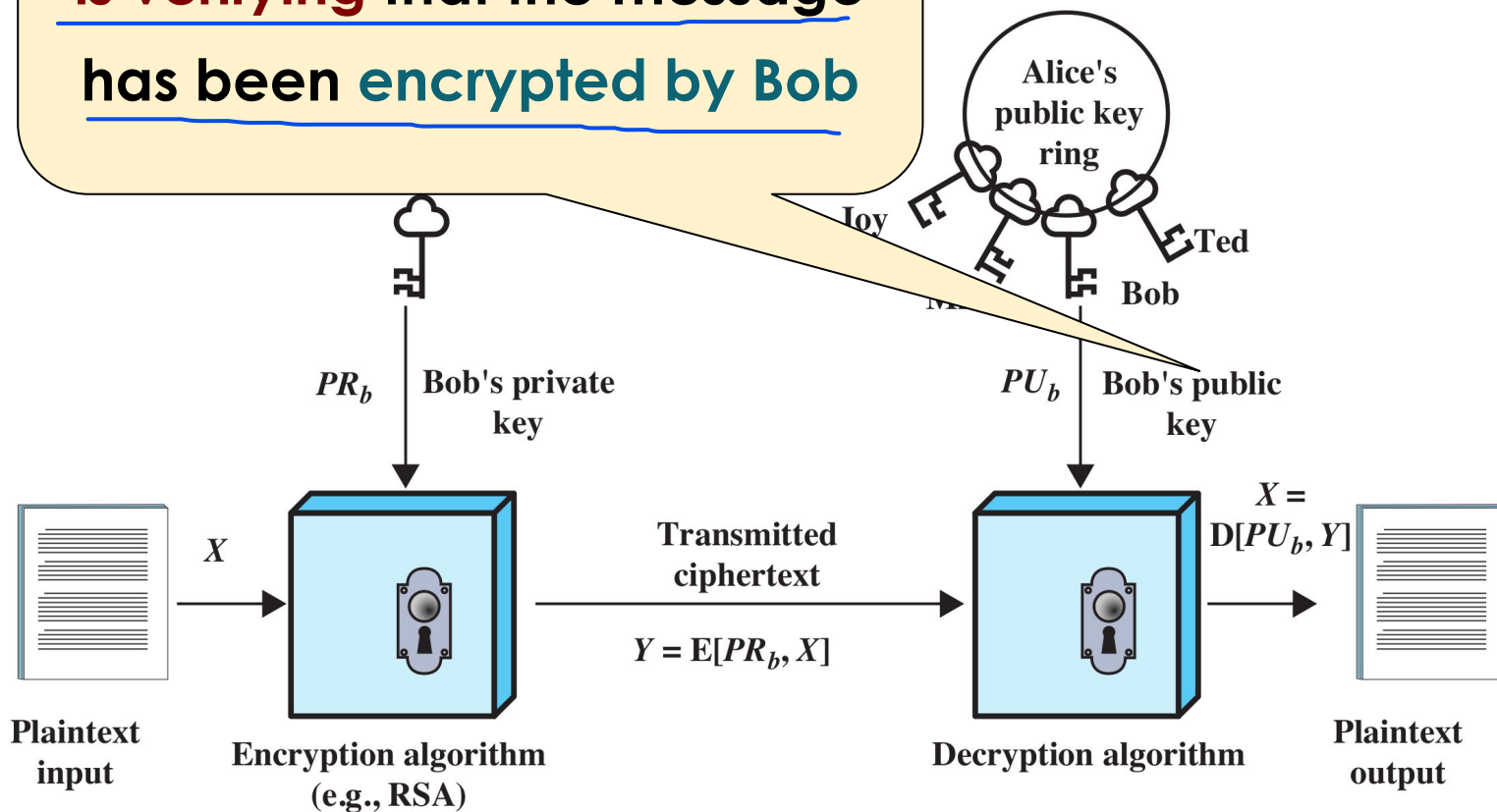
The ciphertext is
transmitted on the
Internet but it is NOT
confidential!



Using Bob's public key, Alice
is verifying that the message
has been encrypted by Bob

A
L
I
C
E

B
O
B

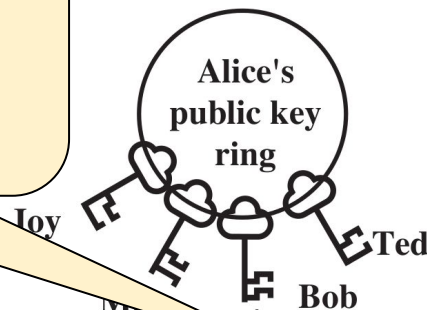


Public-key



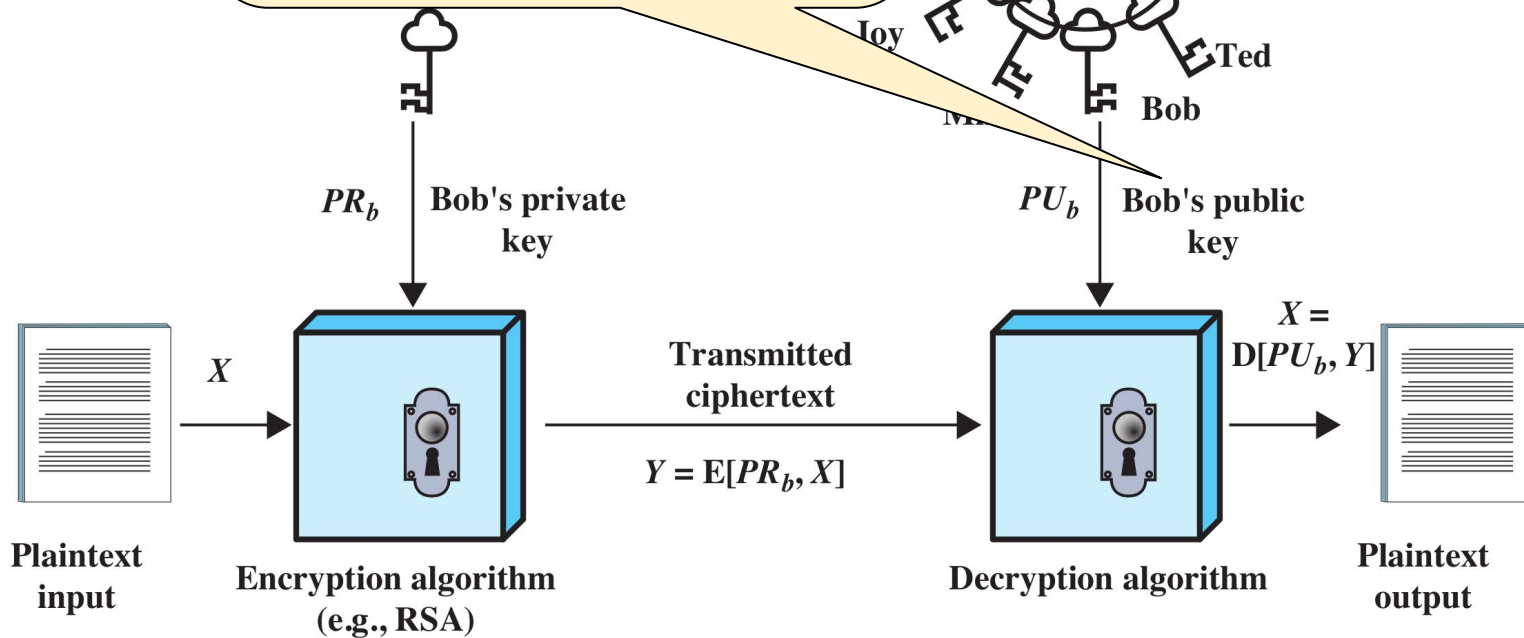
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

This verification is
easy since Bob's
public key is public!

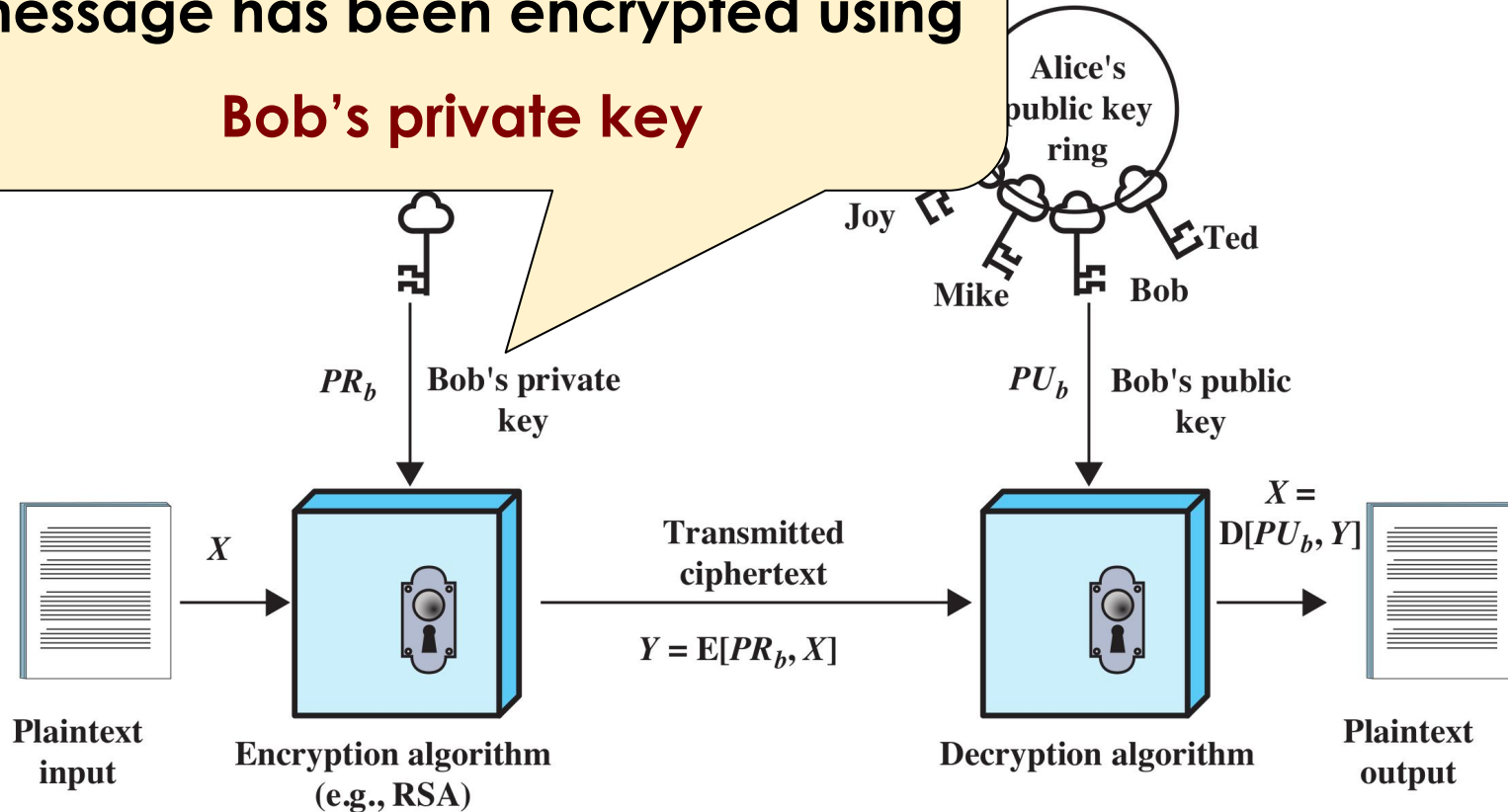


A
L
I
C
E

B
O
B



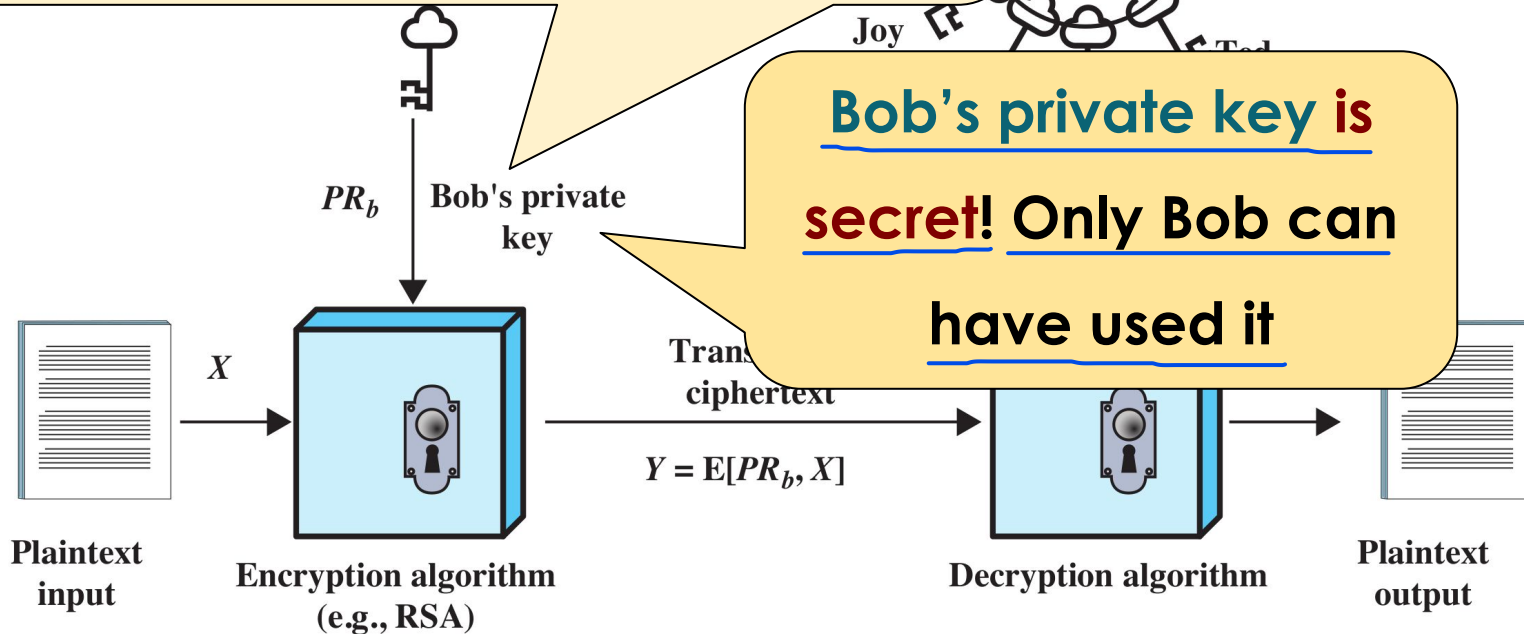
The **verification** is **positive** only if the message has been encrypted using **Bob's private key**

A
L
I
C
EB
O
B

The **verification** is **positive** only if the
message has been encrypted using

Bob's private key

A
L
I
C
E



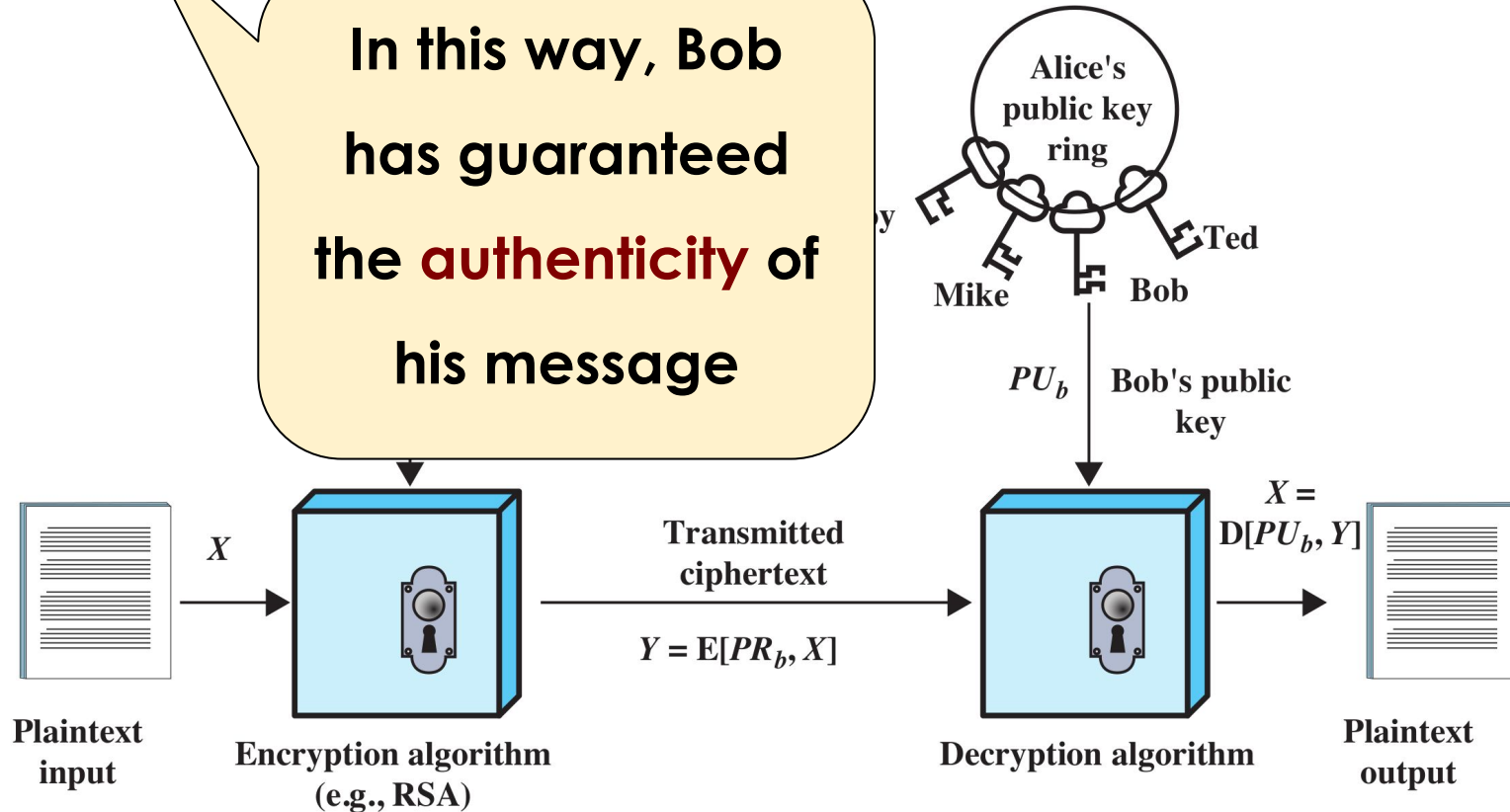
Public-Key Encryption



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

In this way, Bob
has guaranteed
the **authenticity** of
his message

A
L
I
C
E



B
O
B

Public-Key Encryption

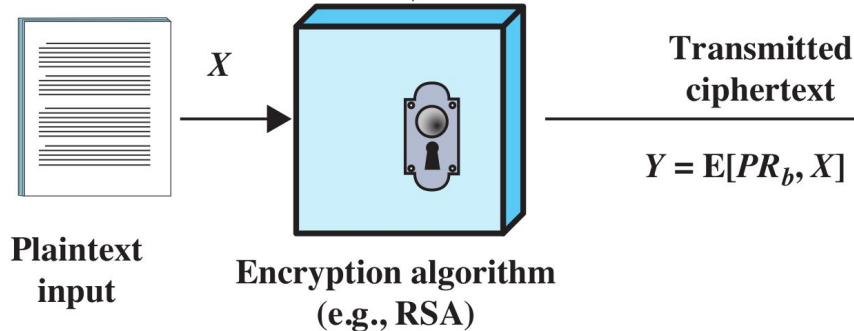


ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

A
L
I
C
E

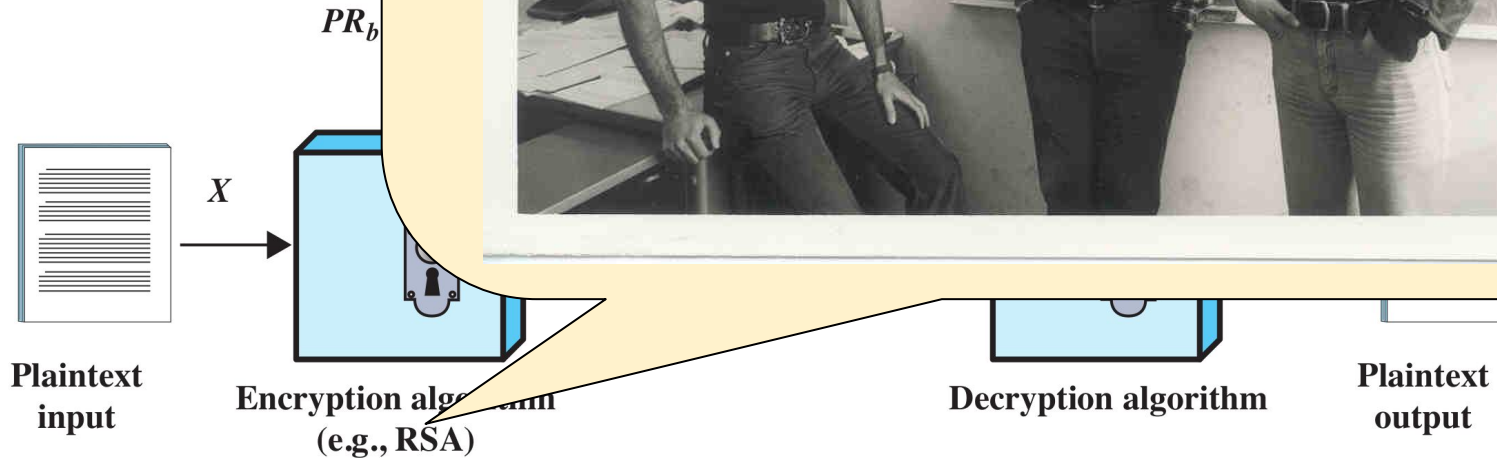
In this way, Bob
has guaranteed
the authenticity of
his message

This message is
really from Bob
and not from an
impostor



Public-Key E

B
O
B



E

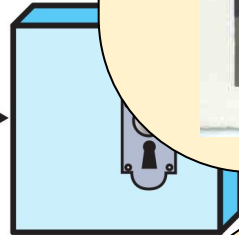
Public-Key E

B
O
B



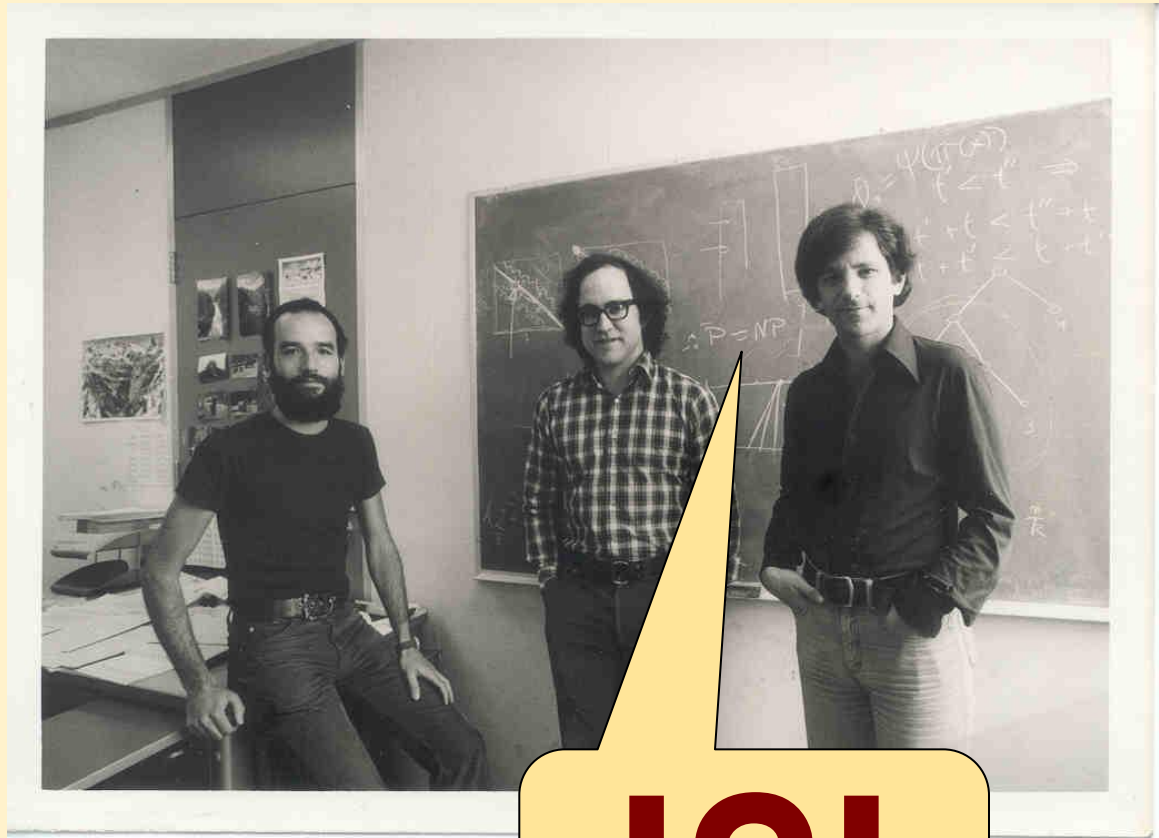
Plaintext
input

X



Encryption algorithm
(e.g., RSA)

PR_b



!?!?

Dec

Text
t

E

Public-Key Encryption: requirements



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

computationally **easy** to
create key pairs

useful if **either key** can
be used for **each role**

computationally **easy** for
sender knowing public key to
encrypt messages

computationally **infeasible**
for opponent to otherwise
recover original message



computationally **easy** for
receiver knowing private key to
decrypt ciphertext

computationally **infeasible** for opponent
to **determine private key** from **public key**

Public-Key Encryption: a big problem



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- The mechanism described before works very well but **there is still a problem**
- How is it possible to **guarantee** (i.e. **verify**) that the **Bob's public key is really the public key of Bob?**
- For example: an impostor could impersonate Bob with the aim to publish a fake "Bob's public key"

Public-Key Encryption: a big problem



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- The mechanism described before **is still a problem**
- How is it possible to **guarantee** (i.e. **really the public key of Bob?**)
- **For example:** *an impostor could impersonate Bob with the aim to publish a fake “Bob’s public key”*

And then?

With what
consequences?



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

GNU Privacy Guard



Usage examples of **GPG**

Encrypted emails?

Check it out!

T4



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Digital Signatures

Digital Signatures



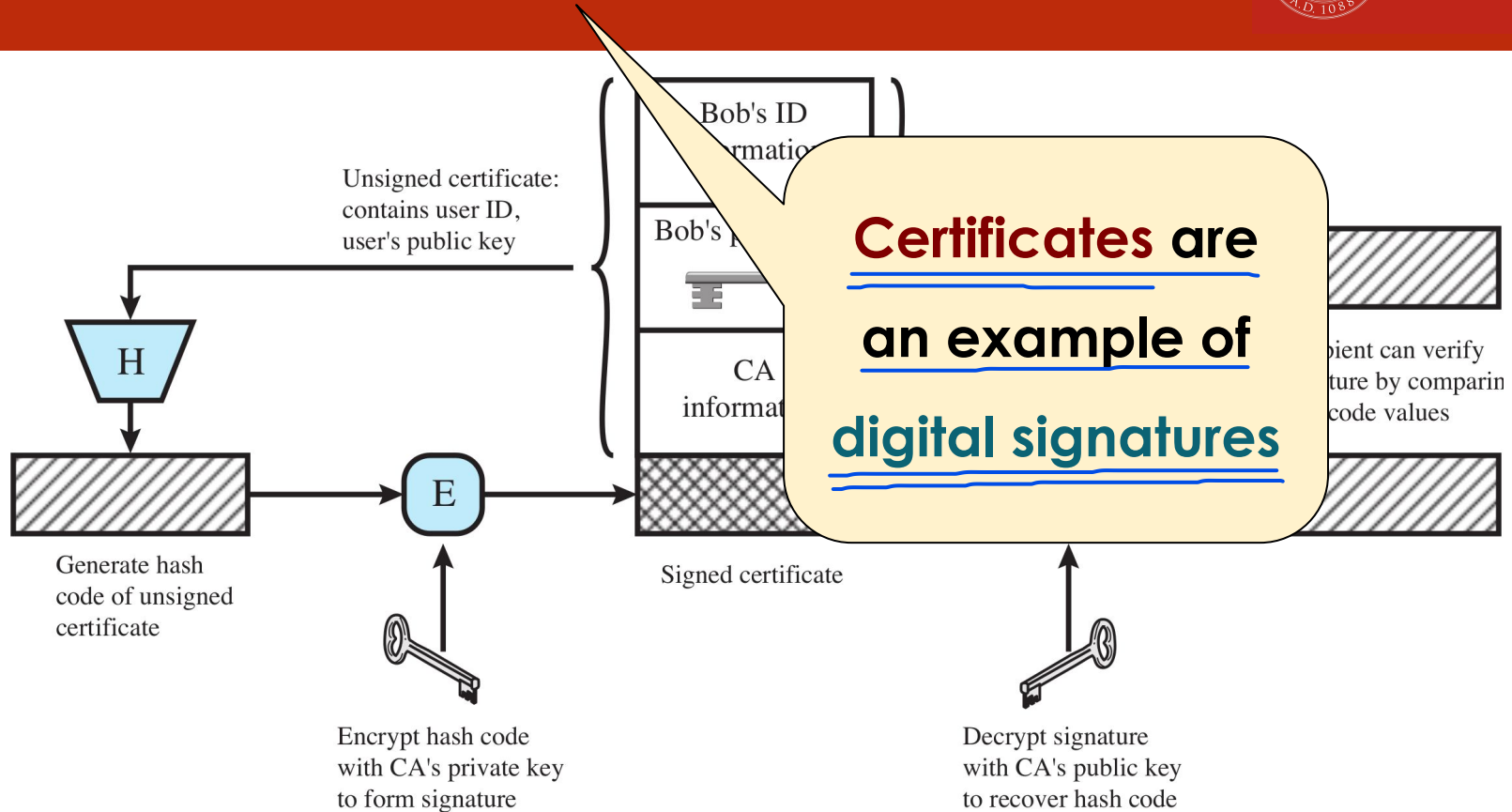
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- The **Digital Signature** is used for **authenticating** both **source** and **data integrity**
- Created by **encrypting hash code with private key**
- Does **not** provide **confidentiality**:
 - even in the case of **complete encryption** (e.g., not only the hash code)
 - the message is **safe from alteration** but **not eavesdropping** in caso di intercettazione

Public-Key Certificates



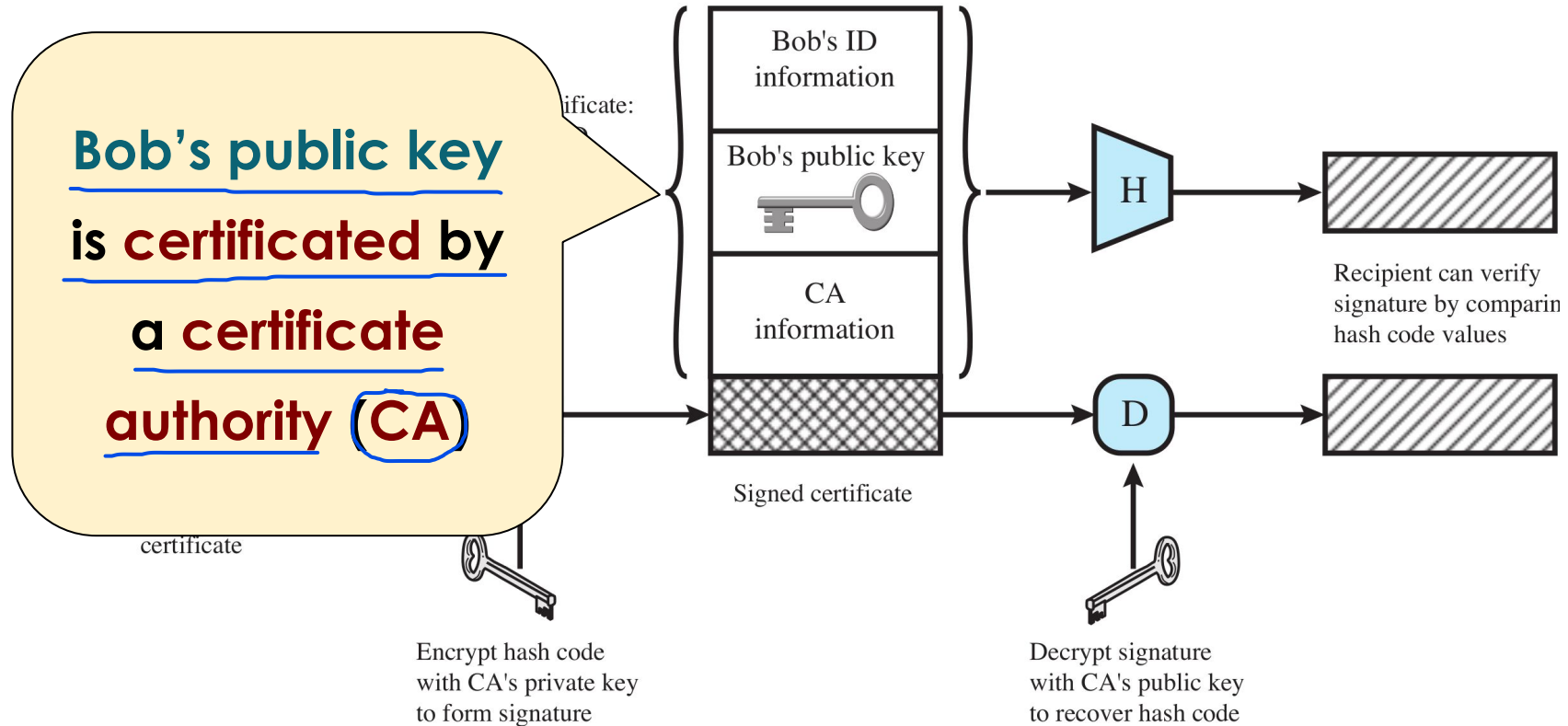
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



Public-Key Certificates



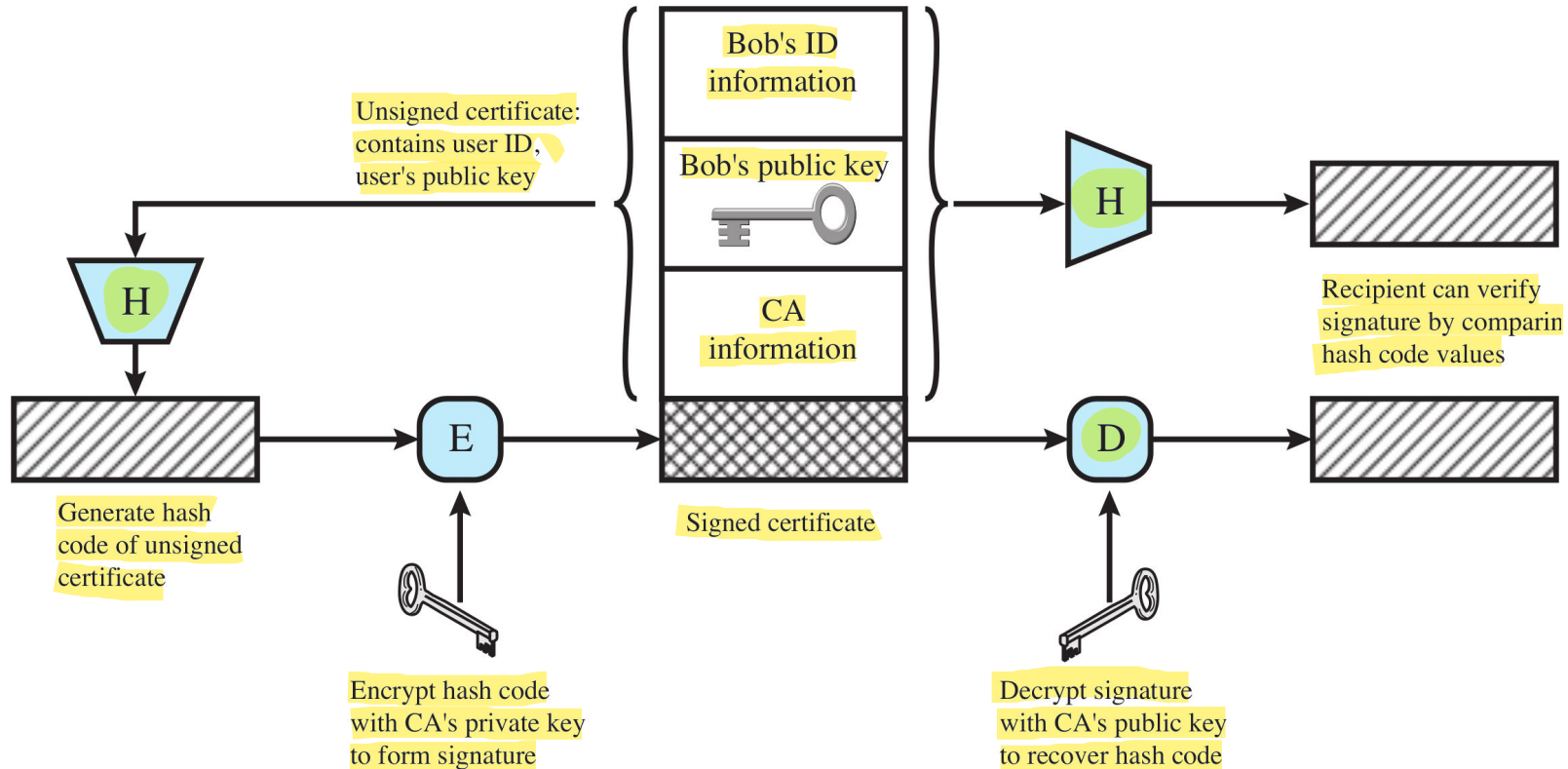
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



Public-Key Certificates



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



Public-Key Certificates

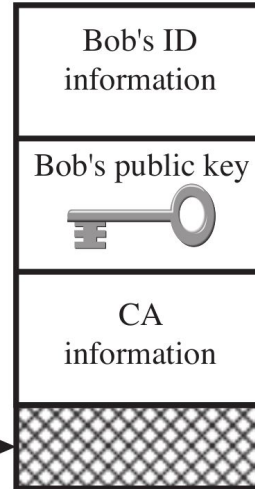


ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

**Step 1: an
unsigned
certificate is
prepared**

Generate hash
code of unsigned
certificate

Encrypt hash code
with CA's private key
to form signature



Signed certificate



Decrypt signature
with CA's public key
to recover hash code



Recipient can verify
signature by comparin
hash code values



Public-Key Certificates

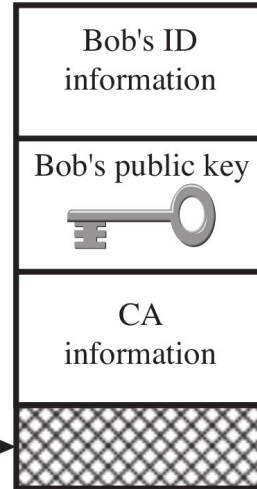


ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

**Step 1: an
unsigned
certificate is
prepared**

Generate hash
code of unsigned
certificate

Encrypt hash code
with CA's private key
to form signature

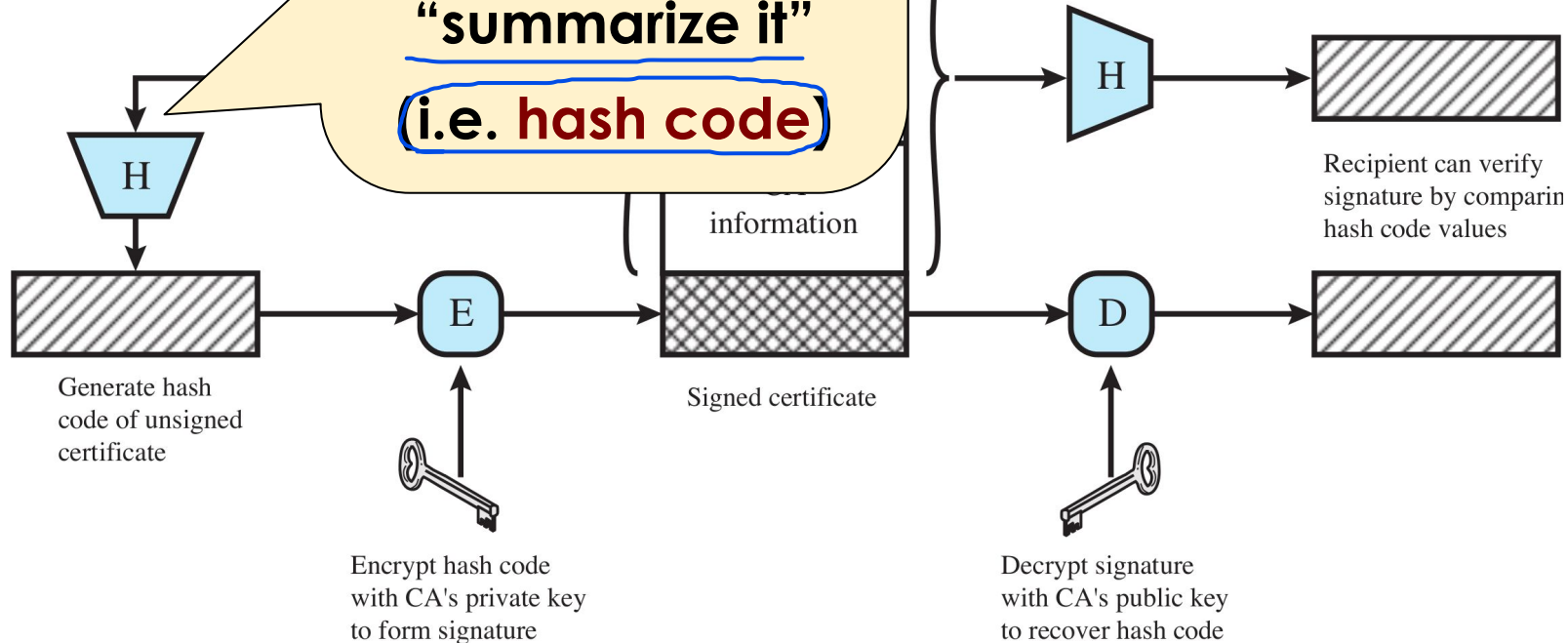


Signed certificate

**Since it is unsigned
then it is easy to
forge the certificate
i.e. (create a fake one)**

Decrypt signature
with CA's public key
to recover hash code

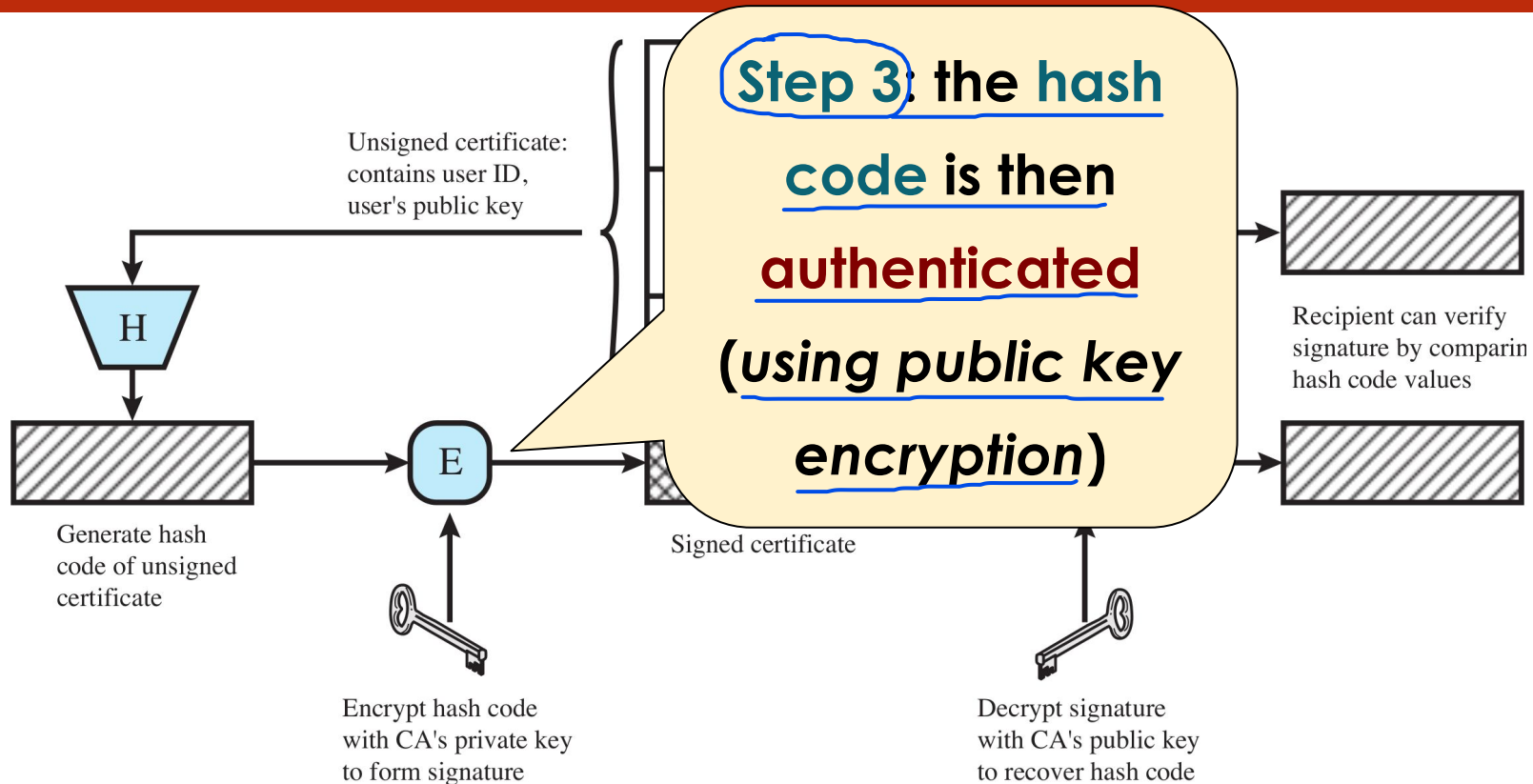
Step 2 to sign the
certificate it is
necessary to
“summarize it”
(i.e. hash code)



Public-Key Certificates



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



Public-Key Certificates



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

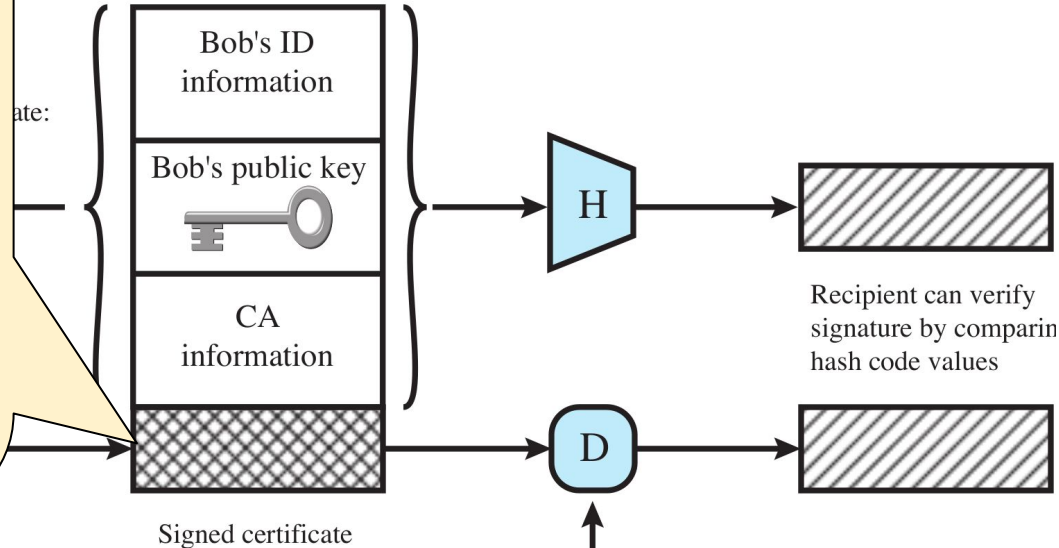
Step 4: ... and then

added to the
certificate that is
now signed (i.e. its
authenticity can be
verified)

Generate hash
code of unsigned
certificate



Encrypt hash code
with CA's private key
to form signature

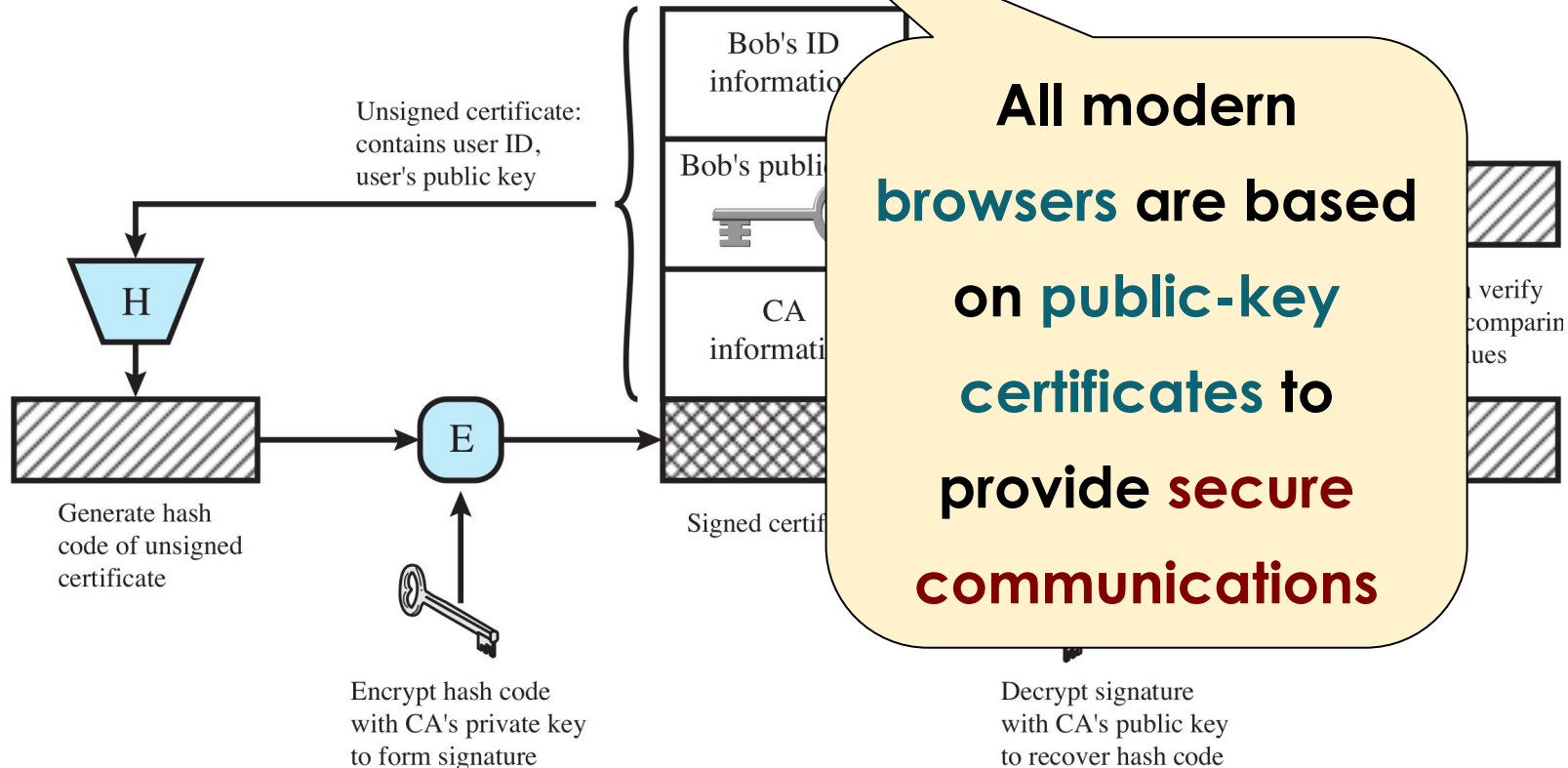


Decrypt signature
with CA's public key
to recover hash code

Public-Key Certificates



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



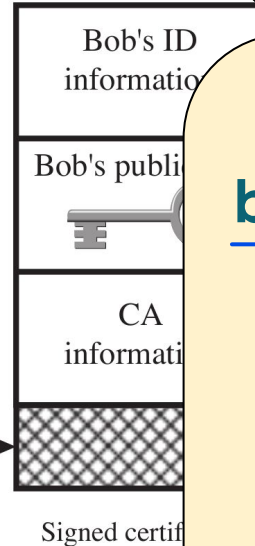
Public-Key Certificates



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

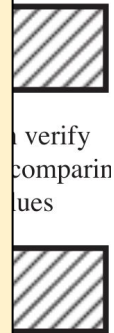
Using this method
the browser can
check that I'm
connected to the
proper website and
not to an impostor

with CA's private key
to form signature



All modern
browsers are based
on public-key
certificates to
provide secure
communications

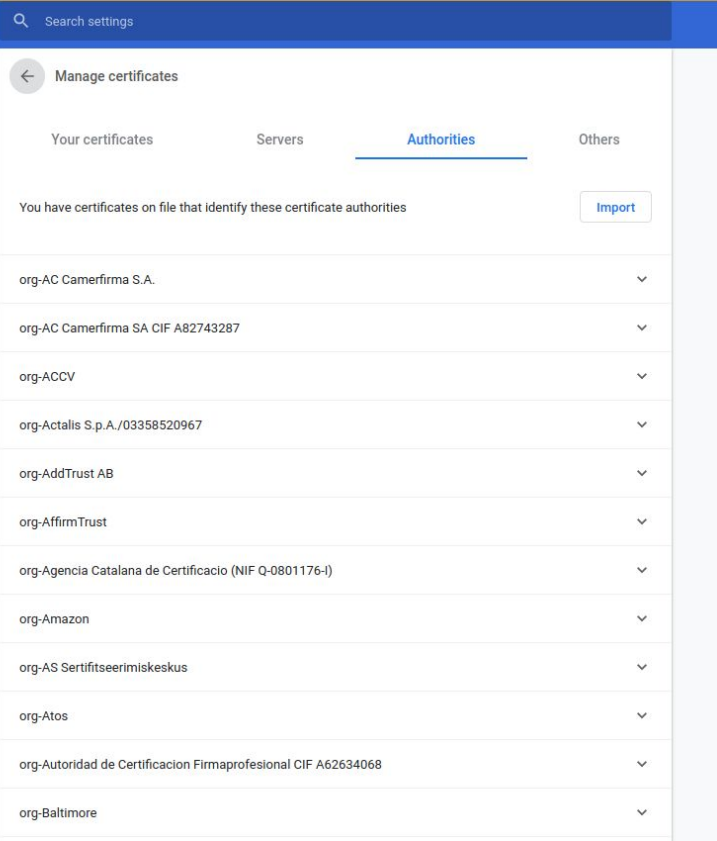
Decrypt signature
with CA's public key
to recover hash code



Web Browsers



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



In Google Chrome (like in any other Web Browser) there is a list of recognized certificates from trusted Certificate Authorities

That are the public keys (of the Certificate Authorities) used to verify the certificates provided by the Web sites (Amazon, Google...)



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Pharming Attacks



Http



Https

Example of **pharming attack**
to **HTTP** and **HTTPS**

How does it works?

Check it out!

T5



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

What's next?

Quantum Computing vs. Encryption



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- “A **quantum computer** is a computer that takes advantage of **quantum mechanical phenomena**” ([Wikipedia](#))
- Thanks to the “**quantum parallelism**” it can **speedup the execution of** **some** **specific computations** **(NOT all)**
- What is the expected effect of quantum computing on encryption?

Quantum Computing vs. Encryption



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- Most current **symmetric cryptographic algorithms** and **hash functions** are considered to be **relatively secure**
- In case, it is possible to switch to **longer keys** and **hashes**
- **This is not true for current public-key algorithms**
- A new generation of **post-quantum cryptography** algorithms is being developed, standardized and adopted in software (e.g. browsers)

Quantum Computing vs. Encryption



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



Barış Ege

PRINCIPAL SECURITY ANALYST,
KEYSIGHT DEVICE SECURITY
TESTING



#Cybersecurity

#Quantum Computer

#Quantum Computing

NETWORK VISIBILITY + SECURITY

Security Highlight: China's Quantum Leap, and Why RSA Isn't at Risk (Yet)

2024-10-28 | 4 min read

Recent headlines suggested that Chinese scientists have the potential to break military-grade RSA encryption using quantum computers, causing concerns over online data security. However, upon closer inspection, these claims do not yet signal the end of modern encryption. Here's what you need to know.

Understanding the breakthrough

The Chinese Journal of Computers recently **published a paper** in which researchers from Shanghai University factored a 22-bit integer using D-Wave's quantum annealing machine. The research showcases how quantum annealing can be leveraged to transform cryptographic attacks into solvable combinatorial optimization problems.

The team factored a 22-bit number, a far cry from the 2048-bit keys currently used for secure encryption. The difference between a 22-bit and a 2048-bit number is exponential, and this study, while a step forward, doesn't yet threaten RSA encryption as we know it.

La realtà dei fatti (Perché RSA non è a rischio oggi): L'articolo sottolinea che, sebbene l'esperimento sia un progresso tecnico interessante, la minaccia non è immediata per due motivi:

- Scala: I ricercatori hanno violato una chiave a 22 bit. Gli standard di sicurezza attuali utilizzano chiavi a 2048 bit. La differenza di complessità tra 22 e 2048 bit non è lineare, ma esponenziale; servirebbero computer quantistici enormemente più potenti di quelli attuali per gestire chiavi reali.

- Metodologia: L'approccio usato (quantum annealing) trasforma l'attacco in un problema di ottimizzazione, ma non è ancora dimostrato che questo metodo possa scalare fino a colpire i sistemi di sicurezza moderni.

Quantum Computing vs. Encryption



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



Barış Ege

NETWORK VISIBILITY + SECURITY

Security Highlight: China's Quantum Leap, and Why RSA Isn't at Risk (Yet)

2024-10-28 | 4 min read

Recent headlines suggested that Chinese scientists have the potential to break military-grade RSA encryption using quantum computers, causing concerns over online data security. However, upon closer inspection, these claims do not yet signal the end of modern encryption. Here's what you need to know.

Understanding the breakthrough

The Chinese Journal of Computers recently published a paper in which researchers from Shanghai University factored a 22-bit integer using D-Wave's quantum annealing machine. The research showcases how quantum annealing can be used to transform cryptographic attacks into solvable combinatorial optimization problems.

The team factored a 22-bit number, a far cry from the 2048-bit keys currently used for secure encryption. The difference between a 22-bit and a 2048-bit number is exponential, and this study, while a step forward, doesn't yet threaten RSA encryption as we know it.

Key-size:

22 vs. 2048 bits

#Quantum Computer

#Quantum Computing

Quantum Computing vs. Encryption



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

ars TECHNICA

AI BIZ & IT CARS CULTURE GAMING HEALTH POLICY SCIENCE SECURITY SPACE TECH **FORUM** | **SUBSCRIBE**

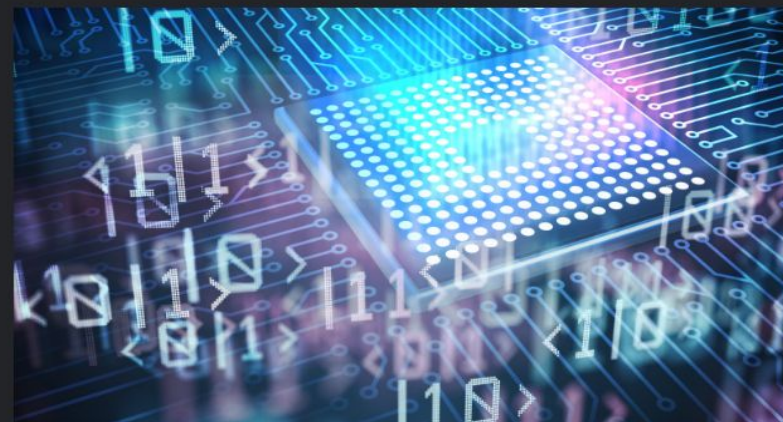


CRYPTOGRAPHICALLY RELEVANT QUANTUM COMPUTING

Quantum computers need vastly fewer resources than thought to break vital encryption

No, the sky isn't falling, but Q Day is coming, and it won't be as expensive as thought.

DAN GOODIN – MAR 31, 2026 8:25 PM | 39



→ Credit: vital



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Come funziona (Step-by-Step)

1. Fase di Cifratura (Il Mittente)

- Il mittente genera una chiave simmetrica temporanea con cui cifra il messaggio (anche di 1GB).
- Prende la Chiave Pubblica del destinatario e cifra solo la piccola chiave simmetrica temporanea (una diversa per ogni messaggio).
- Il "pacchetto" finale (Messaggio cifrato + Chiave cifrata) è la Digital Envelope.

2. Fase di Decifratura (Il Destinatario)

- Il destinatario riceve la busta.
- Usa la propria Chiave Privata per decifrare e recuperare la chiave simmetrica.
- Usa la chiave simmetrica appena recuperata per decifrare il messaggio vero e proprio.

Digital Envelopes

La Digital Envelope (Busta Digitale) è la soluzione definitiva al problema della distribuzione delle chiavi che abbiamo visto prima. È una tecnica ibrida che combina il meglio di due mondi:

- Symmetric (AES): È velocissima, ma ha il problema di come scambiare la chiave.

- Asymmetric (RSA/ECC): Risolve il problema dello scambio, ma è lentissima se devi cifrare file grandi (video, documenti pesanti).



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Digital Envelopes

- They are used to **protect a message** without ~~needing to arrange~~ ^{the sender and recipient} ~~having to agree in advance on a shared secret key~~ ~~beforehand, for sender and receiver, to have the same secret key~~

- It is equivalent ^{locked} ~~It equates to the same thing as a~~ **sealed envelope** containing an

unsigned letter

1. "Sealed envelope" (Busta sigillata) = Riservatezza

La crittografia asimmetrica garantisce che solo tu puoi aprire quella busta. In termini tecnici, significa che il mittente ha usato la tua Chiave Pubblica per "chiudere" il messaggio. Poiché solo tu possiedi la Chiave Privata corrispondente, sei l'unico al mondo che può leggere il contenuto. Risultato: Nessuno ha letto il messaggio durante il tragitto.

2. "Unsigned letter" (Lettera non firmata) = Mancanza di Autenticità

Una volta aperta la busta, trovi un foglio scritto al computer, senza firma e senza timbro. Il problema: Chiunque può aver messo quel foglio nella busta. Anche se la busta era indirizzata correttamente a te, non hai prove matematiche dell'identità del mittente. Potrebbe essere stato un tuo amico, ma potrebbe anche essere stato un impostore che ha semplicemente trovato la tua chiave pubblica e ti ha mandato un messaggio spacciandosi per qualcun altro.

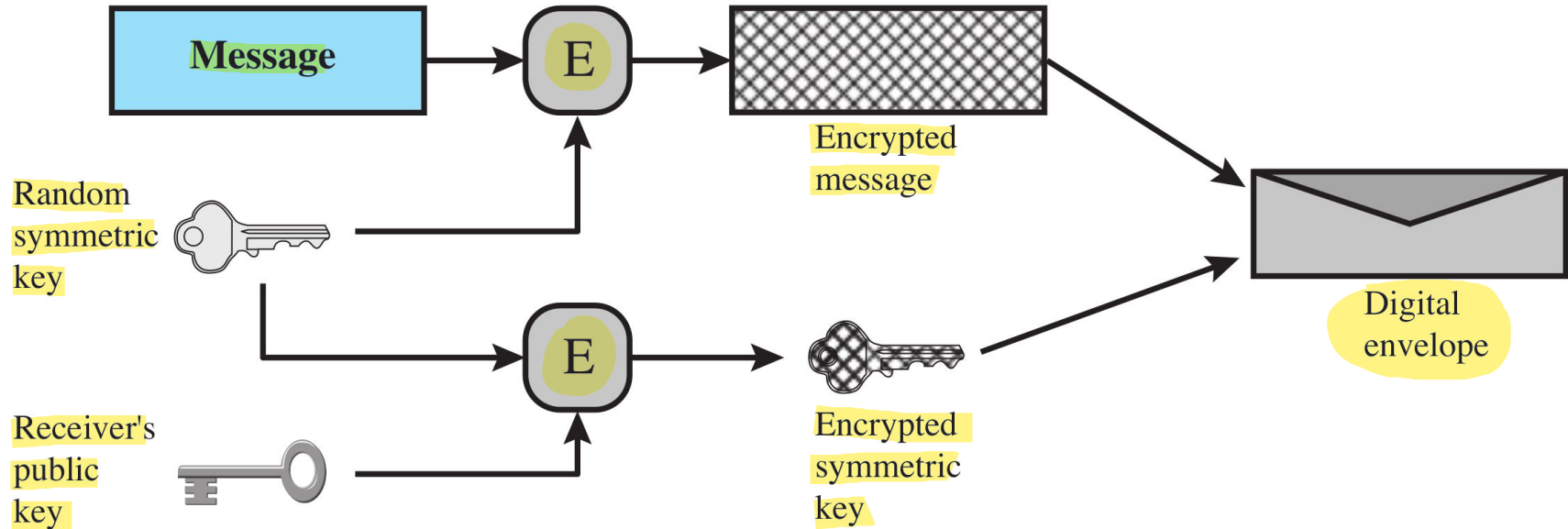
- They have some interesting characteristics

- Efficienza Massima: Non devi cifrare l'intero file con l'algoritmo di crittografia asimmetrica: con esso, cifri solo pochi bit di chiave.
- Nessun Accordo Precedente: Non serve che io e te ci incontriamo prima. Mi basta trovare la tua chiave pubblica e posso mandarti una digital envelope immediatamente.
- One-Time: Ogni volta che mando un messaggio, genero una chiave simmetrica diversa. Se un hacker riesce a rubare la chiave di un messaggio, gli altri rimangono comunque sicuri (Forward Secrecy).

Digital Envelopes: creation



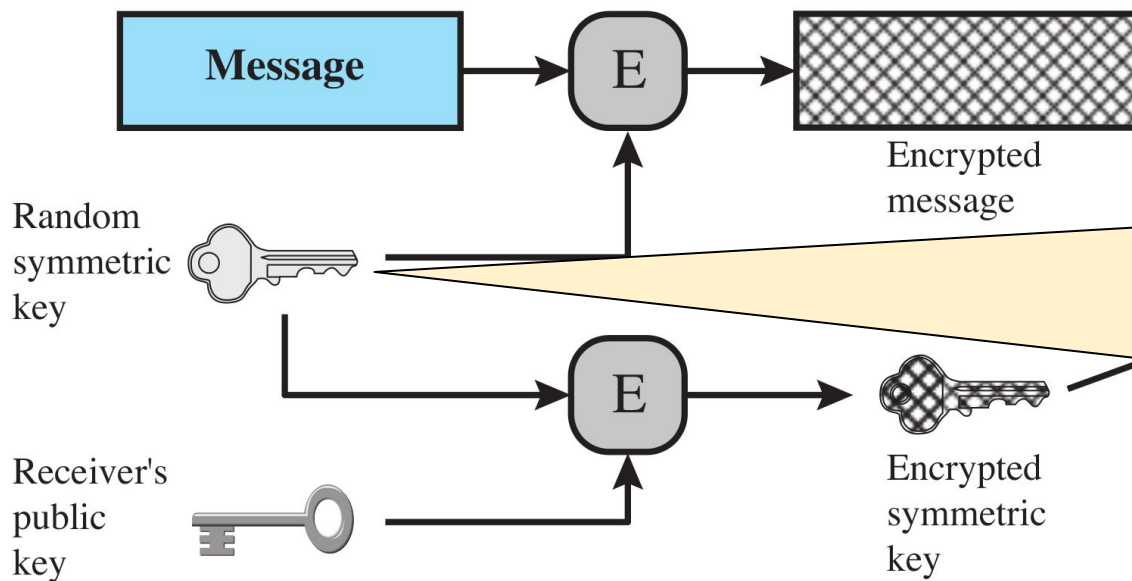
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



Digital Envelopes: creation



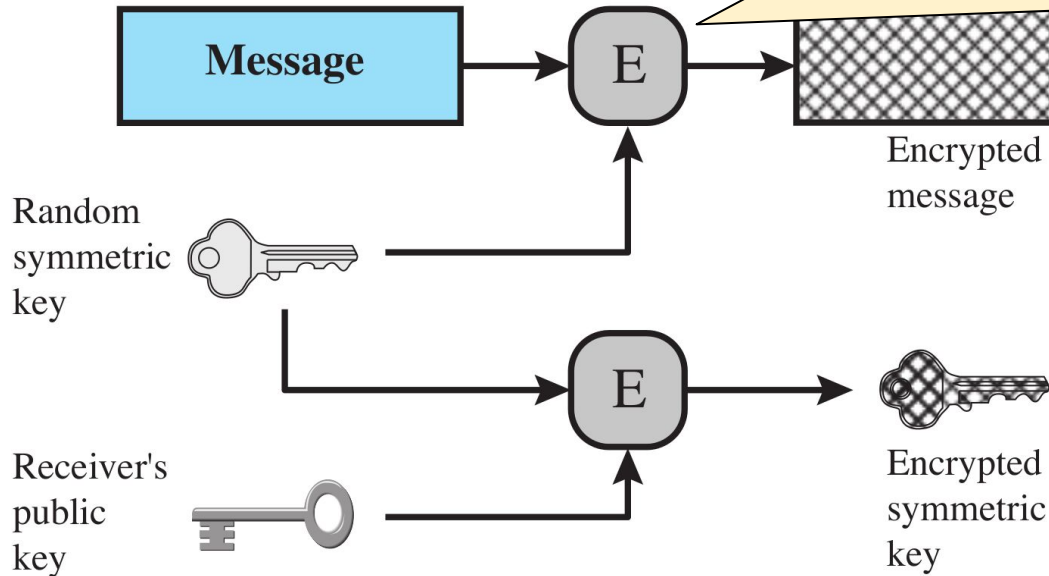
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



Immagina di usare la stessa chiave simmetrica per tutte le buste che mi mandi per un anno. Il rischio: Se un hacker riuscisse a rubare quella singola chiave oggi, potrebbe decifrare non solo i messaggi di oggi, ma anche tutti quelli che hai mandato negli ultimi 12 mesi e che lui ha intercettato e salvato. La protezione: Usando una chiave diversa per ogni busta, se l'hacker ne scopre una, potrà leggere solo quel singolo messaggio. Tutti gli altri rimangono al sicuro.

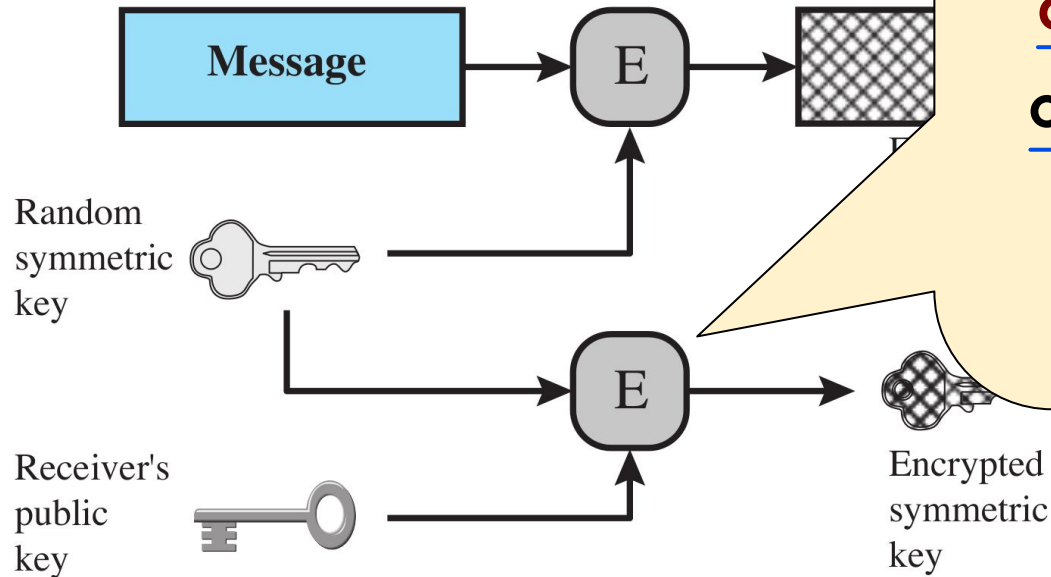
Every digital
envelope is
encrypted using
a new random
key

Digital Envelopes: creation



The digital envelope
is encrypted using a
symmetric algorithm
that is **much faster**
than **public key**
encryption

Digital Envelopes: creation



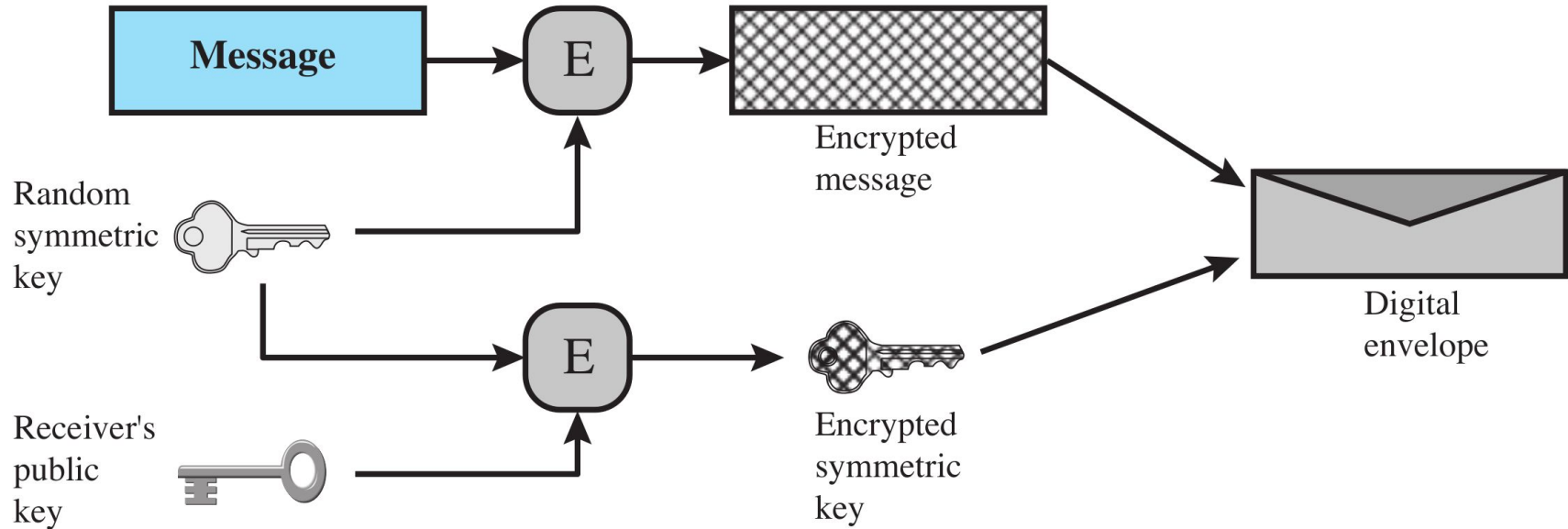
The public key encryption (that is quite slow) is used only for encrypting the random symmetric key

that has less bit size than the message itself

Digital Envelopes: creation



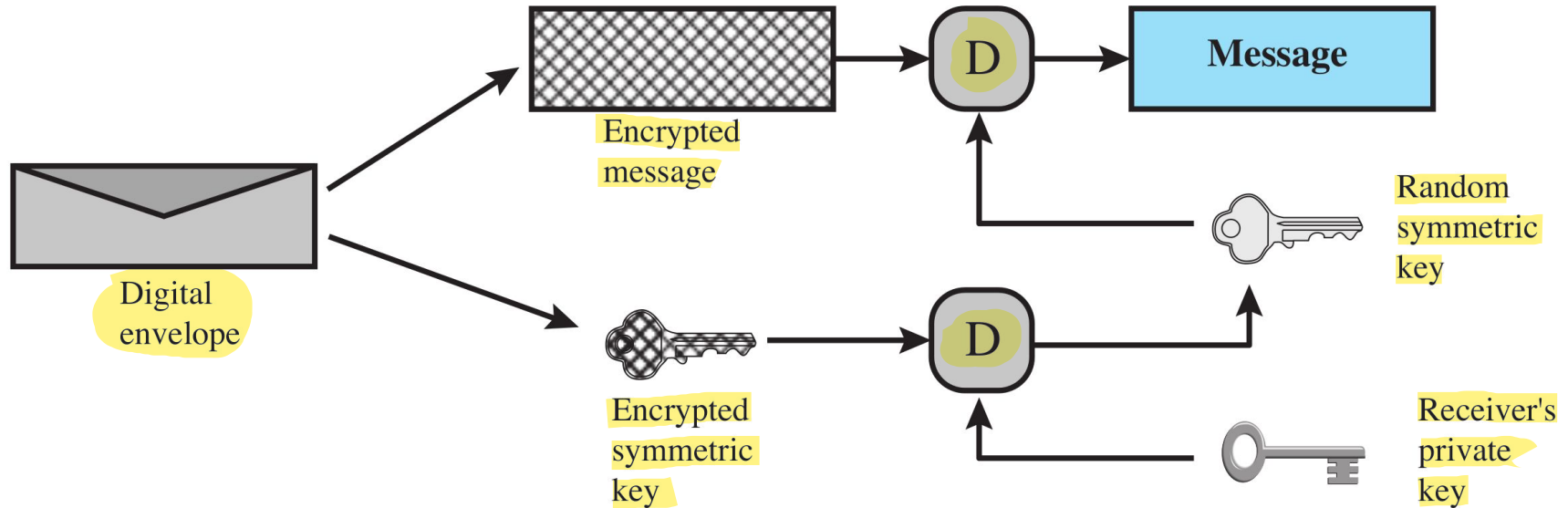
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



Digital Envelopes: opening



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA





ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Random Numbers

Random numbers (generation)



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Random numbers** are required by many different tasks:

- **generation of keys** for **public-key algorithms**

To create an RSA key pair, the computer must select two very large prime numbers. If these numbers are not truly random, an attacker could regenerate your private key.

- **stream key** for **symmetric stream cipher**

Stream ciphers (such as ChaCha20) transform a random number (nonce/IV) into an infinite string of bits to encrypt data. If the initial number repeats, the security is compromised.

- **symmetric key** to use as a **temporary session key** or to create a **digital envelope**

- **handshaking** to **prevent replay attacks**

During the "handshake" between two computers, random numbers called nonces (numbers used once) are exchanged. These ensure that each connection is unique.

- **session key** (i.e., randomly generated symmetric key)

As we have seen, every digital envelope requires a new symmetric key. This key must be unpredictable to prevent anyone from intercepting the envelope and opening it.

Random numbers (**generation**)



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Random numbers** are required by many different tasks:

- **generation of keys** for **public-key**
- **stream key** for **symmetric stream**
- **symmetric key** to use as a **temporary**
digital envelope
- **handshaking** to **prevent** **replay attacks**
- **session key** (i.e., randomly generated symmetric key)

An adversary is
repeating a previously
captured user response

Example: If a hacker records your login process and tries to "replay" it (replay attack), the server will reject it because the random session number from the previous session has already expired.

Random numbers (requirements)



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Randomness criteria:**

- 1 ○ **uniform distribution**: frequency of occurrence of each of the numbers should be **approximately the same**
- 2 ○ **independence**: **no one value** in the sequence can be **inferred from the others**

Random numbers (requirements)



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Randomness criteria:**
 - **uniform distribution:** frequency of occurrence of each of the numbers should be **approximately the same**
 - **independence:** no one value in the sequence can be **inferred from the others**

Unpredictability

Random numbers (requirements)



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

1

Each number is
statistically
independent of other
numbers in the
sequence

2

Opponent should not
be able to predict
future elements of the
sequence on the basis
of earlier elements

Unpredictability

Randomness vs. Pseudo-randomness



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Cryptographic applications** typically use **algorithms** for **random number generation**: algorithms are **deterministic** and therefore produce sequences of numbers that are **not statistically random**
- **Pseudo-random numbers** are:
 - sequences produced that **satisfy statistical randomness tests**
 - likely to be **predictable** →

Even though they pass statistical tests, pseudo-random numbers are generated by a mathematical algorithm. This means that:

- The Seed: It all starts with an initial number called a seed. If I know the algorithm and the seed you used, I can generate exactly the same sequence of numbers as you did.
- Determinism: There is no true chaos. It's like a very long book that's already been written: if you start reading from page 100, the sequence of words will seem random to someone who doesn't know the book, but anyone who has already read it knows exactly which word comes next.
- Security Risk in Cryptography: If a hacker manages to discover or guess the seed (perhaps because your computer generated it based solely on the system time), they can predict all your future "secret keys."

Randomness vs. Pseudo-randomness



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **True Random Number Generator (TRNG):**
 - uses a **nondeterministic source** to produce randomness
 - most operate by measuring **unpredictable natural processes**

(e.g., radiation, gas discharge, leaky capacitors)
 - **increasingly provided on modern processors** →

In the past, achieving “true” randomness required expensive external hardware. Today, manufacturers like Intel, AMD, and ARM integrate a small TRNG circuit directly into the CPU. Example: The RDRAND instruction in Intel processors queries a hardware circuit that uses thermal noise to generate extremely high-quality random bits at a very high speed.

Randomness vs. Pseudo-randomness



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **True Random Number Generator (TRNG):**

- uses a **nondeterministic source** to produce **randomness**
- most operate by measuring **unpredictable natural processes**
(e.g., radiation, gas discharge)
- **increasingly provided on modern processors**

Mitigation: Never Trust a Single Source

Since it is impossible to open up a processor and verify every single transistor while it is working, modern operating systems employ a defense strategy called Entropy Pooling. Instead of using the processor's random number as-is, the operating system mixes it with other sources:

- It takes the random number generated by the processor.
- It mixes them with hard drive noise, mouse movements, and network packet timings.
- It passes everything through a hash function (such as SHA-256).
- Result: Even if the processor were compromised and produced predictable numbers, the addition of other sources of chaos would disrupt the backdoor's pattern, making the final output secure again.

Backdoors?

When a hardware manufacturer (such as Intel or AMD) embeds a "black-box" random number generator directly into the processor, the cryptography community immediately faces a trust issue.

Randomness vs. Pseudo-Randomness

- **True Random Number Generators**

- uses a **nondeterministic** process
- most operate by measuring a physical phenomenon (e.g., radiation, gas discharge, thermal noise, capacitors)
- increasingly provided on **modern processors**



In CySec, nulla è "sicuro" in assoluto, ma tutto è relativo al tuo Threat Model (Modello di Minaccia). Affermare che esista una backdoor nel generatore hardware (TRNG) del tuo processore è un'ipotesi valida, ma la domanda che devi farti è: chi è il mio avversario?

Backdoors? It is
possible but
carefully consider
your threat model

Do you trust your
CPU?

- increasingly provided on **modern processors**

la frase "considera il tuo Threat Model" significa che non esiste una soluzione perfetta per tutti. Per il 99% delle persone, il TRNG hardware è un miracolo di tecnologia che rende internet più sicuro. Per il restante 1%, è un potenziale cavallo di Troia.



Randomness (NOT MANDATORY)



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- If you are interested in **how random numbers generation is managed** in the new versions of the **Linux kernel**: “Uniting the Linux random-number devices”
- If you are interested in a (potential) **backdoor in an encryption standard** (not related to random numbers generation): “Dual Elliptic Curve Deterministic Random Bit Generator”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

CyberSecurity

Gabriele D'Angelo

<g.dangelo@unibo.it>

<https://www.unibo.it/sitoweb/g.dangelo/en>

www.unibo.it

Attributions and Notes



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- Most of the slides are from: “**Computer Security: Principles and Practice**”, 4th edition, W. Stallings and Lawrie Brown
- Some others are from: Dr. **Mehmet H. Gunes** (Department of Computer Science & Engineering - University of Nevada, Reno)
- Some others are from: **Mario Cagalj**, Ph.D. (Faculty of Electrical Engineering, Mechanical Engineering and Naval Architecture (FESB), University of Split)
- I wish to **thank all of them**, **any errors that remain are my sole responsibility**